Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software. PDF

Robert C. Martin

Más Libros Gratis
Escanea para descargar
Escúchalo

Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software.

Principios esenciales para una arquitectura y diseño de software profesional.

Escrito por Bookey

Consulta más sobre el resumen de Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software.

Escuchar Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software. Audiolibro

Más Libros Gratis

Escanea para descargar

Escúchalo

Sobre el libro

En "Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software", el renombrado experto en software Robert C. "Uncle Bob" Martin construye sobre los cimientos establecidos en sus aclamados libros, "El programador limpio" y "Código limpio", para elevar el profesionalismo y la disciplina en la arquitectura y el diseño de aplicaciones. A diferencia de otros textos que simplemente presentan opciones sin orientación, Martin ofrece recomendaciones claras sobre las elecciones arquitectónicas esenciales y explica su importancia para el éxito. Aborda preguntas críticas sobre las estructuras de alto nivel para diversas aplicaciones, los principios fundamentales de la arquitectura y el diseño de software, el verdadero papel de un arquitecto y estrategias para evitar errores de diseño. Este libro es un recurso invaluable para arquitectos de software, analistas de sistemas, diseñadores y programadores aspirantes que deseen mejorar su comprensión e impacto en el campo.

Más Libros Gratis
Escanea para descargar
Escúchalo

Sobre el autor

Robert C. Martin, conocido como "Tío Bob", es una figura sumamente influyente en la comunidad de desarrollo de software, famoso por sus contribuciones a los principios y prácticas de la ingeniería de software. Con más de cinco décadas de experiencia en el campo, es un pionero de la metodología ágil y coautor del Manifiesto Ágil, que ha moldeado el desarrollo moderno de software. Martin también es autor de varios textos fundamentales, incluyendo "Código Limpio" y "El Programador Limpio", que enfatizan la importancia de escribir código mantenible, eficiente y ético. Su trabajo promueve la idea de Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software, abogando por un enfoque estructurado en el diseño de software que mejora la adaptabilidad y la longevidad, lo que lo convierte en un conferencista, consultor e instructor muy demandado en el ámbito de la artesanía del software.

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

Escanear para descargar

Prueba la aplicación Bookey para leer más de 1000 resúmenes de los mejores libros del mundo

Desbloquea de 1000+ títulos, 80+ temas

Nuevos títulos añadidos cada semana

Brand | Liderazgo & Colaboración | Gestión del tiempo | Relaciones & Comunicación | Kno

ategia Empresarial | Creatividad | Memorias | Dinero e Inversiones | Conózcase a sí mismo

nprendimiento | Historia del mundo | Comunicación entre Padres e Hijos | Autocuidado | M

Perspectivas de los mejores libros del mundo

Prueba gratuita con Bookey

Lista de contenido del resumen

Capítulo 1 : ¿Qué es el diseño y la arquitectura?

Capítulo 2 : Una Historia de Dos Valores

Capítulo 3 : Visión General de los Paradigmas

Capítulo 4 : Programación Estructurada

Capítulo 5 : Programación Orientada a Objetos

Capítulo 6 : Programación Funcional

Capítulo 7 : SRP: El Principio de Responsabilidad Única

Capítulo 8 : OCP: El Principio Abierto-Cerrado

Capítulo 9 : LSP: El Principio de Sustitución de Liskov

Capítulo 10 : ISP: El Principio de Segregación de Interfaces

Capítulo 11 : DIP: El Principio de Inversión de Dependencias

Capítulo 12 : Componentes

Capítulo 13 : Cohesión de Componentes

Capítulo 14 : Acoplamiento de Componentes

Capítulo 15 : ¿Qué es la arquitectura?

Más Libros Gratis

Escanea para descargar

Escúchalo

Capítulo 16 : Independencia

Capítulo 17 : Fronteras: Dibujando Líneas

Capítulo 18 : Anatomía de Límites

Capítulo 19 : Política y Nivel

Capítulo 20 : Reglas de Negocio

Capítulo 21 : Arquitectura Gritando

Capítulo 22 : La Arquitectura Limpia

Capítulo 23 : Presentadores y Objetos Humildes

Capítulo 24 : Fronteras Parciales

Capítulo 25 : Capas y Límites

Capítulo 26 : El Componente Principal

Capítulo 27 : Servicios: Grandes y Pequeños

Capítulo 28 : El Límite de las Pruebas

Capítulo 29 : Arquitectura Embebida Limpia

Capítulo 30 : La Base de Datos Es un Detalle

Capítulo 31 : La Web Es un Detalle

Más Libros Gratis

Escanea para descargar

Escúchalo

Capítulo 32 : Los Frameworks Son Detalles

Capítulo 33 : Estudio de Caso: Ventas de Videos

Capítulo 34 : El Capítulo Faltante

Capítulo 35 : Apéndice A Arqueología de la Arquitectura

Más Libros Gratis

Escanea para descargar

Escúchalo

Capítulo 1 Resumen : ¿Qué es el diseño y la arquitectura?

Sección	Resumen
¿QUÉ SON DISEÑO Y ARQUITECTURA?	El diseño y la arquitectura son esencialmente lo mismo en el software, trabajando juntos como un continuo de decisiones que dan forma al sistema.
¿CUÁL ES EL OBJETIVO?	El objetivo de un buen diseño de software es minimizar los recursos humanos necesarios para construir y mantener sistemas; un diseño de calidad conduce a un menor esfuerzo continuo.
ESTUDIO DE CASO	Un estudio de caso muestra a una empresa con un rápido crecimiento de personal enfrentando una disminución en la productividad, lo que indica caos donde la calidad sufre en favor de la velocidad.
LA FIRMA DE UN DESORDEN	Un desorden en el desarrollo surge de una codificación rápida y desorganizada, lo que lleva a una menor productividad y atrapa a los desarrolladores en la gestión de problemas.
LA PERSPECTIVA EJECUTIVA	Los ejecutivos están preocupados por el crecimiento de la nómina sin una funcionalidad correspondiente, pero las acciones específicas para abordar esto suelen ser poco claras.
¿QUÉ SALIÓ MAL?	La fábula de la Tortuga y la Liebre sirve como metáfora de la sobreconfianza de los desarrolladores y la creencia de que pueden corregir errores más tarde, perpetuando un ciclo de desorden.
CONCLUSIÓN	Los desarrolladores deben asumir la responsabilidad de la calidad del código, desafiando la noción de que el desorden equivale a velocidad, ya que la calidad es crucial para el éxito a largo plazo.

¿QUÉ ES EL DISEÑO Y LA ARQUITECTURA?

Más Libros Gratis
Escanea para descargar
Escúchalo

El diseño y la arquitectura a menudo generan confusión en la industria del software, pero en esencial no hay diferencia entre ellos. Tanto la arquitectura de alto nivel como el diseño de bajo nivel trabajan juntos como un todo, creando un continuo de decisiones que moldean el sistema.

¿CUÁL ES EL OBJETIVO?

El objetivo principal de un buen diseño de software es minimizar los recursos humanos necesarios para construir y mantener el sistema. Un diseño de calidad permite un bajo esfuerzo continuo, mientras que un mal diseño resulta en un esfuerzo creciente a lo largo del tiempo.

ESTUDIO DE CASO

Un estudio de caso ilustró los desafíos enfrentados por una empresa que experimentaba un rápido crecimiento en su personal de ingeniería y, sin embargo, una productividad en declive, medida por líneas de código y costos crecientes. Esto significa un proceso caótico donde se sacrifica la calidad por la velocidad, llevando a un ciclo perjudicial.

LA FIRMA DE UN CAOS

Más Libros Gratis
Escanea para descargar
Escúchalo

La situación describe un caos en el desarrollo de software donde la codificación rápida y desorganizada lleva a una productividad en declive. Los desarrolladores quedan atrapados gestionando las repercusiones de este desorden en lugar de innovar o añadir nuevas características.

LA PERSPECTIVA EJECUTIVA

Desde el punto de vista de un ejecutivo, la preocupación por el crecimiento de la nómina sin una funcionalidad correspondiente en los lanzamientos lleva a llamados por un cambio inmediato. Sin embargo, puede no estar claro qué pasos específicos seguir.

¿QUÉ SALIÓ MAL?

La fábula de Esopo de la Tortuga y la Liebre sirve como metáfora para la sobreconfianza que se observa en los desarrolladores modernos. Muchos creen que pueden arreglar los errores más tarde, cayendo en un ciclo de caos sin ver los efectos negativos en la productividad y la calidad del diseño.

CONCLUSIÓN

Más Libros Gratis
Escanea para descargar
Escúchalo

Para abordar la disminución de la productividad, los desarrolladores deben asumir la responsabilidad de la calidad en la codificación. El mito de que el desorden se traduce en velocidad debe ser desafiado, ya que la calidad es clave para el éxito a largo plazo en los proyectos de software. El libro busca aclarar y promover los principios de una arquitectura de software efectiva para construir sistemas con longevidad y un esfuerzo de mantenimiento mínimo.

Más Libros Gratis

Escanea para descargar

Escúchalo

Ejemplo

Punto clave:Calidad sobre Velocidad

Ejemplo:Imagina que estás programando una nueva función para un cliente con una fecha límite cercana. En tu prisa por entregar, optas por soluciones rápidas en lugar de un código limpio y mantenible. Inicialmente, el cliente está satisfecho y la función se lanza a tiempo. Sin embargo, a medida que pasan las semanas, luchas por añadir mejoras o corregir errores porque tu código se ha vuelto un lío, un caso clásico de sacrificar calidad por velocidad. Esta experiencia ilustra el punto clave: priorizar la calidad en el diseño es vital para la productividad a largo plazo y reduce la necesidad de un mantenimiento extenso, lo que, en última instancia, ahorra tiempo y recursos.

Más Libros Gratis

Escanea para descargar

Escúchalo

Pensamiento crítico

Punto clave:La relación entre el diseño de software y la arquitectura es crítica para el desarrollo efectivo de sistemas.

Interpretación crítica:Mientras que Robert C. Martin sostiene que el diseño y la arquitectura son fundamentalmente lo mismo y que un buen diseño minimiza la demanda de recursos, esta perspectiva puede pasar por alto las complejidades y variaciones en diferentes proyectos de software. El autor enfatiza la importancia de mantener la calidad para asegurar la longevidad de un sistema, pero existe el riesgo de simplificación excesiva, ya que cada sistema puede requerir diseños únicos adaptados a contextos específicos. Críticos, como Martin Fowler, sugieren que el diseño no puede ser una solución única para todos y que se deben reconocer las sutilezas de cada proyecto. Esto plantea interrogantes sobre la universalidad de los principios defendidos por Martin y anima a los lectores a considerar puntos de vista y metodologías alternativas.

Punto clave:Un buen diseño minimiza el esfuerzo de mantenimiento en el desarrollo de software.

Más Libros Gratis
Escanea para descargar
Escúchalo

Interpretación crítica:La afirmación de Martin de que un diseño de software efectivo reduce las necesidades de mantenimiento continuas es fundamental, ya que resalta la importancia de construir sistemas robustos. No obstante, esta perspectiva supone que todos los fallos ingenieriles pueden ser abordados únicamente a través del diseño. Esta visión puede merecer un examen crítico porque descuida la naturaleza dinámica de la tecnología y las capacidades del equipo, junto con factores externos potenciales como los cambios en los requisitos del usuario y las prioridades comerciales. Además, estudios, incluidos los de la Agile Alliance, sugieren que el desarrollo de software incluye la adaptabilidad y la mejora iterativa como elementos clave. Por lo tanto, mientras se reconoce el valor de un buen diseño, los lectores también deben considerar cómo la flexibilidad en el diseño y la arquitectura puede ser igualmente crucial para el éxito.

Capítulo 2 Resumen : Una Historia de Dos Valores

Sección	Resumen
Visión General	Los sistemas de software ofrecen dos valores: comportamiento y estructura. Los desarrolladores a menudo favorecen el comportamiento sobre la estructura, lo que puede reducir el valor global del software.
Comportamiento	El valor principal del software es el comportamiento, medido por la satisfacción de los interesados. Los desarrolladores se centran incorrectamente solo en implementar requisitos y corregir errores.
Arquitectura	El segundo valor es la flexibilidad de la arquitectura, que debería permitir cambios fáciles en respuesta a los requerimientos cambiantes de los interesados. La rigidez conduce a un aumento de costes y a la complejidad en futuras modificaciones.
El Valor Mayor	Hay una tensión entre la funcionalidad inmediata y la adaptabilidad a largo plazo. Los programas estáticos pueden volverse obsoletos ante cambios en los requisitos, mientras que los programas adaptables pueden evolucionar para satisfacer nuevas necesidades aunque en el momento no funcionen.
La Matriz de Eisenhower	El comportamiento a menudo se considera urgente pero no importante; la arquitectura es importante pero rara vez urgente. Este desajuste puede llevar a pasar por alto necesidades arquitectónicas críticas para características percibidas como urgentes.
Defender la Arquitectura	Los desarrolladores deben abogar por una arquitectura robusta frente a las presiones para priorizar características urgentes. Asegurar una buena arquitectura es crucial para evitar complicaciones futuras y costes asociados con modificaciones.

2 UNA HISTORIA DE DOS VALORES

Más Libros Gratis
Escanea para descargar
Escúchalo

Resumen

Cada sistema de software proporciona a los interesados dos valores: comportamiento y estructura. Los desarrolladores deben equilibrar ambos, pero a menudo priorizan el valor menos importante, lo que lleva a rendimientos decrecientes en el valor del software.

COMPORTAMIENTO

El valor principal del software es su comportamiento, representado por qué tan bien cumple con los requisitos de los interesados. A menudo, los desarrolladores creen erróneamente que su única responsabilidad es implementar estos requisitos y corregir cualquier error.

ARQUITECTURA

El segundo valor radica en la 'suavidad' del software, lo que significa que debe ser fácilmente modificable. Los cambios en los requisitos de los interesados no deberían ser difíciles de implementar. La discrepancia entre el alcance de los cambios y la forma necesita una arquitectura flexible para

Más Libros Gratis
Escanea para descargar
Escúchalo

evitar el aumento de costos, ya que la rigidez arquitectónica complica las modificaciones futuras.

EL VALOR MAYOR

Surge una pregunta crítica: ¿es más valioso un software que funcione o un software que se pueda ajustar fácilmente? Los gerentes de negocios a menudo priorizan la funcionalidad inmediata sobre la adaptabilidad a largo plazo, lo que puede llevar a problemas significativos. Un programa que funcione perfectamente pero que sea estático se vuelve inútil a medida que evolucionan los requisitos, mientras que un programa no funcional pero fácilmente modificable aún puede ajustarse para satisfacer necesidades cambiantes.

MATRIZ DE EISENHOWER

La matriz de Eisenhower diferencia entre tareas urgentes e importantes. El comportamiento cae en la categoría de urgente, pero no necesariamente importante, mientras que la arquitectura es importante pero rara vez urgente. Esta priorización a menudo lleva a descuidar la integridad arquitectónica en favor de características que no se pueden perder.

Más Libros Gratis
Escanea para descargar
Escúchalo

LUCHA POR LA ARQUITECTURA

Los desarrolladores de software deben abogar por la arquitectura, enfrentándose a luchas contra otros interesados que podrían priorizar características urgentes sobre la estructura subyacente. Los desarrolladores tienen un interés en garantizar que la arquitectura sea sólida para evitar complicaciones futuras y costos crecientes, convirtiéndolo en parte de su deber esencial. La falta de defensa de la arquitectura puede llevar a un sistema que se vuelve costoso y difícil de adaptar con el tiempo.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ejemplo

Punto clave:Equilibrar el comportamiento y la arquitectura en el desarrollo de software es crucial para el valor a largo plazo.

Ejemplo:Imagina que te han encargado desarrollar una nueva aplicación que debe adaptarse a las necesidades cambiantes de los usuarios en los próximos años. Mientras todos están emocionados por entregar una interfaz brillante y rica en funciones, tú entiendes que la arquitectura subyacente debe ser lo suficientemente flexible como para acomodar cambios futuros sin una reestructuración significativa. Recomiendas dedicar tiempo adicional a crear una estructura adaptable en lugar de apresurarte a satisfacer las solicitudes de funciones inmediatas. Al hacer esto, aseguras que, aunque los comportamientos inmediatos pueden no ser perfectos, el software sigue siendo valioso y relevante a largo plazo. Esta previsión al abogar por una arquitectura sólida, en última instancia, ahorrará tiempo y costos a medida que evolucionen los requisitos empresariales, llevando a una solución de software más adaptable y sostenible.

Más Libros Gratis

Escanea para descargar

Escúchalo

Pensamiento crítico

Punto clave:El Equilibrio Entre Comportamiento y Arquitectura

Interpretación crítica:Robert C. Martin enfatiza el equilibrio crítico entre el comportamiento del software y la arquitectura, instando a los desarrolladores a no descuidar la integridad arquitectónica en favor de la funcionalidad inmediata. Sin embargo, se podría argumentar que su perspectiva favorece en gran medida la adaptabilidad a largo plazo sin considerar adecuadamente las realidades empresariales, donde los objetivos a corto plazo a menudo dominan la toma de decisiones. Por ejemplo, en mercados dinámicos, las organizaciones pueden priorizar el despliegue rápido de funciones para mantener la competitividad, lo que puede chocar inherentemente con un enfoque arquitectónico más estructurado. Los críticos de la opinión de Martin podrían hacer referencia a los principios del Lean Startup, que abogan por iterar rápidamente en función de la retroalimentación de los usuarios, a veces a expensas de las consideraciones arquitectónicas.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 3 Resumen : Visión General de los Paradigmas

3 VISIÓN GENERAL DE LOS PARADIGMAS

El capítulo presenta una visión general de tres paradigmas de programación primarios: programación estructurada, programación orientada a objetos y programación funcional.

PROGRAMACIÓN ESTRUCTURADA

La programación estructurada fue popularizada por primera vez por Edsger W. Dijkstra en 1968, quien argumentó en contra del uso dañino de saltos sin control (instrucciones goto) en los programas. Dijkstra introdujo constructos como if/then/else y do/while/until para controlar el flujo del programa, enfatizando que la programación estructurada impone disciplina en la transferencia de control directo.

PROGRAMACIÓN ORIENTADA A OBJETOS

La programación orientada a objetos surgió en 1966, fundada

Más Libros Gratis
Escanea para descargar
Escúchalo

por Ole Johan Dahl y Kristen Nygaard. Su trabajo permitió que las variables locales de las funciones persistan después de la ejecución de la función, llevando al concepto de clases y métodos. Este paradigma promueve una cierta disciplina en relación con la transferencia de control indirecta.

PROGRAMACIÓN FUNCIONAL

La programación funcional se desarrolló antes que los dos primeros paradigmas, arraigada en la inve por Alonzo Church en 1936, que más tarde influyó en el lenguaje LISP. Un concepto clave en la programación funcional es la inmutabilidad, que significa que las variables no pueden cambiar su valor arbitrariamente, imponiendo así disciplina en la asignación.

ALIMENTO PARA EL PENSAMIENTO

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

Escanear para descargar

Por qué Bookey es una aplicación imprescindible para los amantes de los libros

Contenido de 30min
Cuanto más profunda y clara sea la interpretación que proporcionamos, mejor comprensión tendrás de cada título.

Formato de texto y audio
Absorbe conocimiento incluso en tiempo fragmentado.

Preguntas
Comprueba si has dominado lo que acabas de aprender.

Y más
Múltiples voces y fuentes, Mapa mental, Citas, Clips de ideas...

Prueba gratuita con Bookey

Capítulo 4 Resumen : Programación Estructurada

PROGRAMACIÓN ESTRUCTURADA

Introducción a Edsger Dijkstra

Edsger Wybe Dijkstra, una figura prominente en la computación, comenzó su carrera como el primer programador de los Países Bajos en 1952. Finalmente, cambió su enfoque de la física teórica a la programación debido a sus desafíos intelectuales y el potencial para desarrollar la programación como una disciplina reconocida.

El Desafío de la Programación

Dijkstra identificó que la programación es inherentemente compleja, con muchos detalles difíciles de gestionar para los humanos. Para combatir esto, propuso la aplicación de técnicas de prueba matemática para crear una jerarquía que los programadores pudieran utilizar similar a la de las

Más Libros Gratis
Escanea para descargar
Escúchalo

matemáticas. Descubrió que el uso indiscriminado de declaraciones 'goto' dificultaba descomponer las tareas de programación en componentes manejables.

Fundamentos de la Programación Estructurada

Dijkstra sugirió que ciertas estructuras de control, a saber, secuencia, selección e iteración (previamente establecidas por Böhm y Jacopini), permiten una metodología de desarrollo—programación estructurada—que enfatiza la claridad y el rigor. Este método permite la descomposición recursiva de tareas en unidades más simples y verificables.

La Declaración Go To Considerada Dañina

En un artículo de 1968, Dijkstra criticó el uso de declaraciones 'goto', lo que generó un extenso debate en la comunidad de programación. Con el tiempo, a medida que los lenguajes de programación evolucionaron, el uso de 'goto' desapareció, alineándose con los principios de programación estructurada.

Descomposición Funcional

Más Libros Gratis
Escanea para descargar
Escúchalo

La programación estructurada permite la descomposición funcional, donde los problemas complejos se dividen en funciones más simples y de alto nivel. Esto llevó a métodos como el análisis y diseño estructurado, promovidos por figuras como Ed Yourdon y Tom DeMarco.

No Hay Pruebas Formales

A pesar de su promesa teórica, la aplicación de pruebas formales en la programación ha quedado en gran parte sin cumplir. La mayoría de los programadores hoy en día no persiguen pruebas formales para la corrección del software, ya que no han demostrado ser prácticas en el desarrollo cotidiano.

Ciencia al Rescate

El enfoque matemático de Dijkstra contrastaba con el método científico, que enfatiza la falsabilidad en lugar de la demostrabilidad. Las pruebas, como señaló Dijkstra, solo pueden revelar errores pero no pueden probar la corrección, estableciendo así la importancia de la capacidad de prueba en el software.

Más Libros Gratis
Escanea para descargar
Escúchalo

Pruebas

Las pruebas son cruciales porque identifican errores pero no pueden confirmar la ausencia de fallos. Se alinean bien con el enfoque de la programación estructurada, que se centra en producir funciones pequeñas y verificables.

Conclusión

La programación estructurada sigue siendo relevante hoy en día al fomentar la creación de unidades de software verificables y manejables, reforzando así su importancia en las prácticas de programación modernas. Los principios de la programación estructurada continúan influyendo en cómo se aborda la arquitectura de software tanto a niveles micro como macro.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 5 Resumen : Programación Orientada a Objetos

PROGRAMACIÓN ORIENTADA A OBJETOS

Introducción a la POO

La Programación Orientada a Objetos (POO) se centra en la integración de datos y funciones. Aunque definiciones comunes como "modelar el mundo real" son imprecisas, los componentes centrales de la POO incluyen la encapsulación, la herencia y el polimorfismo.

Encapsulación

La encapsulación asegura que los datos y las funciones estén agrupados, manteniendo los datos ocultos del acceso externo, salvo a través de funciones específicas. Este concepto es común en los lenguajes de POO, pero también era alcanzable en lenguajes más antiguos como C. Los lenguajes modernos de POO, como C++ y Java, han introducido complejidades

Más Libros Gratis
Escanea para descargar
Escúchalo

que debilitan la encapsulación, ya que los detalles internos de las estructuras de datos a menudo deben hacerse conocidos durante la compilación, lo que conlleva a una exposición indirecta.

Herencia

La herencia permite la creación de nuevas clases basadas en clases existentes, facilitando la reutilización del código. Antes de los lenguajes de POO, los programadores de C emulaban la herencia a través de la gestión manual de estructuras. A pesar de presentar una sintaxis más conveniente, la POO no inventó el concepto, sino que lo hizo más fácil de implementar.

Polimorfismo

El polimorfismo permite que las funciones se comporten de manera diferente según el tipo de entrada. Aunque C no admitía el verdadero polimorfismo en un sentido de POO, el principio de utilizar punteros a funciones ha existido desde los primeros días de la programación. Sin embargo, los lenguajes de POO ofrecen una forma más segura y estructurada de lograr el polimorfismo, eliminando muchos

Más Libros Gratis
Escanea para descargar
Escúchalo

de los riesgos inherentes a la gestión manual de punteros a funciones.

El Poder del Polimorfismo

El polimorfismo permite que los sistemas trabajen con nuevos tipos sin problemas, sin requerir cambios en el código existente, apoyando una arquitectura de plugin que promueve la flexibilidad estructural. La POO permite una separación de políticas de alto nivel de implementaciones de bajo nivel, facilitando el despliegue y desarrollo independientes.

Inversión de Dependencias

El polimorfismo permite invertir las dependencias del código fuente, otorgando a los arquitectos control sobre la dirección de las dependencias dentro de un sistema. Este cambio crea una separación clara entre diferentes componentes, llevando a una capacidad de despliegue y desarrollo independiente.

Conclusión

Para los arquitectos de software, la POO se trata fundamentalmente de utilizar el polimorfismo para controlar

Más Libros Gratis
Escanea para descargar
Escúchalo

las dependencias del código fuente, permitiendo la creación de sistemas que pueden adaptarse y evolucionar sin reescrituras costosas. Este control es vital para formar una **Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software.**

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 6 Resumen : Programación Funcional

PROGRAMACIÓN FUNCIONAL

En muchos aspectos, los conceptos de la programación funcional son anteriores a la propia programación, con raíces en el cálculo » inventado por Alonzo Chur 1930.

CUADRADOS DE NÚMEROS ENTEROS

Para ilustrar la programación funcional, podemos comparar implementaciones que imprimen los cuadrados de los primeros 25 números enteros en diferentes lenguajes. En Java, se utiliza una variable de control del bucle mutable, mientras que Clojure demuestra la inmutabilidad sin cambiar los valores de las variables. La marcada diferencia resalta la característica crítica de la programación funcional: las variables en lenguajes funcionales no varían.

INMUTABILIDAD Y ARQUITECTURA

Más Libros Gratis
Escanea para descargar
Escúchalo

Los arquitectos deben preocuparse por la mutabilidad de las variables, ya que las variables mutables conducen a condiciones de carrera, bloqueos y otros problemas de concurrencia en las aplicaciones. Al asegurar la inmutabilidad, podemos minimizar estos problemas, ya que las aplicaciones concurrentes no enfrentan dificultades cuando ninguna variable es actualizada. Sin embargo, lograr una inmutabilidad completa es sutil y puede requerir compromisos en aplicaciones prácticas.

SEGREGACIÓN DE LA MUTABILIDAD

Un enfoque común es segregar las aplicaciones en componentes mutables e inmutables. Los componentes inmutables operan sin variables mutables, mientras que los componentes mutables manejan cambios de estado. Utilizar memoria transaccional puede ayudar a proteger las variables

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

App Store
Selección editorial

22k reseñas de 5 estrellas

Escanear para descargar

Retroalimentación Positiva

Alondra Navarrete

itas después de cada resumen
en a prueba mi comprensión,
cen que el proceso de
rtido y atractivo."

¡Fantástico! Beltrán Fuentes

Me sorprende la variedad de libros e idiomas que soporta Bookey. No es solo una aplicación, es una puerta de acceso al conocimiento global. Además, ganar puntos para la caridad es un gran plus!

Fi

Lo
re
co
pr

a Vásquez

nábito de
le y sus
o que el
todos.

¡Me encanta! Darian Rosales

Bookey me ofrece tiempo para repasar las partes importantes de un libro. También me da una idea suficiente de si debo o no comprar la versión completa del libro. ¡Es fácil de usar!

¡Ahorra tiempo!

Bookey es mi aplicación de
crecimiento intelectual. Lo
perspicaces y bellamente c
acceso a un mundo de con

icación increíble! Elvira Jiménez

ncantan los audiolibros pero no siempre tengo tiempo
escuchar el libro entero. ¡Bookey me permite obtener
esumen de los puntos destacados del libro que me
esa! ¡Qué gran concepto! ¡Muy recomendado!

Aplicación hermosa

Esta aplicación es un salvavidas para los a
los libros con agendas ocupadas. Los resu
precisos, y los mapas mentales ayudan a
que he aprendido. ¡Muy recomendable!

Prueba gratuita con Bookey

Capítulo 7 Resumen : SRP: El Principio de Responsabilidad Única

Sección	Resumen
Título	7SRP: EL PRINCIPIO DE RESPONSABILIDAD ÚNICA
Definición	El SRP establece que un módulo debe tener una, y solo una, razón para cambiar, enfocándose en un interesado o actor específico.
Síntomas de Violación del SRP	Síntoma 1: Duplicación Accidental - Un módulo contiene múltiples responsabilidades, lo que lleva a errores debido a cambios entrelazados de diferentes actores. Síntoma 2: Fusiones - Conflictos de código frecuentes durante la integración surgen cuando varios desarrolladores modifican clases compartidas sin adherirse al SRP.
Soluciones a la Violación del SRP	Separar funciones en diferentes clases o módulos basados en responsabilidades; usar el patrón Facade para delegar funciones mientras se mantiene el SRP.
Conclusión	El SRP es crucial para un diseño modular, mejorando la mantenibilidad, reduciendo errores y facilitando cambios en la arquitectura del software.

7SRP: EL PRINCIPIO DE RESPONSABILIDAD ÚNICA

De los principios SOLID, el Principio de Responsabilidad Única (SRP) a menudo se malinterpreta debido a su nombre, que podría implicar que cada módulo debe hacer solo una cosa. En realidad, el SRP establece que un módulo debe tener una, y solo una, razón para cambiar, que está relacionada con su responsabilidad hacia un interés específico o "actor". Un módulo debe ser cohesivo, lo que significa que sus funciones

Más Libros Gratis
Escanea para descargar
Escúchalo

y estructuras de datos deben servir a los requerimientos de un solo actor.

Síntomas de Violación del SRP

Síntoma 1: Duplicación Accidental

Un ejemplo de una violación del SRP se observa en una clase Empleado que contiene métodos con diferentes responsabilidades: `calcularPago()`, `reportarHoras()`, y `guardar()`. Cada método atiende a diferentes actores (contabilidad, recursos humanos y administradores de bases de datos, respectivamente). Combinar estos métodos en una sola clase puede llevar a efectos secundarios no deseados cuando los cambios de un actor afectan a otros, causando en última instancia errores y costosos fallos.

Síntoma 2: Mezclas

Las mezclas en el código fuente pueden volverse comunes cuando varios desarrolladores modifican una clase socavando el SRP. Por ejemplo, los cambios realizados por un equipo de recursos humanos pueden entrar en conflicto con los de los

Más Libros Gratis
Escanea para descargar
Escúchalo

administradores de bases de datos, lo que lleva a situaciones arriesgadas y complicaciones durante la integración del código.

Soluciones a la Violación del SRP

Para adherirse al SRP, se recomienda separar las funciones en diferentes clases o módulos según sus responsabilidades. Al aislar funciones (por ejemplo, a través de un objeto EmployeeData), se puede reducir la duplicación accidental de código. Un método eficaz es usar el patrón Facade, que permite que una clase sirva como delegado para otras clases, manteniendo las funciones cruciales mientras apoya el SRP.

Conclusión

En última instancia, el SRP enfatiza la importancia del diseño modular en la arquitectura de software. Conduce a una mejor mantenibilidad, reduce el riesgo de errores y crea una estructura clara que facilita un uso extenso y cambios en diferentes componentes a un nivel arquitectónico superior. Los principios del SRP se extienden a otros conceptos relacionados con el diseño de componentes y arquitectónico, los cuales se discutirán en capítulos posteriores.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ejemplo

Punto clave:La importancia de una única responsabilidad en el diseño de software

Ejemplo:Imagina que eres un desarrollador de software encargado de diseñar un sistema de procesamiento de pagos. En lugar de incorporar todas las funciones, como la gestión de cuentas de usuario, la generación de informes de transacciones y el manejo de reembolsos en una sola clase de Pago, las separas en clases distintas, como GestorDeCuentas, InformadorDeTransacciones y ManejadorDeReembolsos. Esto significa que cuando un contador solicita cambios en las funciones de informes, puedes adaptar el InformadorDeTransacciones sin arriesgar la funcionalidad de los usuarios, asegurando que tu código permanezca estable y sea más fácil de mantener.

Más Libros Gratis

Escanea para descargar

Escúchalo

Pensamiento crítico

Punto clave:Entender el SRP más allá de su interpretación simplista es crucial para un diseño de software efectivo.

Interpretación crítica:El Principio de Responsabilidad Única (SRP) a menudo se interpreta erróneamente como simplemente requerir que un módulo realice una tarea singular. La realidad es más matizada; el SRP enfatiza que un módulo solo debería tener una razón para cambiar, lo que se alinea con sus restricciones en relación con los interesados específicos. Esta interpretación puede impactar significativamente las decisiones de diseño, como se observa en el ejemplo de la clase Employee, donde los métodos sirven a actores distintos pero interconectados. Por lo tanto, mezclar múltiples responsabilidades en un solo módulo puede llevar a complicaciones y conflictos innecesarios, complicando el mantenimiento del código y estirando los límites de la modularidad. Sin embargo, aunque el SRP proporciona una guía valiosa para la arquitectura de software, es necesario tener cuidado al aceptarlo de manera acrítica. Perspectivas alternativas sugieren que un exceso de granularidad en el diseño podría llevar a

Más Libros Gratis
Escanea para descargar
Escúchalo

un número abrumador de clases, haciendo que los sistemas sean engorrosos. Literatura de apoyo, como 'Patterns of Enterprise Application Architecture' de Martin Fowler, explora esta tensión, abogando por principios de diseño equilibrados que pesen la cohesión contra la complejidad.

Capítulo 8 Resumen : OCP: El Principio Abierto-Cerrado

Sección	Resumen
OCP: El Principio Abierto-Cerrado	El Principio Abierto-Cerrado establece que el software debería estar abierto a la extensión pero cerrado a la modificación, enfatizando la necesidad de una buena arquitectura de software para evitar cambios extensos por extensiones simples.
Un Experimento Mental	Un ejemplo ilustra la necesidad de cambios mínimos al adaptar un sistema para formatos de salida adicionales, destacando la importancia del Principio de Responsabilidad Única y de las dependencias organizadas.
Arquitectura de Diseño	La separación de tareas como el cálculo de datos y el formato permite un código estructurado con mínimas interdependencias, utilizando dependencias unidireccionales para mantener la estabilidad entre componentes.
Control Direccional	Se aseguran las direcciones correctas de dependencia mediante el uso de interfaces como FinancialDataGateway, permitiendo que los componentes interactúen sin dependencias directas.
Ocultamiento de Información	El funcionamiento interno se oculta a través de interfaces, protegiendo a los componentes de dependencias innecesarias y alineándose con el Principio de Segregación de Interfaces.
Conclusión	El OCP conduce a una arquitectura que soporta extensiones fáciles con cambios mínimos, promoviendo una jerarquía de componentes y dependencias unidireccionales para la robustez y adaptabilidad.

8 OCP: El Principio Abierto-Cerrado

El Principio Abierto-Cerrado (OCP) establece que un artefacto de software debe estar abierto a la extensión pero cerrado a la modificación, permitiendo que se amplíe sin cambiar la base de código existente. Este principio destaca la importancia de la arquitectura de software, ya que una arquitectura deficiente que requiere cambios extensos para extensiones simples refleja un fallo en el diseño.

Más Libros Gratis
Escanea para descargar
Escúchalo

Un Experimento Mental

Considera un sistema que muestra un resumen financiero que debe adaptarse para crear un informe imprimible. Idealmente, se debería cambiar poco del código antiguo si la arquitectura del sistema está bien diseñada, enfatizando la separación de responsabilidades (Principio de Responsabilidad Única - SRP) y dependencias organizadas (Principio de Inversión de Dependencias).

Arquitectura de Diseño

Al separar tareas como calcular los datos reportados y formatearlos para diferentes salidas, el código puede estructurarse con mínimas interdependencias. Componentes como Controladores, Interactores, Bases de Datos, Presentadores y Vistas deben tener dependencias unidireccionales para asegurar que los componentes de alto nivel permanezcan inalterados ante cambios en niveles inferiores. El Interactor actúa como el núcleo, conteniendo las reglas de negocio de la aplicación, protegido de preocupaciones periféricas.

Control Direccional

Más Libros Gratis
Escanea para descargar
Escúchalo

La complejidad del diseño sirve para asegurar las direcciones correctas de las dependencias. El uso de interfaces como FinancialDataGateway y FinancialReportPresenter apoya este control direccional al permitir que los componentes interactúen sin dependencias directas.

Ocultamiento de Información

Ocultar el funcionamiento interno a través de interfaces protege a los componentes de dependencias innecesarias. Esto se alinea con el Principio de Segregación de Interfaces, asegurando que los componentes se concentren únicamente en lo que utilizan directamente.

Conclusión

El OCP promueve una arquitectura de sistema que facilita extensiones fáciles sin cambios significativos. Implica crear una jerarquía de componentes y establecer dependencias unidireccionales para minimizar el impacto de los cambios en niveles inferiores sobre estructuras de niveles superiores. El objetivo principal es una arquitectura robusta y extensible que siga siendo adaptable a lo largo del tiempo.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ejemplo

Punto clave:Abierto a la extensión, cerrado a la modificación

Ejemplo:Imagina que estás desarrollando una aplicación de comercio electrónico que necesita una función de recompensas de lealtad. En lugar de modificar directamente los procedimientos de compra existentes cada vez que introduces nuevas reglas de lealtad, creas una interfaz para integraciones de programas de lealtad, lo que te permite añadir nuevos sistemas de recompensas sin alterar el proceso de pago fundamental. Este enfoque no solo mantiene tu código existente intacto, sino que también se alinea con el Principio Abierto-Cerrado, permitiendo que tu aplicación evolucione de manera flexible mientras mantiene una funcionalidad robusta.

Más Libros Gratis
Escanea para descargar
Escúchalo

Pensamiento crítico

Punto clave:La viabilidad del Principio Abierto-Cerrado puede depender en gran medida de aplicaciones específicas.

Interpretación crítica:Si bien Robert C. Martin aboga por el Principio Abierto-Cerrado (OCP) como una buena práctica en el diseño de software, es esencial reconocer que su efectividad puede variar según el contexto y la implementación. No todos los proyectos pueden beneficiarse de una adhesión rígida al OCP, particularmente en sistemas más pequeños o menos complejos donde el esfuerzo de implementar una arquitectura así podría superar los beneficios propuestos. Los críticos argumentan que, cuando se lleva el OCP demasiado lejos, puede conducir a una sobreingeniería y a una complejidad innecesaria (S. Martin, 'Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software.'). Además, las metodologías ágiles promueven iteraciones rápidas que pueden requerir ajustes que contradicen los principios del OCP. Por lo tanto, aunque el OCP presenta un modelo aspiracional atractivo para un diseño robusto, es crucial sopesar su aplicabilidad cuidadosamente frente a

Más Libros Gratis
Escanea para descargar
Escúchalo

las limitaciones del mundo real y los requisitos específicos del proyecto. La literatura de apoyo, como "Refactoring: Improving the Design of Existing Code" de Martin Fowler, sugiere que a veces, las modificaciones directas al código existente pueden ser más prácticas que mantener una arquitectura que cumpla estrictamente con el OCP.

Capítulo 9 Resumen : LSP: El Principio de Sustitución de Liskov

Sección	Resumen
Definición de LSP	El Principio de Sustitución de Liskov (LSP) establece que un objeto del tipo S puede ser sustituido por un objeto del tipo T sin afectar las propiedades deseables del programa, confirmando que S es un subtipo de T.
Orientación sobre el Uso de la Herencia	Una ilustración del LSP se proporciona a través de la clase License y sus subclases, PersonalLicense y BusinessLicense, que pueden calcular tarifas de manera intercambiable sin alterar el comportamiento de una aplicación de facturación.
El Problema del Cuadrado/Rectángulo	Este problema ejemplifica una violación del LSP, donde el Cuadrado hereda incorrectamente del Rectángulo debido a sus dimensiones dependientes, causando inconsistencias cuando se espera una sustitución.
LSP y Arquitectura	Aunque inicialmente era una guía para la herencia, el LSP ahora se aplica al diseño de software, enfatizando la importancia de interfaces bien definidas que fomenten la sustituibilidad, especialmente en contextos como los servicios REST.
Ejemplo de Violación del LSP	Un ejemplo muestra un sistema de despacho de taxis donde las desviaciones de diferentes compañías de una interfaz REST consistente complican la arquitectura a través de lógica condicional adicional, comprometiendo la estabilidad y la seguridad.
Conclusión	El Principio de Sustitución de Liskov subraya la importancia de mantener la sustituibilidad para asegurar una arquitectura de sistema limpia y adaptable, ya que las violaciones pueden conducir a complejidad e ineficiencia.

9 LSP: EL PRINCIPIO DE SUSTITUCIÓN DE LISKOV

Definición de LSP

En 1988, Barbara Liskov definió el Principio de Sustitución de Liskov (LSP) de la siguiente manera: Si un objeto del tipo

Más Libros Gratis
Escanea para descargar
Escúchalo

S puede ser reemplazado por un objeto del tipo T sin alterar las propiedades deseables del programa, entonces S es un subtipo de T.

Guía para el Uso de la Herencia

Un ejemplo del LSP se ilustra con una clase llamada Licencia, con sus subclases LicenciaPersonal y LicenciaEmpresarial. Ambos subtipos implementan sus propios métodos de cálculo de tarifas, pero son intercambiables sin afectar el comportamiento de la aplicación de Facturación.

El Problema del Cuadrado/Rectángulo

Una violación clásica del LSP es el problema del cuadrado/rectángulo. En este caso, el Cuadrado no hereda correctamente del Rectángulo debido a la mutabilidad

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

Escanear para descargar

Leer, Compartir, Empoderar

Completa tu desafío de lectura, dona libros a los niños africanos.

El Concepto

BOOKS FOR AFRICA × ×

Esta actividad de donación de libros se está llevando a cabo junto con Books For Africa. Lanzamos este proyecto porque compartimos la misma creencia que BFA: Para muchos niños en África, el regalo de libros realmente es un regalo de esperanza.

La Regla

Gana 100 puntos → **Canjea un libro** → **Dona a África**

Tu aprendizaje no solo te brinda conocimiento sino que también te permite ganar puntos para causas benéficas. Por cada 100 puntos que ganes, se donará un libro a África.

Prueba gratuita con Bookey

Capítulo 10 Resumen : ISP: El Principio de Segregación de Interfaces

Sección	Resumen
Resumen del ISP	El Principio de Segregación de Interfaces aboga por que los clientes no dependan de métodos no utilizados, promoviendo la segregación de interfaces para mantener la relevancia.
Escenario de Ejemplo	Un ejemplo con una clase Java OPS ilustra que segregar las operaciones en interfaces separadas para los usuarios evita la recompilación innecesaria de métodos no utilizados.
ISP y Tipos de Lenguaje	Los lenguajes de tipo estático como Java enfrentan problemas con dependencias innecesarias, mientras que los lenguajes dinámicos como Ruby y Python manejan las dependencias en tiempo de ejecución, reduciendo el acoplamiento.
ISP y Arquitectura	Los arquitectos deben evitar dependencias en módulos con características innecesarias para prevenir ineficiencias y riesgos durante las readaptaciones debido a cambios no relacionados.
Conclusión	El ISP enfatiza la minimización de dependencias en el código y la arquitectura para mejorar la flexibilidad y evitar complicaciones, alineándose con principios arquitectónicos más amplios.

10 ISP: EL PRINCIPIO DE SEGREGACIÓN DE INTERFACES

Descripción general del ISP

El Principio de Segregación de Interfaces (ISP) sugiere que ningún cliente debe verse obligado a depender de métodos que no utiliza. Este principio promueve la idea de segregar interfaces para garantizar que los clientes solo interactúen con los métodos que son relevantes para ellos.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escenario de ejemplo

Considera una clase OPS en Java con múltiples métodos de operación (op1, op2, op3). Diferentes usuarios (Usuario1, Usuario2, Usuario3) interactúan solo con métodos específicos. Si el Usuario1 solo utiliza op1, los cambios en op2 o op3 obligarán innecesariamente a recompilar para el Usuario1. Al segregar estas operaciones en interfaces distintas por usuario (por ejemplo, U1Ops para el Usuario1), evitamos dependencias innecesarias.

ISP y tipos de lenguajes

El problema de las dependencias innecesarias es particularmente crítico en lenguajes de tipado estático como Java, donde deben incluirse las declaraciones. Los lenguajes dinámicos como Ruby y Python evitan este problema, ya que sus dependencias se infieren en tiempo de ejecución, lo que conduce a sistemas menos acoplados.

ISP y arquitectura

Más allá de las prácticas de codificación, el ISP tiene implicaciones arquitectónicas significativas. Los arquitectos

Más Libros Gratis
Escanea para descargar
Escúchalo

deben evitar depender de módulos que traigan "carga" innecesaria. Por ejemplo, si un sistema depende de un marco vinculado a una base de datos específica, cualquier cambio no relacionado en la base de datos puede obligar a reimplementaciones, lo cual es ineficiente y arriesgado.

Conclusión

La lección principal del ISP es minimizar las dependencias excesivas tanto en el código como en la arquitectura para mantener la flexibilidad y reducir complicaciones imprevistas. Esta perspectiva se alinea con consideraciones y principios arquitectónicos de nivel superior, como el Principio de Reutilización Común.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ejemplo

Punto clave:Entender e implementar el ISP puede mejorar significativamente la flexibilidad del sistema.

Ejemplo:Imagina que eres un desarrollador de software encargado de crear una aplicación para usuarios que interactúa con diversos servicios. Mientras codificas, te das cuenta de que tu aplicación incluye métodos para muchas características, incluyendo algunas que ciertos usuarios nunca utilizarán. Al adherirte al Principio de Segregación de Interfaces, tomas las medidas necesarias para crear interfaces más pequeñas y específicas para cada rol de usuario. De esta manera, cuando se realiza un cambio, digamos, en una característica utilizada solo por un grupo de usuarios, las interfaces del resto de tus usuarios permanecen intactas, evitando la necesidad de recompilaciones y nuevas implementaciones. Esta segregación proactiva ayuda a que tu aplicación se mantenga ágil y receptiva a los cambios, mejorando en última instancia la colaboración y reduciendo los posibles cambios disruptivos para tus clientes.

Más Libros Gratis
Escanea para descargar
Escúchalo

Pensamiento crítico

Punto clave:Cuestionando la absolutidad del ISP.

Interpretación crítica:Mientras que el Principio de Segregación de Interfaces (ISP) aboga por mínimas dependencias segregando interfaces para mejorar la flexibilidad, es necesario evaluar críticamente su aplicabilidad a lo largo de varios paradigmas de programación y escenarios contextuales. La afirmación de Martin de que los clientes solo deben depender de los métodos necesarios puede pasar por alto situaciones donde una interfaz más amplia podría agilizar los esfuerzos de desarrollo o mejorar la cohesión dentro de un módulo altamente integrado. Además, en ciertos sistemas, una excesiva segregación de interfaces podría llevar a una mayor fragmentación, complicando la comprensión de las relaciones entre los componentes. Es esencial balancear los principios del ISP con las necesidades prácticas de desarrollo, como se destaca en fuentes como "Patrones de Diseño" de Gamma et al. o "Diseño Guiado por el Dominio" de Eric Evans, que enfatizan la importancia del contexto en las decisiones arquitectónicas.

Más Libros Gratis

Escanea para descargar

Escúchalo

Capítulo 11 Resumen : DIP: El Principio de Inversión de Dependencias

Sección	Resumen
DIP: EL PRINCIPIO DE INVERSION DE DEPENDENCIAS	El Principio de Inversión de Dependencias (DIP) se centra en asegurar que las dependencias del código fuente se refieran a abstracciones en lugar de clases concretas, lo que promueve la adaptabilidad del sistema y reduce el riesgo de cambios en las implementaciones.
Abstracciones Estables	Las abstracciones estables son cruciales para arquitecturas estables. Los cambios en las interfaces abstractas a menudo conducen a cambios correspondientes en las implementaciones concretas. Las prácticas de diseño deben favorecer interfaces abstractas estables sobre clases concretas volátiles.
Guías Clave	- Evitar referencias a clases concretas volátiles; preferir abstracciones.- Tener precaución al derivar de clases concretas volátiles debido a las fuertes dependencias.- Definir funciones como abstractas para evitar dependencias no deseadas.- No hacer referencia explícita a componentes concretos volátiles.
Fábricas	Ajustarse al DIP utilizando el patrón de Fábrica para instanciar objetos concretos volátiles, como la Fábrica Abstracta, que abstrae el proceso de instanciación a través de una interfaz de fábrica de servicios.
Componentes Concretos	Los componentes concretos pueden contener algunas violaciones del DIP, pero deben ser mínimas y gestionadas de manera efectiva para mantener una arquitectura limpia.
Conclusión	El DIP es un principio clave en la estructuración de dependencias en sistemas de software, promoviendo arquitecturas más limpias y mantenibles que se revisitarán a lo largo del libro.

11 DIP: EL PRINCIPIO DE INVERSION DE DEPENDENCIAS

El Principio de Inversión de Dependencias (DIP) enfatiza sistemas flexibles asegurando que las dependencias del código fuente se centren únicamente en abstracciones, en lugar de clases concretas o implementaciones. En lenguajes de programación tanto estáticamente como dinámicamente tipados, las dependencias no deberían referirse a módulos

Más Libros Gratis
Escanea para descargar
Escúchalo

concretos, esto mejora la adaptabilidad del sistema y reduce el riesgo asociado con cambios en implementaciones concretas.

Abstracciones Estables

Las abstracciones estables forman la columna vertebral de arquitecturas estables. Los cambios en las interfaces abstractas a menudo llevan a la necesidad de cambios en las implementaciones concretas, mientras que los cambios en clases concretas típicamente no requieren modificaciones en las interfaces. Las buenas prácticas de diseño favorecen el movimiento hacia interfaces abstractas y estables, lo que puede involucrar varias prácticas de codificación específicas:
- Evitar referencias a clases concretas volátiles; preferir interfaces abstractas.
- Derivar de clases concretas volátiles debe abordarse con precaución, particularmente en lenguajes estáticamente tipados donde la herencia crea dependencias fuertes.
- Evitar anular funciones concretas para prevenir heredar dependencias no deseadas; en su lugar, definir funciones como abstractas y proporcionar diversas implementaciones.
- No hacer referencia explícita a componentes concretos y volátiles.

Más Libros Gratis
Escanea para descargar
Escúchalo

Fábricas

Para cumplir con el DIP, la instanciación de objetos concretos volátiles debe utilizar un patrón de Fábrica. Un ejemplo incluye la Fábrica Abstracta, donde la aplicación abstrae el proceso de instanciación a través de una interfaz de fábrica de servicio, previniendo así dependencias directas en las clases concretas subyacentes.

Componentes Concretos

Típicamente, los componentes concretos pueden aún contener algunas violaciones del DIP, pero deberían limitarse a un número reducido de ellas. Si bien es impráctico eliminar todas las dependencias concretas, deberían ser gestionadas de manera eficiente para mantener una
Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software.

Conclusión

El DIP es un principio fundamental que se repetirá a lo largo

Más Libros Gratis
Escanea para descargar
Escúchalo

de las discusiones arquitectónicas en este libro, estableciendo una guía clara para estructurar las dependencias en los sistemas de software, llevando a arquitecturas más limpias y mantenibles. A medida que avancemos, la comprensión del DIP dará forma a los límites arquitectónicos y mejorará la calidad general del diseño.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ejemplo

Punto clave:Enfócate en las abstracciones para crear sistemas adaptables.

Ejemplo:Imagina que estás programando una aplicación de carrito de compras. En lugar de acoplar fuertemente tu pasarela de pago a un procesador específico, como CreditCardProcessor o PayPalProcessor, creas una abstracción llamada PaymentProcessor. De esta manera, cuando quieras añadir nuevos métodos de pago como criptomonedas, simplemente implementas la interfaz PaymentProcessor sin cambiar el sistema existente. Esta flexibilidad facilita la adaptación a nuevas tendencias de pago sin necesidad de rehacer significativamente, mostrando la esencia del Principio de Inversión de Dependencias.

Más Libros Gratis
Escanea para descargar
Escúchalo

Pensamiento crítico

Punto clave:El énfasis excesivo en las abstracciones puede llevar a una arquitectura demasiado compleja.

Interpretación crítica:Mientras que Robert C. Martin sostiene que los sistemas de software deben priorizar la estabilidad y la flexibilidad a través del Principio de Inversión de Dependencias (DIP), es esencial considerar que una dependencia excesiva de las abstracciones puede crear un diseño enrevesado que confunda a los desarrolladores y obstaculice el mantenimiento en lugar de facilitarlo. Hay escenarios, como se expone en fuentes como 'Domain-Driven Design' de Eric Evans, donde enfoques pragmáticos hacia implementaciones concretas pueden facilitar ciclos de desarrollo más rápidos y comprensibles. En general, aunque el DIP promueve una Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software, su aplicación debe equilibrarse con la practicidad para evitar complejidades innecesarias.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 12 Resumen : Componentes

Sección	Resumen
Definición de Componentes	Los componentes son las unidades desplegables más pequeñas en un sistema, representadas por archivos como jar, gem o DLL, que funcionan como complementos desplegables de manera independiente.
Una Breve Historia de los Componentes	La programación inicialmente implicaba la especificación manual de la ubicación de la memoria, lo que llevó a desafíos que se resolvieron con binarios relocatables y cargadores inteligentes.
Relocalización y Enlazadores	Los binarios relocatables permitieron la compilación de código por separado y la resolución eficiente de referencias externas, pero condujeron a cuellos de botella, lo que llevó a un enfoque por fases para enlazar y cargar para mejorar la velocidad.
evolución a lo largo de las Décadas	El aumento de la eficiencia del hardware en los años 80 y 90 fomentó la arquitectura de complementos de componentes, facilitando el enlace rápido de componentes como archivos .jar y DLL.
Conclusión	La moderna arquitectura de complementos de componentes, que soporta el enlace dinámico en tiempo de ejecución, representa avances significativos en el diseño de software a lo largo de las décadas.

12 COMPONENTES

Definición de Componentes

Los componentes son las unidades desplegables más pequeñas en un sistema, variando según el lenguaje de programación, como archivos jar en Java, archivos gem en Ruby o DLL en .NET. Pueden compilarse en un ejecutable o agregarse a archivos de archivo como los archivos .war y pueden funcionar como complementos independientes.

Más Libros Gratis
Escanea para descargar
Escúchalo

Una Breve Historia de los Componentes

Inicialmente, los programadores de software especificaban manualmente las ubicaciones de memoria dentro de sus programas, lo que generaba desafíos debido al aumento de los límites de memoria y funcionalidad. La solución llegó con la introducción de binarios reubicables, que el cargador inteligente podía adaptar a diversas direcciones de memoria, permitiendo prácticas de programación más flexibles.

Reubicabilidad y Enlazadores

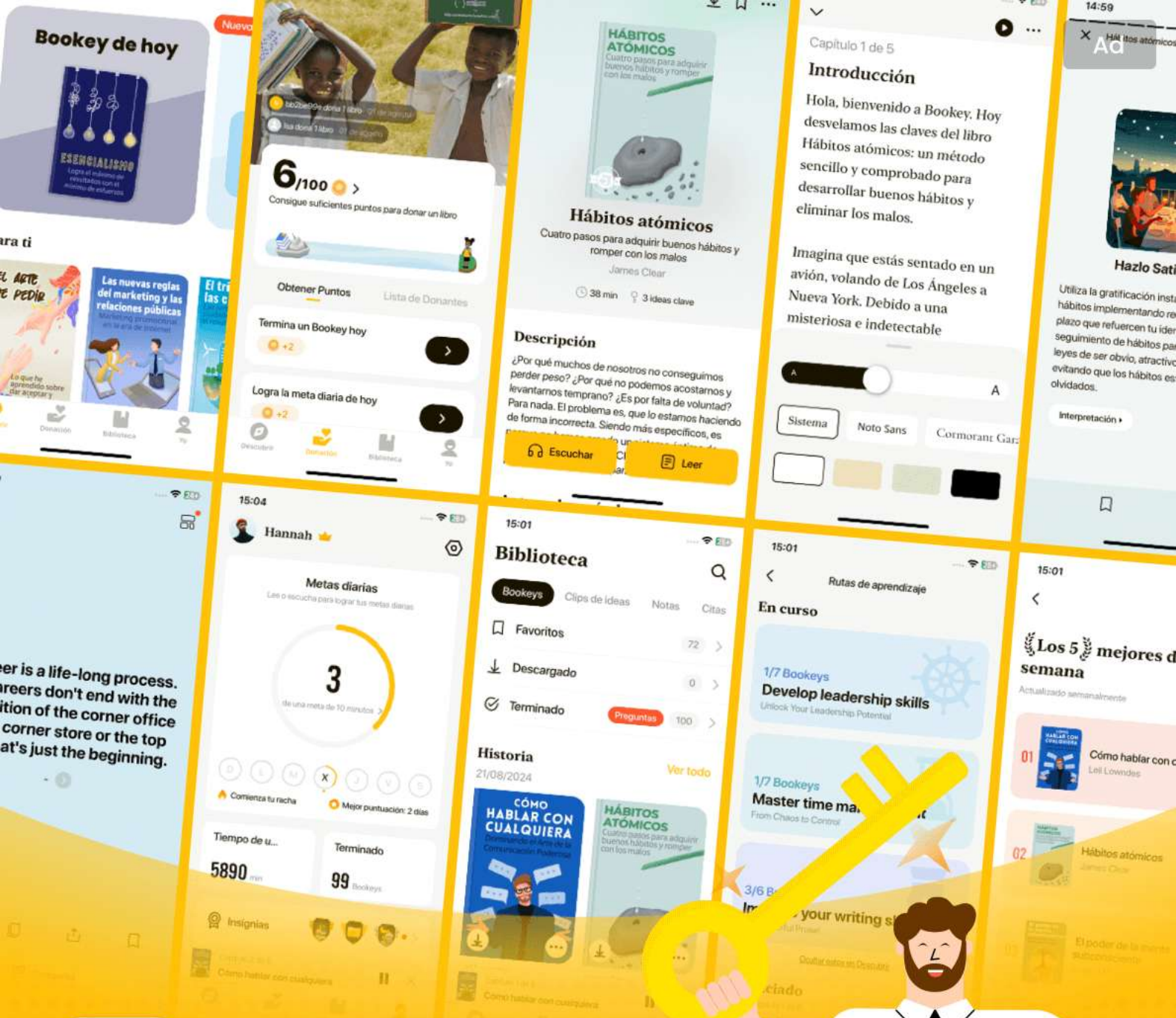
La llegada de binarios reubicables permitió a los programadores compilar código por separado y resolver referencias externas de manera más eficiente. Con la evolución de programas más grandes en los años 60 y 70, los enlazadores se convirtieron en un cuello de botella, lo que impulsó la separación del enlace y la carga en una fase

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo



Las mejores ideas del mundo desbloquean tu potencial

Prueba gratuita con Bookey

Escanear para descargar

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 13 Resumen : Cohesión de Componentes

COHESIÓN DE COMPONENTES

Decidir qué clases pertenecen a qué componentes es una decisión crucial informada por sólidos principios de ingeniería de software. Este capítulo discute tres principios clave de la cohesión de componentes:

-
REP
: El Principio de Equivalencia de Reutilización/Libertad

-
CCP
: El Principio de Cierre Común

-
CRP
: El Principio de Reutilización Común

EL PRINCIPIO DE EQUIVALENCIA DE REUTILIZACIÓN/LIBERTAD

Más Libros Gratis
Escanea para descargar
Escúchalo

Este principio establece que la unidad de reutilización es la unidad de liberación. Dado que la reutilización de software se ha vuelto común, es vital que los desarrolladores hagan seguimiento de los componentes a través de un proceso de liberación y asignen números de versión. Las clases y módulos dentro de un componente deben ser cohesivos y compartir un propósito común. Además, todos los elementos en un componente deben poder liberarse juntos bajo el mismo número de versión y documentación.

EL PRINCIPIO DE CIERRE COMÚN

Agrupa las clases que cambian juntas por las mismas razones. Este principio enfatiza que los componentes no deben tener múltiples razones para cambiar, similar al Principio de Responsabilidad Única (SRP) para clases. Al limitar los cambios a componentes individuales, se minimiza la necesidad de volver a desplegar múltiples componentes, lo que mejora la mantenibilidad.

SIMILITUD CON EL SRP

Tanto el CCP como el SRP coinciden en la idea de agrupar entidades relacionadas en función de la frecuencia de

Más Libros Gratis
Escanea para descargar
Escúchalo

cambio.

EL PRINCIPIO DE REUTILIZACIÓN COMÚN

El CRP dicta que las clases que se reutilizan juntas deben estar en el mismo componente, mientras que aquellas que no están fuertemente acopladas o dependientes no deberían agruparse. Este principio ayuda a prevenir dependencias innecesarias y asegura que los componentes sean lo más autónomos posible.

EL DIAGRAMA DE TENSIÓN PARA LA COHESIÓN DE COMPONENTES

Los principios de cohesión pueden entrar en conflicto. El REP y el CCP promueven componentes más grandes, mientras que el CRP aboga por componentes más pequeños. Un buen diseño arquitectónico busca navegar la tensión entre estos principios, adaptándose a medida que las necesidades del proyecto evolucionan con el tiempo.

CONCLUSIÓN

Los principios de cohesión de componentes ofrecen una

Más Libros Gratis
Escanea para descargar
Escúchalo

visión matizada de cómo agrupar efectivamente clases dentro de componentes. Equilibrar la reutilización y la mantenibilidad es un proceso dinámico, y la composición correcta probablemente cambie a medida que el enfoque del proyecto se desplace.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 14 Resumen : Acoplamiento de Componentes

14 ACOPLAMIENTO DE COMPONENTES

Los siguientes tres principios abordan las relaciones entre componentes, navegando la tensión entre la desarrollabilidad y el diseño lógico; esto implica gestionar fuerzas técnicas, políticas y volátiles.

EL PRINCIPIO DE DEPENDENCIAS ACÍCLICAS

- **Definición**: No permitir ciclos en el gráfico de dependencias de componentes.
- **Síndrome del Día Después**: Este fenómeno ocurre cuando múltiples desarrolladores cambian código compartido, causando inestabilidad y problemas de integración, especialmente en proyectos

Más Libros Gratis
Escanea para descargar
Escúchalo

grandes.

-

Soluciones

:

-

Compilación Semanal

: Una práctica donde los desarrolladores trabajan en copias privadas del código e integran cambios una vez a la semana, pero este enfoque puede volverse ineficiente con el tiempo.

-

Principio de Dependencias Acíclicas (ADP)

: Luchar por liberar componentes que puedan funcionar de manera independiente, reduciendo así las dependencias directas y los ciclos.

ELIMINANDO CICLOS DE DEPENDENCIA

Una solución para gestionar la escala del proyecto y la integración de manera eficiente es dividir el desarrollo en componentes liberables, permitiendo a los equipos trabajar de forma independiente e integrarse en pequeños incrementos.

DIAGRAMAS DE COMPONENTES

Más Libros Gratis
Escanea para descargar
Escúchalo

Los componentes se representan como un gráfico acíclico dirigido (DAG), asegurando que la gestión de dependencias sea sencilla.

EFECTO DE LOS CICLOS

- Crear dependencias cíclicas dificulta el lanzamiento y las pruebas, llevando a un acoplamiento estrecho donde los cambios en un componente requieren cambios en otro y complican la integración.

ROMPIENDO CICLOS

1. **Principio de Inversión de Dependencias (DIP)**
: Introducir interfaces para invertir las dependencias.
2. **Creación de Nuevos Componentes**
: Mover clases compartidas a un nuevo componente para eliminar ciclos.

DISEÑO DE ARRIBA HACIA ABAJO

Más Libros Gratis
Escanea para descargar
Escúchalo

El diseño de la estructura de componentes debe evolucionar con el crecimiento del sistema en lugar de ser dictado inicialmente. Esto es necesario para gestionar la volatilidad y los cambios de manera efectiva.

EL PRINCIPIO DE DEPENDENCIAS ESTABLES

Depender en la dirección de la estabilidad. El diseño no puede ser totalmente estático; la volatilidad es inherente pero debe ser gestionada. Los componentes volátiles no deben cargar a los estables con dependencias.

MEDICIÓN DE LA ESTABILIDAD

La estabilidad puede medirse a través de métricas como Fan-in, Fan-out e Inestabilidad (I), que indican cuántas dependencias existen y su dirección.

No TODOS LOS COMPONENTES DEBEN SER ESTABLES

Un sistema saludable tiene una mezcla de componentes estables e inestables donde se esperan cambios y son manejables, promoviendo una arquitectura flexible.

Más Libros Gratis
Escanea para descargar
Escúchalo

EL PRINCIPIO DE ABSTRACCIONES ESTABLES

Un componente debe ser tan abstracto como estable: los componentes estables deben ser abstractos y flexibles, mientras que los componentes inestables pueden contener implementaciones concretas.

SECUENCIA PRINCIPAL

Los componentes posicionados en la línea que conecta los máximos estables y abstractos (0, 1) y los máximos inestables y concretos (1, 0) representan un estado ideal, equilibrando abstracción y estabilidad.

CONCLUSIÓN

Este capítulo enfatiza la gestión del acoplamiento de componentes a través de principios y métricas sólidas, buscando mantener diseños eficientes y flexibles en la arquitectura de software que evoluciona con el desarrollo de la aplicación.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 15 Resumen : ¿Qué es la arquitectura?

¿QUÉ ES LA ARQUITECTURA?

El término "arquitectura" evoca un sentido de poder e importancia, especialmente en el desarrollo de software. Un arquitecto de software es principalmente un programador que participa activamente en tareas de programación mientras también guía los esfuerzos de diseño del equipo para mejorar la productividad. Una buena arquitectura de software implica dividir un sistema en componentes bien definidos y establecer una comunicación efectiva entre ellos. El objetivo principal es mantener el desarrollo, la implementación, la operación y el mantenimiento eficientes y flexibles, enfatizando la importancia de dejar opciones abiertas para futuros cambios.

DESARROLLO

Una arquitectura efectiva hace que un sistema de software sea fácil de desarrollar. Para equipos más pequeños, una

Más Libros Gratis
Escanea para descargar
Escúchalo

estructura flexible puede ser beneficiosa al principio, mientras que equipos más grandes requieren definiciones claras de componentes e interfaces estables para la productividad y el éxito.

IMPLEMENTACIÓN

Una implementación eficiente es crucial para la utilidad de un sistema de software. A menudo, las estrategias de implementación no se consideran al principio del desarrollo, lo que puede llevar a una arquitectura que complica la implementación. Las decisiones sobre la arquitectura deben incluir consideraciones para un proceso de implementación sin problemas.

OPERACIÓN

Si bien la arquitectura influye significativamente en el

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

Escanear para descargar

Prueba la aplicación Bookey para leer más de 1000 resúmenes de los mejores libros del mundo

Desbloquea de 1000+ títulos, 80+ temas

Nuevos títulos añadidos cada semana

Brand | Liderazgo & Colaboración | Gestión del tiempo | Relaciones & Comunicación | Kno

ategia Empresarial | Creatividad | Memorias | Dinero e Inversiones | Conózcase a sí mismo

nprendimiento | Historia del mundo | Comunicación entre Padres e Hijos | Autocuidado | M

Perspectivas de los mejores libros del mundo

Prueba gratuita con Bookey

Capítulo 16 Resumen : Independencia

INDEPENDENCIA

Resumen de los Objetivos de la Arquitectura

Una buena arquitectura debe soportar varios aspectos clave de un sistema:
- Casos de uso y soporte operativo.
- Facilidad de mantenimiento.
- Eficiencia en el desarrollo.
- Facilidad de despliegue.

Casos de Uso

Los arquitectos deben asegurarse de que la arquitectura esté alineada con los casos de uso previstos del sistema, haciendo que estos casos sean claros y prominentes. La arquitectura debe reflejar la funcionalidad del sistema, haciendo que los comportamientos sean fácilmente identificables sin requerir una búsqueda extensa.

Más Libros Gratis
Escanea para descargar
Escúchalo

Operación

La arquitectura necesita permitir que el sistema maneje métricas de rendimiento requeridas, tales como tiempos de respuesta y capacidad de procesamiento. Una arquitectura robusta debe tener en cuenta diversas configuraciones operativas, desde microservicios hasta ejecuciones monolíticas, permitiendo flexibilidad a medida que evolucionan las demandas.

Desarrollo

La Ley de Conway postula que el diseño de un sistema refleja la estructura de comunicación de un equipo. Por lo tanto, la arquitectura debe facilitar el desarrollo independiente de componentes, permitiendo que los equipos trabajen sin interferencias entre sí, aislando componentes de manera efectiva.

Despliegue

Un sistema bien arquitecturado debe soportar un despliegue sencillo e inmediato después de la construcción, evitando configuraciones complejas. Un adecuado aislamiento y

Más Libros Gratis
Escanea para descargar
Escúchalo

estructura de los componentes pueden aumentar significativamente la eficiencia del despliegue.

Mantener Opciones Abiertas

Una arquitectura exitosa encuentra un equilibrio entre sus diversas preocupaciones mientras mantiene la flexibilidad para adaptarse a cambios a lo largo de su ciclo de vida. Debe permitir modificaciones fáciles manteniendo las opciones abiertas durante el mayor tiempo posible, apoyada por principios arquitectónicos sólidos que respaldan la partición efectiva de los componentes.

Desacoplar Capas

La arquitectura debe gestionar por separado los componentes que cambian por diferentes razones y mantener las reglas de negocio distintas de los elementos de la interfaz de usuario. Este desacoplamiento estratégico asegura que el sistema pueda acomodar diversas demandas sin complicaciones innecesarias.

Desacoplar Casos de Uso

Más Libros Gratis
Escanea para descargar
Escúchalo

Los propios casos de uso también deben estar desacoplados para evitar que los cambios en uno afecten a otro. Esta separación permite un desarrollo independiente y una expansión más fácil de la funcionalidad.

Modo de Desacoplamiento

El desacoplamiento puede ocurrir en diferentes niveles: código fuente, despliegue o nivel de servicio. Cada modo tiene un propósito, y la elección de qué modo implementar puede evolucionar a lo largo de la vida del proyecto, acomodando necesidades cambiantes.

Desarrollo y Despliegue Independientes

Componentes bien desacoplados reducen la interferencia entre equipos y ofrecen flexibilidad en el despliegue. Los sistemas deberían permitir idealmente que los componentes sean “intercambiables en caliente”, mejorando la adaptabilidad sin afectar la integridad general del sistema.

Preocupaciones por la Duplicación

Si bien evitar la duplicación es vital, los arquitectos deben

Más Libros Gratis
Escanea para descargar
Escúchalo

diferenciar entre duplicación verdadera y duplicación accidental, asegurando que el sistema se mantenga manejable y flexible.

Niveles de Desacoplamiento

Hay tres niveles de desacoplamiento:

-
Nivel de Código Fuente:
Dependencias de módulo simples.

-
Nivel de Despliegue:
Unidades manejables que pueden ser desplegadas independientemente.

-
Nivel de Servicio:
Los componentes operan de manera independiente a través de una red.

Una buena arquitectura acomoda estas transiciones a lo largo del tiempo mientras protege la funcionalidad central de una volatilidad excesiva. Debe permitir una evolución significativa mientras se mantiene manejable, facilitando adaptaciones en el modo de despliegue según sea necesario.

Más Libros Gratis
Escanea para descargar
Escúchalo

Conclusión

Una buena arquitectura debe apoyar la naturaleza en evolución de los sistemas de software. La capacidad de cambiar el modo de desacoplamiento permite respuestas apropiadas a las variadas necesidades operativas, de desarrollo y de despliegue sin sacrificar la integridad o claridad del sistema.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 17 Resumen : Fronteras: Dibujando Líneas

FRONTERAS: DIBUJANDO LÍNEAS

La arquitectura de software implica crear fronteras que separan los elementos de software y restringen las interacciones entre ellos. Estas fronteras ayudan a retrasar decisiones que podrían influir prematuramente en la lógica de negocio central. El objetivo es minimizar los recursos requeridos para construir y mantener el sistema, evitando el acoplamiento con decisiones prematuras que no se relacionan con los requisitos del negocio.

UN PAR DE HISTORIAS TRISTES

Las historias de advertencia de dos empresas, P y W, ilustran las consecuencias de decisiones arquitectónicas prematuras. La empresa P, que originalmente tuvo éxito con una aplicación monolítica, complicó en exceso su arquitectura cuando se vio obligada a adaptarse a las tecnologías web, lo que llevó a un esfuerzo de desarrollo excesivo sin una

Más Libros Gratis
Escanea para descargar
Escúchalo

necesidad comercial proporcional. De manera similar, el arquitecto de la empresa W impuso una arquitectura empresarial enrevesada, resultando en un sistema laborioso que obstaculizó el desarrollo y la flexibilidad. Estos ejemplos muestran cómo la falta de previsión en las elecciones arquitectónicas puede multiplicar innecesariamente la carga de trabajo.

FITNESSE

Una historia de éxito arquitectónico es el desarrollo de FitNesse, que evitó compromisos prematuros con frameworks o bases de datos al trazar fronteras claras entre las reglas de negocio y el acceso a datos. La decisión de mantener la funcionalidad abstracta permitió flexibilidad y experimentación, ilustrando cómo una arquitectura bien pensada puede diferir decisiones difíciles.

¿QUÉ LÍNEAS DIBUJAS Y CUÁNDO LAS DIBUJAS?

Las fronteras efectivas se dibujan entre elementos que tienen niveles de importancia variados para las reglas de negocio fundamentales. Por ejemplo, la interfaz gráfica y la base de

Más Libros Gratis
Escanea para descargar
Escúchalo

datos deben permanecer ajenas entre sí y a la lógica de negocio, permitiendo flexibilidad en las elecciones de implementación. El principio enfatiza dibujar líneas entre componentes relevantes e irrelevantes.

ARQUITECTURA DE PLUGINS

Una arquitectura de plugins permite la separación de funciones centrales de componentes opcionales. Esta estructura permite que el sistema reemplace o actualice partes sin afectar significativamente al sistema en su conjunto. Al tratar componentes como interfaces gráficas o bases de datos como plugins, se vuelve posible adaptarse a diversas tecnologías de implementación manteniendo la independencia de la lógica de negocio central.

EL ARGUMENTO DEL PLUGIN

Utilizando ejemplos de diferentes empresas, el texto destaca los beneficios de una arquitectura de plugins para minimizar las dependencias. Así como ReSharper opera de manera independiente de Visual Studio a pesar de su relación, un sistema bien estructurado debería asegurar que los cambios en un componente no afecten negativamente a otros,

Más Libros Gratis
Escanea para descargar
Escúchalo

mejorando la estabilidad y reduciendo la fragilidad a través de fronteras bien definidas.

CONCLUSIÓN

Para dibujar fronteras efectivas en la arquitectura de software, los sistemas deben ser primero particionados en componentes claramente definidos, evitando decisiones prematuras respecto a los detalles de implementación. La arquitectura debería facilitar el flujo de cambios desde componentes de bajo nivel hacia las reglas de negocio centrales, adhiriéndose así a principios como la Inversión de Dependencias y las Abstracciones Estables. Este enfoque promueve la mantenibilidad, adaptabilidad y claridad en el diseño arquitectónico.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 18 Resumen : Anatomía de Límites

18 ANATOMÍA DE LÍMITES

Arquitectura y Límites

La arquitectura de un sistema consiste en componentes de software y los límites que los separan, los cuales pueden manifestarse de diversas formas. Este capítulo explora la importancia de estos límites, especialmente en lo que respecta a los traspasos de límites en la arquitectura del sistema.

Traspaso de Límites

En tiempo de ejecución, un traspaso de límite ocurre cuando una función llama a otra función a través de un límite, transfiriendo datos. La gestión efectiva de las dependencias del código fuente es crucial, ya que los cambios en un módulo pueden requerir modificaciones o recompilaciones

Más Libros Gratis
Escanea para descargar
Escúchalo

en otros. Los límites sirven como un medio para protegerse contra tales cambios.

El Temido Monolito

El límite arquitectónico más básico a menudo carece de una representación física y es efectivamente una segregación disciplinada de la funcionalidad dentro de un solo espacio de dirección. Desde una perspectiva de implementación, esto se manifiesta como una aplicación monolítica, compuesta por un único archivo ejecutable. Aunque los límites en sistemas monolíticos pueden no ser visibles, aún juegan un papel crítico en la gestión de dependencias internas a través del polimorfismo dinámico. Los monolitos mal estructurados pueden llevar a una peligrosa dependencia de punteros de función, arriesgando el desacoplamiento.

Componentes de Implementación

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

Escanear para descargar

Por qué Bookey es una aplicación imprescindible para los amantes de los libros

Contenido de 30min
Cuanto más profunda y clara sea la interpretación que proporcionamos, mejor comprensión tendrás de cada título.

Formato de texto y audio
Absorbe conocimiento incluso en tiempo fragmentado.

Preguntas
Comprueba si has dominado lo que acabas de aprender.

Y más
Múltiples voces y fuentes, Mapa mental, Citas, Clips de ideas...

Prueba gratuita con Bookey

Capítulo 19 Resumen : Política y Nivel

19 POLÍTICA Y NIVEL

Sistemas de Software como Declaraciones de Política

Los sistemas de software son, fundamentalmente, expresiones de política, describiendo cómo se transforman las entradas en salidas a través de diversas declaraciones de política más pequeñas, como reglas de negocio y validación de datos. Una arquitectura de software efectiva implica separar estas políticas según cómo cambian.

Niveles de Política

Un "nivel" en este contexto se refiere a la distancia de una política respecto a las entradas y salidas; las políticas de nivel superior están más alejadas y generalmente cambian con menos frecuencia que las políticas de nivel inferior.

Diseño de Arquitectura y Dependencias

Más Libros Gratis
Escanea para descargar
Escúchalo

Una buena arquitectura de software organiza los componentes en un grafo acíclico dirigido, donde los nodos representan componentes en el mismo nivel y las dependencias (relaciones de código fuente) dictan cómo interactúan. Las dependencias deben apuntar desde componentes de nivel inferior hacia componentes de nivel superior.

Comprensión del Nivel

La definición estricta de "nivel" se relaciona con su distancia de las entradas y salidas, donde las políticas gestionan interacciones directas en el nivel más bajo. Usando un ejemplo de flujo de datos, componentes como Translate destacan el diseño de nivel superior al desacoplar la funcionalidad de las funciones de entrada y salida específicas.

La Importancia de la Separación

Separar las políticas de nivel superior de los detalles de nivel inferior minimiza el impacto de los cambios. Los cambios en las políticas de entrada/salida tienen menos probabilidades de afectar a las políticas de nivel superior, fomentando

Más Libros Gratis
Escanea para descargar
Escúchalo

flexibilidad y reduciendo los costos de mantenimiento.

Contemplación del Diseño y Dependencias

Los errores de diseño pueden surgir cuando las funciones de alto nivel dependen directamente de las operaciones de bajo nivel. Una arquitectura adecuada asegura que cada política funcione de manera distinta, manteniendo flexibilidad para los cambios en el sistema.

Conclusión

En esencia, el diseño de las arquitecturas de software refleja una compleja interrelación de varios principios como SRP, OCP, CCP, entre otros, enfatizando la evaluación continua y la gestión de las dependencias de los componentes para mejorar la flexibilidad y la mantenibilidad.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 20 Resumen : Reglas de Negocio

REGLAS DE NEGOCIO

Las reglas de negocio dictan las operaciones que pueden generar o ahorrar dinero para una empresa. Estas incluyen las Reglas de Negocio Críticas, que siguen siendo aplicables independientemente de si están automatizadas o no, además de los Datos Críticos de Negocio necesarios para su ejecución. Las entidades, que encarnan estas Reglas de Negocio Críticas utilizando Datos Críticos de Negocio, sirven como objetos específicos dentro del sistema donde reside la lógica empresarial pertinente.

ENTIDADES

Las entidades son objetos del sistema que encapsulan las reglas y datos críticos de negocio, ofreciendo una interfaz clara para la lógica empresarial. Se mantienen aisladas de preocupaciones externas como bases de datos o interfaces de usuario.

Más Libros Gratis
Escanea para descargar
Escúchalo

CASOS DE USO

No todas las reglas de negocio pueden vincularse directamente a entidades; muchas implican restricciones operativas específicas sobre cómo funcionan los sistemas automatizados. Los casos de uso delinean estas operaciones, especificando las interacciones del usuario mientras mantienen los detalles de implementación abstractos e independientes de la interfaz de usuario. Los casos de uso conservan las reglas de negocio mientras controlan su orden de ejecución y accesibilidad.

MODELOS DE SOLICITUD Y RESPUESTA

Los casos de uso generan estructuras de datos de solicitud y respuesta que no dependen de frameworks específicos, asegurando que la lógica empresarial se mantenga desacoplada de las especificidades de la interfaz de usuario. Esta independencia permite interacciones flexibles y a prueba de futuro entre los usuarios y las reglas de negocio.

CONCLUSIÓN

Más Libros Gratis
Escanea para descargar
Escúchalo

Las reglas de negocio son esenciales para el éxito operativo y deben estar encapsuladas dentro de entidades bien definidas, aisladas de preocupaciones de implementación como interfaces de usuario o bases de datos para mantener la integridad y adaptabilidad del sistema.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 21 Resumen : Arquitectura Gritando

ARQUITECTURA GRITANDO

Entendiendo la Arquitectura

- La arquitectura puede compararse con los planos de un edificio, que indican el uso previsto de la estructura (por ejemplo, casas frente a bibliotecas).
- De manera similar, la arquitectura de aplicaciones debe comunicar su propósito claramente: ¿es un “Sistema de Salud” o simplemente una pila tecnológica como “Rails” o “Spring”?

TEMA DE UNA ARQUITECTURA

- Según Ivar Jacobson, las arquitecturas de software deben apoyar los casos de uso de un sistema, enfatizando que la arquitectura no se trata meramente de frameworks.
- Los frameworks son herramientas y no deben dictar las

Más Libros Gratis
Escanea para descargar
Escúchalo

decisiones arquitectónicas.

PROPÓSITO DE UNA ARQUITECTURA

- Una buena arquitectura enfatiza los casos de uso, permitiendo flexibilidad en la elección de frameworks, bases de datos y entornos hasta más adelante en el desarrollo.
- La arquitectura debe garantizar la usabilidad sin comprometerse de inmediato con implementaciones específicas.

¿PERO QUÉ PASA CON LA WEB?

- La web es un mecanismo de entrega, y la arquitectura de aplicaciones debe permanecer ajena a los detalles de entrega hasta que sea necesario, manteniendo la capacidad de adaptarse a diferentes formatos.

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

App Store
Selección editorial

22k reseñas de 5 estrellas

Escanear para descargar

Retroalimentación Positiva

Alondra Navarrete

itas después de cada resumen
en a prueba mi comprensión,
cen que el proceso de
rtido y atractivo."

¡Fantástico! Beltrán Fuentes

Me sorprende la variedad de libros e idiomas que soporta Bookey. No es solo una aplicación, es una puerta de acceso al conocimiento global. Además, ganar puntos para la caridad es un gran plus!

Fi

Lo
re
co
pr

a Vásquez

nábito de
le y sus
o que el
todos.

¡Me encanta! Darian Rosales

Bookey me ofrece tiempo para repasar las partes importantes de un libro. También me da una idea suficiente de si debo o no comprar la versión completa del libro. ¡Es fácil de usar!

¡Ahorra tiempo!

Bookey es mi aplicación de
crecimiento intelectual. Lo
perspicaces y bellamente c
acceso a un mundo de con

icación increíble! Elvira Jiménez

ncantan los audiolibros pero no siempre tengo tiempo
escuchar el libro entero. ¡Bookey me permite obtener
esumen de los puntos destacados del libro que me
esa! ¡Qué gran concepto! ¡Muy recomendado!

Aplicación hermosa

Esta aplicación es un salvavidas para los a
los libros con agendas ocupadas. Los resu
precisos, y los mapas mentales ayudan a
que he aprendido. ¡Muy recomendable!

Prueba gratuita con Bookey

Capítulo 22 Resumen : La Arquitectura Limpia

LA ARQUITECTURA LIMPIA

Resumen de Ideas Arquitectónicas

- La evolución de la arquitectura de software incluye conceptos como Arquitectura Hexagonal (Puertos y Adaptadores), DCI y BCE, todos con el objetivo de lograr la separación de preocupaciones a través de un diseño en capas.

Características de las Capas

- Independencia de los frameworks: Permite usarlos como herramientas.
- Testabilidad: Las reglas de negocio pueden ser testeadas de manera independiente de elementos externos.
- Independencia de la Interfaz de Usuario: La interfaz de usuario se puede cambiar sin afectar el sistema central.

Más Libros Gratis
Escanea para descargar
Escúchalo

- Independencia de la Base de Datos: Permite cambiar bases de datos sin impactar las reglas de negocio.
- Independencia de Agencias Externas: Las reglas de negocio permanecen indiferentes a las interfaces externas.

Regla de Dependencia

- Las dependencias deben apuntar hacia adentro, hacia políticas de alto nivel, asegurando que los círculos internos no sean conscientes de los detalles de los círculos externos.

Entidades

- Las entidades encarnan reglas de negocio críticas y contienen datos empresariales, cambiando con menos frecuencia que su contexto operacional.

Casos de Uso

- Los casos de uso implementan reglas específicas de la aplicación, orquestando el flujo de datos entre entidades, y están aislados de cambios externos.

Más Libros Gratis
Escanea para descargar
Escúchalo

Adaptadores de Interfaz

- Convertir formatos de datos entre casos de uso/entidades y agencias externas (como bases de datos), incluyendo componentes como controladores y vistas.

Frameworks y Controladores

- La capa más externa abarca frameworks, que están abstraídos de la lógica de negocio central para minimizar restricciones.

Capas Arquitectónicas

- La arquitectura sigue círculos concéntricos que representan niveles de software, desde políticas de alto nivel (círculos internos) hasta detalles de bajo nivel (círculos externos).

Cruzando Fronteras

- Comunicarse a través de fronteras implica controlar las dependencias del código fuente para ajustarse a la Regla de Dependencia, permitiendo una arquitectura flexible.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ejemplo de Escenario Típico

- Un flujo de trabajo de sistema basado en la web incluye controladores, casos de uso y gestión de datos de salida, respetando la Regla de Dependencia con objetos personalizados simplificados.

Conclusión

- La arquitectura en capas apoya la testabilidad, la independencia y la adaptabilidad. Una adecuada separación de preocupaciones conduce a un sistema más mantenible y versátil.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ejemplo

Punto clave:Ejemplo inmersivo de independencia por capas

Ejemplo:Imagina que estás desarrollando una plataforma de comercio electrónico. Visualiza un escenario en el que necesitas actualizar la interfaz de usuario para mejorar la experiencia de compra. En lugar de reescribir grandes cantidades de código, simplemente sustituyes la capa de interfaz de usuario por un marco moderno. La belleza de la Arquitectura Limpia radica en sus capas estructuradas: tu lógica de negocio permanece intacta y funcional, demostrando cómo la separación de preocupaciones te permite innovar sin interrumpir los procesos clave.

Más Libros Gratis
Escanea para descargar
Escúchalo

Pensamiento crítico

Punto clave:Regla de Dependencia e Independencia de Marcos

Interpretación crítica:Un concepto crítico en Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software es la 'Regla de Dependencia', que sugiere que todas las dependencias en un sistema deben apuntar hacia componentes de alto nivel para mejorar la mantenibilidad y la capacidad de prueba. Este principio desafía la visión tradicional de que los marcos deben dictar el diseño de la aplicación, abogando por un modelo en el que la lógica empresarial central permanezca aislada de los cambios en marcos y tecnologías externas. Los críticos, incluidos aquellos alineados con las tendencias emergentes en el diseño de software, argumentan que la adhesión estricta a tales reglas puede llevar a una complejidad adicional y obstaculizar el desarrollo rápido, sugiriendo un equilibrio entre la adherencia a los principios arquitectónicos y la flexibilidad pragmática. A medida que las prácticas en la industria del software evolucionan, es crucial interrogar la aplicabilidad de reglas arquitectónicas rígidas frente a las dinámicas de

Más Libros Gratis
Escanea para descargar
Escúchalo

proyectos del mundo real, donde los requisitos cambiantes a menudo exigen adaptaciones más rápidas. La literatura de apoyo se puede encontrar en obras como 'Designing Data-Intensive Applications' de Martin Kleppmann, que discuten la flexibilidad requerida en las arquitecturas de aplicaciones contemporáneas.

Capítulo 23 Resumen : Presentadores y Objetos Humildes

23 PRESENTADORES Y OBJETOS HUMILDES

El Patrón de Objeto Humilde

El patrón de Objeto Humilde aborda el desafío de separar comportamientos difíciles de probar de aquellos que son fácilmente verificables. Divide los comportamientos en dos clases: el objeto humilde, que contiene comportamientos no probables, y la otra clase que alberga comportamientos probables. En el contexto de las interfaces gráficas, el Presentador es probatorio, mientras que la Vista es humilde y se encarga principalmente de mostrar datos sin procesarlos.

Presentadores y Vistas

La Vista carga datos para su visualización, pero no los procesa. En su lugar, el Presentador formatea los datos recibidos de la aplicación y los prepara para la Vista a través

Más Libros Gratis
Escanea para descargar
Escúchalo

de un Modelo de Vista. El Presentador gestiona diversas presentaciones de datos—fechas, divisas y cadenas de texto—manteniendo así las responsabilidades de la Vista al mínimo y de manera sencilla.

Pruebas y Arquitectura

La capacidad de prueba es vital para una buena arquitectura. El patrón de Objeto Humilde ilustra esto al resaltar la importancia de dividir los comportamientos basados en su capacidad de prueba, definiendo límites arquitectónicos, especialmente entre el Presentador y la Vista.

Puertas de Base de Datos

Las puertas de base de datos conectan los interactores de casos de uso y la base de datos. Proporcionan interfaces para operaciones CRUD y se clasifican como objetos humildes que simplemente gestionan el acceso a los datos utilizando SQL u otras interfaces de base de datos.

Mapeadores de Datos

Los mapeadores de datos (o ORM) transfieren datos entre

Más Libros Gratis
Escanea para descargar
Escúchalo

objetos y bases de datos. Pertenecen a la capa de base de datos, creando un límite que separa la lógica de aplicación de las operaciones de datos.

Escuchadores de Servicios

Los servicios también representan límites creados por el patrón de Objeto Humilde. Los datos se cargan en estructuras simples para servicios externos, y los datos del servicio entrante se formatean en estructuras utilizables para la aplicación.

Conclusión

El uso del patrón de Objeto Humilde en los límites arquitectónicos mejora la capacidad de prueba del sistema al marcar visualmente las distinciones entre elementos difíciles de probar y fácilmente verificables, lo que conduce a arquitecturas más claras y mantenibles.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 24 Resumen : Fronteras Parciales

FRONTERAS PARCIALES

Las fronteras arquitectónicas completas son costosas y requieren un esfuerzo significativo para su diseño y mantenimiento. Un buen arquitecto puede preferir implementar una frontera parcial en su lugar, reservando espacio para una completa si es necesario más adelante. Este diseño anticipado a menudo es criticado en Agile por violar YAGNI (You Aren't Gonna Need It). Sin embargo, los arquitectos pueden considerar de manera pragmática las necesidades futuras al implementar.

SALTAR EL ÚLTIMO PASO

Una forma de crear una frontera parcial es realizar todo el trabajo de diseño necesario y luego compilar y desplegar todos los componentes como una única unidad. Esto ahorra en la carga administrativa, como el seguimiento de versiones y la gestión de lanzamientos, mientras se mantiene la

Más Libros Gratis
Escanea para descargar
Escúchalo

configuración de una potencial frontera completa futura. Un ejemplo de este enfoque es la estrategia inicial para FitNesse, donde los componentes se pretendían separables pero inicialmente se agruparon debido a la intención de simplificar la experiencia del usuario.

FRONTERAS UNIDIMENSIONALES

Una frontera arquitectónica completa implica interfaces recíprocas para el aislamiento en ambas direcciones, pero esto puede ser costoso. Una estructura más simple puede mantener una separación básica, a menudo ilustrada a través del patrón Strategy. Una interfaz ServiceBoundary aísla a los clientes de las clases ServiceImpl, preparando el terreno para una futura expansión hacia fronteras completas. Sin embargo, sin interfaces mutuas, puede ocurrir degradación, permitiendo que se formen dependencias de canal reverso, lo que puede complicar el mantenimiento.

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

Escanear para descargar

Leer, Compartir, Empoderar

Completa tu desafío de lectura, dona libros a los niños africanos.

El Concepto

BOOKS FOR AFRICA × ×

Esta actividad de donación de libros se está llevando a cabo junto con Books For Africa. Lanzamos este proyecto porque compartimos la misma creencia que BFA: Para muchos niños en África, el regalo de libros realmente es un regalo de esperanza.

La Regla

Gana 100 puntos → **Canjea un libro** → **Dona a África**

Tu aprendizaje no solo te brinda conocimiento sino que también te permite ganar puntos para causas benéficas. Por cada 100 puntos que ganes, se donará un libro a África.

Prueba gratuita con Bookey

Capítulo 25 Resumen : Capas y Límites

CAPEZAS Y LÍMITES

Visión General de los Componentes del Sistema

- Los sistemas generalmente se pueden simplificar en tres componentes: Interfaz de Usuario (UI), Reglas de Negocio y Base de Datos.
- Sin embargo, los sistemas más complejos requieren una descomposición más matizada.

Ejemplo de Cazar al Wumpus

- Un clásico juego basado en texto que demuestra cómo separar la UI y las reglas del juego a través de una API independiente del lenguaje, permitiendo flexibilidad en la UI.
- Las reglas del juego se comunican con un sistema de almacenamiento de datos utilizando una API, manteniendo una clara separación de los detalles de implementación.

Aplicación de Arquitectura Limpia

Más Libros Gratis
Escanea para descargar
Escúchalo

- El concepto de Arquitectura Limpia promueve límites arquitectónicos bien definidos, que pueden evaluarse a lo largo de diversos ejes de cambio, como lenguaje y mecanismos de comunicación.

Desacoplamiento de Casos de Uso

- Una arquitectura de sistema efectiva reconoce los diferentes ejes de cambio y organiza los componentes de acuerdo a ello, creando APIs que permiten flexibilidad y aislamiento de diferentes funcionalidades dentro del sistema.

Flujo de Datos Cruzados

- A medida que los sistemas crecen en complejidad, los flujos de datos pueden multiplicarse.
- El componente GameRules puede tener una política de nivel superior que interactúa con varios subsistemas.

Agregar Complejidad

- La introducción de componentes adicionales como servicios de red requiere una organización cuidadosa pero preserva la

Más Libros Gratis
Escanea para descargar
Escúchalo

estructura del componente central para procesar datos.

Análisis de Límites

- Una arquitectura efectiva requiere identificar límites y asegurar que los componentes de nivel superior mantengan independencia de los de nivel inferior para evitar entrelazamientos y mejorar la mantenibilidad.

Conclusión

- Reconocer la necesidad de límites en la arquitectura de software ayuda a entender las interacciones y dependencias, garantizando que se tomen decisiones de diseño efectivas para soportar ajustes futuros y escalabilidad sin altos costos.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 26 Resumen : El Componente Principal

26 EL COMPONENTE PRINCIPAL

El Detalle Definitivo

El Componente Principal es una parte crucial de cualquier sistema de software que crea, coordina y supervisa otros componentes. Sirve como el punto de entrada inicial para el sistema y es donde se inyectan las dependencias, normalmente utilizando un marco de Inyección de Dependencias. Este componente a menudo se llama el "más sucio" porque maneja la gestión de dependencias y la orquestación del sistema sin exponer funcionalidades de alto nivel.

Ejemplo de Componente Principal

Por ejemplo, en una versión del juego "Cazar al Wumpus", el Componente Principal inicializa el juego cargando las

Más Libros Gratis
Escanea para descargar
Escúchalo

configuraciones necesarias y emitiendo comandos basados en la entrada del usuario. Coordina varios aspectos del juego, asegurando que trabajen juntos de manera fluida mientras permanece desacoplado de los detalles de bajo nivel.

Conclusión

El Componente Principal juega un papel vital en el mantenimiento de la limpieza y la organización dentro de una arquitectura de software. Establece la conexión fundamental entre las funcionalidades abstractas de alto nivel y los detalles de bajo nivel, facilitando así a los desarrolladores la gestión de dependencias y el comportamiento del sistema.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 27 Resumen : Servicios: Grandes y Pequeños

27 SERVICIOS: GRANDES Y PEQUEÑOS

¿Arquitectura de Servicios?

- Las arquitecturas orientadas a servicios y de microservicios son populares.
- Se cree que proporcionan un fuerte desacoplamiento e independencia en el desarrollo, lo cual es solo parcialmente cierto.
- Utilizar servicios no define inherentemente una arquitectura; a menudo son solo costosas llamadas a funciones.

¿Beneficios de los Servicios?

- Los beneficios de la arquitectura de servicios a menudo se exageran.
-

Más Libros Gratis
Escanea para descargar
Escúchalo

La Falacia del Desacoplamiento
: Los servicios se perciben como desacoplados porque se ejecutan en diferentes procesos. Sin embargo, aún pueden estar fuertemente acoplados a través de datos e interfaces compartidas.
-
La Falacia del Desarrollo y Despliegue Independiente
: Se supone que los servicios son gestionados de manera independiente por diferentes equipos, pero los cambios en los datos o en el comportamiento pueden requerir una coordinación significativa entre los equipos.

El Problema del Gatito

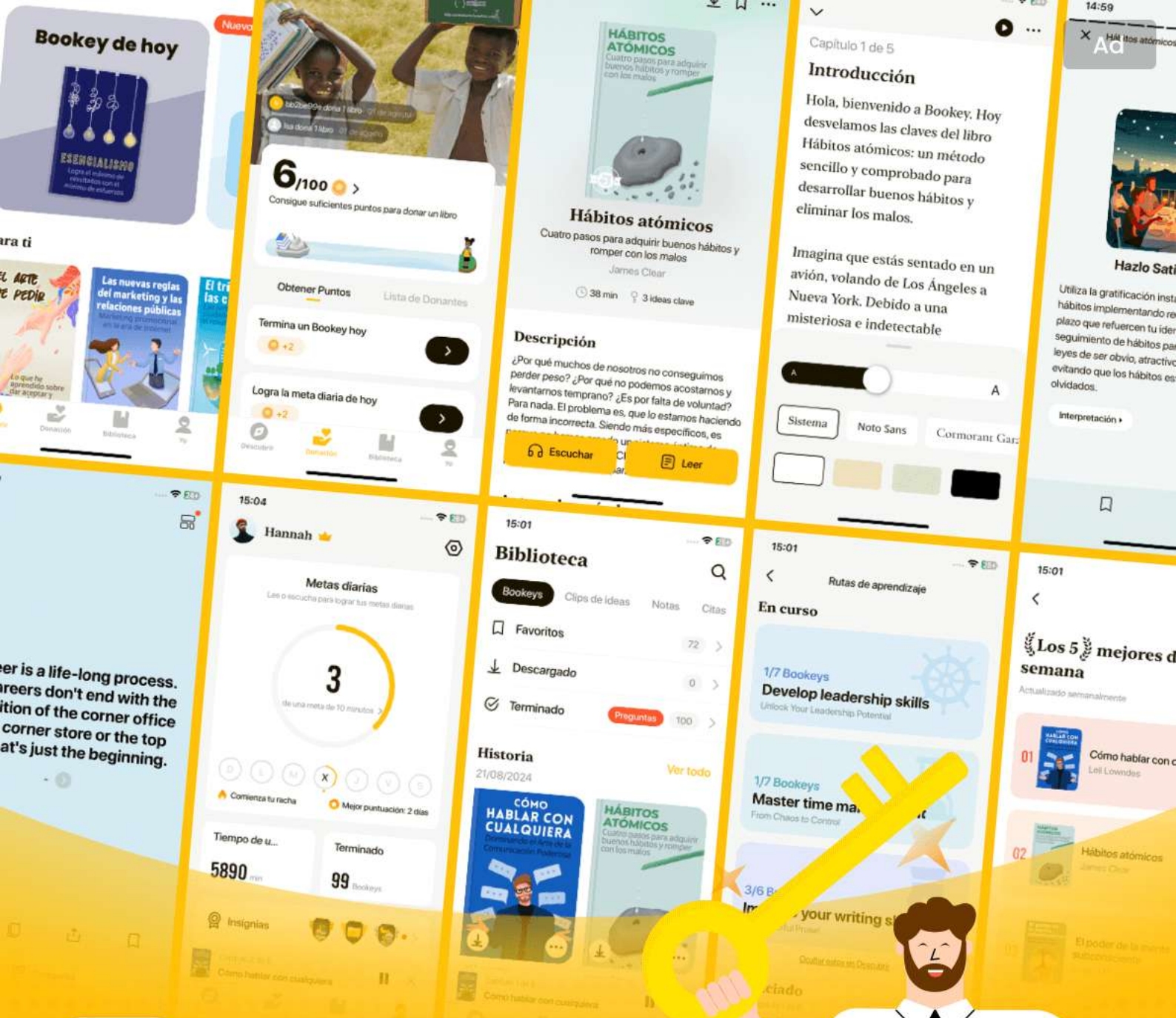
- El ejemplo de un sistema agregador de taxis ilustra que cambios, como incorporar un servicio de entrega de gatitos, requerirían modificaciones en todos los servicios

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo



Las mejores ideas del mundo desbloquean tu potencial

Prueba gratuita con Bookey

Escanear para descargar

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 28 Resumen : El Límite de las Pruebas

EL LÍMITE DE LAS PRUEBAS

Pruebas como Componentes del Sistema

Las pruebas son parte integral de la arquitectura de un sistema, participando al igual que cualquier otra parte. Existe confusión respecto a la naturaleza de las pruebas, pero desde una perspectiva arquitectónica, todas las pruebas son equivalentes—ya sean pruebas unitarias, pruebas de integración, pruebas de aceptación, etc. Siguen la Regla de Dependencia, dependiendo del código que se está probando y operando como el círculo externo en la arquitectura del sistema. Las pruebas son desplegables de forma independiente y no afectan el funcionamiento del sistema; su papel principal es apoyar el desarrollo.

Diseño para la Testabilidad

Más Libros Gratis
Escanea para descargar
Escúchalo

La aislamiento de las pruebas a menudo conduce al concepto erróneo de que están desconectadas del diseño del sistema, lo que puede resultar en pruebas frágiles que obstaculizan los cambios en el sistema. Un fuerte acoplamiento entre las pruebas y el sistema provoca fallos generalizados en las pruebas con modificaciones aparentemente menores del sistema, mostrando el Problema de las Pruebas Frágiles. La solución radica en diseñar para la testabilidad, evitando dependencias de elementos volátiles, como las interfaces gráficas. Las pruebas deben evaluar las reglas de negocio sin necesidad de interacción con la interfaz gráfica.

La API de Pruebas

Para verificar las reglas de negocio, se puede crear una API especializada para pruebas, diseñada para eludir restricciones y facilitar la prueba de estados dentro de la aplicación, asegurando independencia de la estructura de la aplicación. El acoplamiento estructural, una forma perjudicial de acoplamiento de pruebas, ocurre cuando las clases de prueba se correlacionan directamente con las clases de producción. Reducir el acoplamiento estructural a través de una API de pruebas permite que el código de producción y las pruebas evolucionen por separado, promoviendo flexibilidad y

Más Libros Gratis
Escanea para descargar
Escúchalo

generalidad en el código de producción mientras se asegura que las pruebas sigan siendo adaptables.

Conclusión

Las pruebas son componentes vitales que deben integrarse en el diseño de un sistema. Las pruebas mal diseñadas conducen a fragilidad y aumentan las cargas de mantenimiento, destacando la necesidad de un marco de pruebas estructurado dentro de la arquitectura general.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 29 Resumen : Arquitectura Embebida Limpia

29 ARQUITECTURA EMBEBIDA LIMPIA

Introducción

- James Grenning reflexiona sobre las implicaciones de una declaración hecha por Doug Schmidt sobre la necesidad de actualizaciones de software debido a la evolución del firmware y hardware.
- Se enfatiza la distinción entre software (que no se desgasta) y firmware (que se vuelve obsoleto).

Entendiendo el Firmware

- Se discuten las definiciones de firmware, señalando que está vinculado a dependencias de hardware, más que a su medio de almacenamiento.
- Las dependencias no gestionadas en hardware pueden complicar el software embebido y afectar negativamente su

Más Libros Gratis
Escanea para descargar
Escúchalo

duración.

Prueba de Aptitud de Aplicación

- El enfoque en hacer funcional rápidamente el código embebido a menudo lleva a una mala estructuración para el mantenimiento a largo plazo.
- Las tres etapas del desarrollo de software de Kent Beck resaltan la omisión común de la correcta estructuración al alcanzar funcionalidad.

Culpa del Hardware Objetivo

- Los desarrolladores embebidos enfrentan desafíos únicos, incluidas limitaciones de memoria y restricciones en tiempo real.
- Muchos sistemas embebidos están entrelazados con hardware, creando retos en pruebas y depuración.

Capas en la Arquitectura Embebida

- Se propone una arquitectura limpia que incluye la separación entre hardware y software, permitiendo adaptabilidad a medida que el hardware cambia.

Más Libros Gratis
Escanea para descargar
Escúchalo

- Se introduce la Capa de Abstracción de Hardware (HAL) para mitigar dependencias que contaminen el software.

Sistema Operativo como un Detalle

- Se sugiere una Capa de Abstracción de Sistema Operativo (OSAL) para mitigar dependencias de sistemas operativos específicos, ofreciendo mayor flexibilidad.
- Los principios de arquitectura limpia enfatizan que el software embebido debe seguir siendo testeable sin depender de hardware o sistemas operativos.

Programar a Interfaces

- Seguir las mejores prácticas de programar a interfaces y asegurar el desacoplamiento es vital para el mantenimiento y testabilidad.
- Se discuten las limitaciones sobre el contenido de archivos de encabezado para evitar dependencias innecesarias.

Conclusión

- Se reitera la necesidad de una arquitectura embebida limpia para la salud a largo plazo de los productos.

Más Libros Gratis
Escanea para descargar
Escúchalo

- Se anima a los desarrolladores a aprender de prácticas más amplias de desarrollo de software para evitar incorporar firmware innecesario en sus bases de código.

Este resumen encapsula las ideas principales del Capítulo 29 de "Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software," enfatizando la necesidad de una estructuración efectiva en sistemas embebidos para asegurar longevidad y mantenibilidad.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 30 Resumen : La Base de Datos Es un Detalle

Resumen del Capítulo 30: La Base de Datos Es un Detalle

Descripción General

La base de datos se considera una no entidad en la arquitectura del software, similar a una manija de puerta en la arquitectura de una casa. Si bien el modelo de datos es crucial, la base de datos en sí es solo software que actúa como una utilidad para el acceso a los datos—un detalle de bajo nivel del que no debería comprometerse la integridad arquitectónica.

Bases de Datos Relacionales

Desarrolladas a partir de los principios de Edgar Codd en la década de 1970, las bases de datos relacionales se convirtieron en la tecnología dominante debido a su

Más Libros Gratis
Escanea para descargar
Escúchalo

elegancia y robustez. A pesar de su utilidad, su estructura no se refiere a la arquitectura. Se podría argumentar que los casos de uso deberían permanecer indiferentes a la configuración de tablas relacionales, que solo debería concernir a funciones de utilidad de bajo nivel.

Prevalencia de los Sistemas de Bases de Datos

Los sistemas de bases de datos dominan las empresas de software principalmente debido a la tecnología de discos, la cual ha evolucionado, pero también sufre de limitaciones de velocidad—requiriendo indexación compleja y optimizaciones. Los discos han evolucionado con el tiempo, pero siguen siendo un cuello de botella en comparación con la memoria (RAM). En contextos sin discos físicos, los datos se gestionarían de manera más efectiva con estructuras de datos optimizadas para la velocidad en lugar de modelos relacionales.

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

Escanear para descargar

Prueba la aplicación Bookey para leer más de 1000 resúmenes de los mejores libros del mundo

Desbloquea de 1000+ títulos, 80+ temas

Nuevos títulos añadidos cada semana

Brand | Liderazgo & Colaboración | Gestión del tiempo | Relaciones & Comunicación | Kno

ategia Empresarial | Creatividad | Memorias | Dinero e Inversiones | Conózcase a sí mismo

nprendimiento | Historia del mundo | Comunicación entre Padres e Hijos | Autocuidado | M

Perspectivas de los mejores libros del mundo

Prueba gratuita con Bookey

Capítulo 31 Resumen : La Web Es un Detalle

LA WEB ES UN DETALLE

El Péndulo Infinito

La percepción de la web como una fuerza transformadora en la tecnología es engañosa; es simplemente la más reciente de muchas oscilaciones en la industria de la computación. Históricamente, hemos alternado entre centralizar el poder de computación en servidores y distribuirlo a terminales. La web encaja en este patrón, reflejando paradigmas cambiantes en lugar de representar un cambio fundamental.

La Conclusión

La clave es que la web, como interfaz de usuario gráfica (GUI), debe considerarse un detalle y no un componente central de la arquitectura del sistema. Los arquitectos deben centrarse en aislar las reglas de negocio de las

Más Libros Gratis
Escanea para descargar
Escúchalo

preocupaciones de presentación para mantener la flexibilidad ante la evolución de las tecnologías.

Desacoplamiento de la UI y la Lógica de Negocio

Un ejemplo práctico destaca la importancia de esta separación: la Empresa Q intentó adaptar su popular software de finanzas para imitar el diseño de un navegador web, lo que finalmente causó insatisfacción entre los usuarios y llevó a un regreso a una interfaz de escritorio tradicional. La implicación para los arquitectos es la necesidad de desacoplar la lógica de negocio de las GUI para evitar una influencia indebida de las tendencias de marketing o cambios superficiales.

Independencia del Dispositivo

Siguiendo principios establecidos en décadas anteriores, los arquitectos deben considerar la web solo como otro dispositivo de E/S, enfatizando la necesidad de independencia del dispositivo. La arquitectura debe permitir que la lógica de negocio opere de manera independiente de los detalles específicos de la GUI y otros detalles, promoviendo flexibilidad y adaptabilidad al cambio con el

Más Libros Gratis
Escanea para descargar
Escúchalo

tiempo.

Conclusión

Enfatizar esto puede llevar a desafíos, particularmente al reconciliar los atributos ricos y únicos de las interfaces web con la necesidad de abstracción. Sin embargo, establecer límites entre la UI y la lógica de negocio es esencial para protegerse contra la volatilidad impulsada por presiones externas, asegurando que las funcionalidades centrales permanezcan sin afectar por cambios superficiales.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 32 Resumen : Los Frameworks Son Detalles

32 LOS FRAMEWORKS SON DETALLES

Autores de Frameworks

Los frameworks han ganado popularidad y pueden ser útiles, pero no son equivalentes a la arquitectura de software. Los autores a menudo crean frameworks para abordar sus propios problemas, no los problemas específicos de los usuarios. Aunque puede haber cierta superposición en los desafíos que enfrentan, esto no significa que los frameworks sean inherentemente adecuados para tus necesidades.

Matrimonio Asimétrico

El compromiso con un framework es muy unilateral. Los usuarios invierten mucho en el framework y no reciben garantías de sus autores. Los autores de frameworks generalmente presionan para un acoplamiento estrecho con

Más Libros Gratis
Escanea para descargar
Escúchalo

sus frameworks, dificultando que los usuarios se despeguen de ellos más adelante.

Los Riesgos

Varios riesgos surgen del uso de frameworks:
- Los frameworks pueden violar la Regla de Dependencia, llevando a un acoplamiento estricto con la lógica de negocio central.
- Los frameworks pueden apoyar el desarrollo inicial, pero pueden convertirse en un obstáculo a medida que el producto madura.
- Las actualizaciones de los frameworks pueden introducir desafíos, como características obsoletas o problemas de compatibilidad.
- Puede surgir un mejor framework, pero el cambio podría ser difícil.

La Solución

Evita un acoplamiento estrecho a un framework. Mantenlo a distancia y trátalo como un detalle en las capas externas de la arquitectura. Por ejemplo, si utilizas Spring, evita dispersar anotaciones específicas del framework en la lógica de

Más Libros Gratis
Escanea para descargar
Escúchalo

negocio central. En su lugar, haz que el componente principal esté al tanto del framework, que debería ser el encargado de manejar las interacciones con el framework.

Ahora los Declaro...

Algunos frameworks son inevitables, como STL en C++ o la biblioteca estándar de Java. Reconoce las implicaciones de vincular tu aplicación a un framework, ya que esto puede afectar el ciclo de vida y la flexibilidad de la aplicación.

Conclusión

Acércate a los frameworks con precaución. Idealmente, mantiene límites que permitan minimizar la influencia del framework, permitiendo flexibilidad en el desarrollo futuro.

Más Libros Gratis

Escanea para descargar

Escúchalo

Capítulo 33 Resumen : Estudio de Caso: Ventas de Videos

Estudio de Caso: Ventas de Videos

El Producto

Este estudio de caso se centra en un sitio web de comercio electrónico que vende videos, inspirado en cleancoders.com. La plataforma permite a los individuos transmitir o comprar videos, mientras que las empresas pueden comprar licencias al por mayor. Los actores clave incluyen autores de video, administradores y tanto usuarios individuales como empresariales.

Análisis de Casos de Uso

Identificar a los actores y sus casos de uso es crucial. Usando el Principio de Responsabilidad Única, el sistema se estructurará para asegurar que los cambios para un actor no afecten a los demás. Los casos de uso descritos no son

Más Libros Gratis
Escanea para descargar
Escúchalo

exhaustivos; algunos son abstractos, atendiendo a escenarios más generalizados.

Arquitectura de Componentes

Una arquitectura preliminar divide el sistema en vistas, presentadores, interactores y controladores, correspondientes a cada actor. Cada componente puede representar una unidad desplegable, como un archivo .jar o .dll, simplificando las actualizaciones independientes mientras se permite flexibilidad en la combinación de entregables.

Gestión de Dependencias

La arquitectura sigue la Regla de Dependencia. El control fluye de los controladores a los interactores, de los interactores a los presentadores, y finalmente de los presentadores a las vistas, manteniendo una clara dirección

Instalar la aplicación Bookey para desbloquear texto completo y audio

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

Escanear para descargar

Por qué Bookey es una aplicación imprescindible para los amantes de los libros

Contenido de 30min
Cuanto más profunda y clara sea la interpretación que proporcionamos, mejor comprensión tendrás de cada título.

Formato de texto y audio
Absorbe conocimiento incluso en tiempo fragmentado.

Preguntas
Comprueba si has dominado lo que acabas de aprender.

Y más
Múltiples voces y fuentes, Mapa mental, Citas, Clips de ideas...

Prueba gratuita con Bookey

Capítulo 34 Resumen : El Capítulo Faltante

34 EL CAPÍTULO FALTANTE

Introducción a los Enfoques de Diseño

El capítulo resalta la importancia de los detalles de implementación en el diseño de software, especialmente al aplicar principios de fuentes como "Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software." Ilustra el concepto utilizando un ejemplo práctico de diseño de una librería en línea.

Paquete por Capa

- Enfoque tradicional donde el código se divide en capas (web, lógica de negocio, persistencia).
- Simplifica el desarrollo inicial, pero puede volverse engorroso a medida que el sistema escala.
- Las capas no incorporan inherentemente conceptos del

Más Libros Gratis
Escanea para descargar
Escúchalo

dominio de negocio, lo que lleva a estructuras similares en diferentes proyectos.

Paquete por Función

- Un enfoque alternativo que organiza el código por características o raíces agregadas.
- Ofrece una mejor visibilidad del contexto empresarial y puede simplificar la búsqueda de funcionalidades relacionadas.
- Se considera una mejora menor sobre el paquete por capa, pero aún se considera subóptima.

Puertos y Adaptadores

- Enfatiza una separación entre el código enfocado en el dominio y los detalles técnicos de implementación.
- Aboga por crear una arquitectura donde las dependencias externas, como la interfaz de usuario y bases de datos, dependan de la lógica de negocio desde la perspectiva de una arquitectura limpia.

Paquete por Componente

Más Libros Gratis
Escanea para descargar
Escúchalo

- Sugiere un enfoque más integrado, combinando funcionalidades relacionadas para un solo componente.
- Se enfoca en reducir la complejidad durante el desarrollo mientras mantiene una separación clara de preocupaciones.

Detalles de Implementación

- Revela cómo las prácticas de implementación inadecuadas, como modificadores de acceso demasiado amplios (por ejemplo, público), pueden socavar las intenciones arquitectónicas.
- Discute las implicaciones de las restricciones de acceso y la importancia de las dependencias controladas dentro de la arquitectura.

Conclusión

- Resalta la necesidad de una implementación reflexiva para asegurar que los principios de diseño se traduzcan efectivamente en código mantenible.
- Fomenta la conciencia de posibles trampas y enfatiza la necesidad de adherirse a las reglas arquitectónicas para promover una experiencia de desarrollo óptima.

Este capítulo refuerza la idea de que buenas prácticas

Más Libros Gratis
Escanea para descargar
Escúchalo

arquitectónicas, cuando se combinan con una atención cuidadosa a la implementación, pueden llevar a mejores resultados de software, al mismo tiempo que advierte sobre trampas comunes que podrían descarrilar estos esfuerzos.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 35 Resumen : Apéndice A Arqueología de la Arquitectura

ARQUEOLOGÍA DE LA ARQUITECTURA

Este capítulo realiza un viaje reflexivo a través de varios proyectos que abarcan 45 años, con el objetivo de descubrir principios de una arquitectura de software efectiva.

Sistema de Contabilidad UNION

A finales de la década de 1960, ASC Tabulating se asoció con el Local 705 del Sindicato de Teamsters para crear un sistema de contabilidad. Utilizando una enorme computadora GE Datanet 30, el proyecto enfrentó desafíos como la falta de un sistema operativo o sistema de archivos, lo que obligó al equipo a desarrollar sus propias soluciones.

EL ORFANATO DE MINICOMPUTADORAS

Al pasar al minicomputador Varian 620/f, el equipo utilizó un innovador sistema de superposición para gestionar la

Más Libros Gratis
Escanea para descargar
Escúchalo

memoria limitada de manera efectiva. Se creó un supervisor preventivo, mostrando límites arquitectónicos esenciales y gestión de dependencias.

CORTE CON LÁSER

En Teradyne, el equipo desarrolló un sistema de corte con láser para componentes electrónicos. La computadora M365 utilizada requería cargar y editar manualmente las cintas, ilustrando las limitaciones de la tecnología de esa época y la complejidad de las arquitecturas de sistemas fuertemente acopladas.

MONITOREO DE FUNDICIÓN DE ALUMINIO

Mientras estaba en Outboard Marine Corporation, se construyó un sistema de monitoreo en tiempo real utilizando el System/7 de IBM, sin bibliotecas ni sistemas operativos, destacando la dificultad en el mantenimiento de software debido a arquitecturas fuertemente acopladas.

4-TEL

Desarrollado para probar automáticamente las líneas

Más Libros Gratis
Escanea para descargar
Escúchalo

telefónicas, este sistema cambió de una estructura monolítica a una arquitectura más desacoplada, separando las funciones de recolección y análisis de datos en dos computadoras (COLTs y SACs), mejorando los tiempos de respuesta y la eficiencia de la memoria.

DETERMINACIÓN DE DESPACHO

Encontrar problemas con un código rígido y poco comprendido llevó a una realización sobre la importancia de una arquitectura limpia y mantenible, lo que influyó en los enfoques de diseño en proyectos futuros.

EL GRAN REDISEÑO EN EL CIELO

Los esfuerzos por modernizar los sistemas condujeron a desafíos en el mantenimiento del código legado en diferentes áreas geográficas, con diferentes equipos haciendo cambios contradictorios, enfatizando las complicaciones del reutilización de software e integración.

LENGUAJE C Y BOSS

La transición al uso de C y el desarrollo del Sistema

Más Libros Gratis
Escanea para descargar
Escúchalo

Operativo Básico y el Programador (BOSS) permitió al equipo abordar una variedad de nuevos proyectos con mayor eficiencia y flexibilidad al lograr tareas concurrentes.

VRS: SISTEMA DE RESPUESTA VOZ

Un prototipo para un sistema de respuesta de voz contribuyó con ideas sobre arquitecturas orientadas a servicios con métodos para manejar respuestas y control en tiempo real, aunque enfrentó desafíos con integraciones adicionales de bases de datos.

COMUNICACIONES CLARAS

En esta startup, el autor experimentó entornos caóticos y de ritmo rápido, destacando las trampas de la sobreingeniería y la lucha por mantener proyectos viables en medio de constantes cambios y una falta de interés en el mercado.

EXAMEN DEL REGISTRO DE ARQUITECTOS

En un rol de consultoría, el desarrollo de marcos reutilizables para aplicaciones demostró los desafíos para asegurar una verdadera reutilización, requiriendo en última instancia

Más Libros Gratis
Escanea para descargar
Escúchalo

paralelismos con la retrospectiva y la experiencia del mundo real para crear sistemas efectivos.

CONCLUSIÓN

El capítulo proporciona una visión general de alto nivel de las experiencias entrelazadas y las lecciones aprendidas en varios proyectos a lo largo de la carrera del autor, culminando en la realización de que la arquitectura de software debe adaptarse a las matices de requisitos y entornos específicos. Esta narrativa sirve como base para entender los principios de Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software en el futuro.

Más Libros Gratis
Escanea para descargar
Escúchalo

Pensamiento crítico

Punto clave:La importancia de una arquitectura adaptable ante los diferentes requisitos de los proyectos y las limitaciones tecnológicas.

Interpretación crítica:Mientras Robert C. Martin enfatiza la necesidad de Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software para mantener los sistemas de software, se podría argumentar que las particularidades de cada proyecto pueden requerir elecciones arquitectónicas flexibles que se aparten de sus principios. Críticos como Martin Fowler en "Refactoring" señalan que la adaptabilidad, y no la adhesión estricta a la arquitectura limpia, es a veces fundamental para la innovación y la capacidad de respuesta ante cambios. Es posible que atenerse de manera rígida a ciertos ideales arquitectónicos pueda sofocar la creatividad y las soluciones prácticas en el desarrollo de software.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

App Store
Selección editorial

22k reseñas de 5 estrellas

Escanear para descargar

Retroalimentación Positiva

Alondra Navarrete

itas después de cada resumen
en a prueba mi comprensión,
cen que el proceso de
rtido y atractivo."

¡Fantástico! Beltrán Fuentes

Me sorprende la variedad de libros e idiomas que soporta Bookey. No es solo una aplicación, es una puerta de acceso al conocimiento global. Además, ganar puntos para la caridad es un gran plus!

Fi

Lo
re
co
pr

a Vásquez

nábito de
le y sus
o que el
todos.

¡Me encanta! Darian Rosales

Bookey me ofrece tiempo para repasar las partes importantes de un libro. También me da una idea suficiente de si debo o no comprar la versión completa del libro. ¡Es fácil de usar!

¡Ahorra tiempo!

Bookey es mi aplicación de
crecimiento intelectual. Lo
perspicaces y bellamente c
acceso a un mundo de con

icación increíble! Elvira Jiménez

ncantan los audiolibros pero no siempre tengo tiempo
escuchar el libro entero. ¡Bookey me permite obtener
esumen de los puntos destacados del libro que me
esa! ¡Qué gran concepto! ¡Muy recomendado!

Aplicación hermosa

Esta aplicación es un salvavidas para los a
los libros con agendas ocupadas. Los resu
precisos, y los mapas mentales ayudan a
que he aprendido. ¡Muy recomendable!

Prueba gratuita con Bookey

Mejores frases del Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software. por Robert C. Martin con números de página

Ver en el sitio web de Bookey y generar imágenes de citas hermosas

Capítulo 1 | Frases de las páginas 38-82

1....no hay diferencia entre ellos. Ninguna en absoluto.

2.El objetivo de la arquitectura de software es minimizar los recursos humanos necesarios para construir y mantener el sistema requerido.

3.Si ese esfuerzo es bajo y se mantiene bajo a lo largo de la vida del sistema, el diseño es bueno. Si ese esfuerzo crece con cada nueva versión, el diseño es malo. Así de simple.

4.La única forma de ir rápido es ir bien.

5.Esas curvas drenarán catastróficamente las ganancias del modelo de negocio y llevarán a la empresa a una desaceleración, si no a un colapso total.

6.Si se le da su camino, [el desorden creciente de código]

Más Libros Gratis
Escanea para descargar
Escúchalo

reducirá la productividad a cero en cuestión de meses.

7. Pero para aquellos que no han sido engañados por la excesiva confianza de la Liebre, el resultado es esperado, porque conocen esta simple verdad del desarrollo de software: La única forma de ir rápido es ir bien.

Capítulo 2 | Frases de las páginas 83-134

1. Cada sistema de software proporciona dos valores diferentes a los interesados: comportamiento y estructura. Los desarrolladores de software son responsables de garantizar que ambos valores se mantengan altos.
2. El software fue inventado para ser 'blando'. Se pensó como una forma de cambiar fácilmente el comportamiento de las máquinas.
3. Si me das un programa que funciona perfectamente pero es imposible de cambiar, entonces no funcionará cuando cambien los requisitos, y no podré hacerlo funcionar. Por lo tanto, el programa se volverá inútil.
4. Aquellas cosas que son urgentes rara vez son de gran

Más Libros Gratis
Escanea para descargar
Escúchalo

importancia, y aquellas cosas que son importantes son pocas veces de gran urgencia.

5. Los equipos de desarrollo de software efectivos afrontan esa lucha de frente. No dudan en discutir con todos los demás interesados como iguales.

Capítulo 3 | Frases de las páginas 138-190

1. La programación estructurada impone disciplina sobre la transferencia directa de control.
2. La programación orientada a objetos impone disciplina sobre la transferencia indirecta de control.
3. La programación funcional impone disciplina sobre la asignación.
4. Cada uno de los paradigmas elimina capacidades del programador. Ninguno de ellos añade nuevas capacidades.
5. Los tres paradigmas juntos eliminan las instrucciones goto, punteros a funciones y asignaciones.
6. Por lo tanto, es probable que estos tres paradigmas sean los únicos tres que veremos—al menos, los únicos tres que son negativos.

Más Libros Gratis
Escanea para descargar
Escúchalo

7. Una noción fundamental del cálculo lambda es la inmutabilidad—es decir, la noción de que los valores de los símbolos no cambian.
8. Cada paradigma nos dice más qué no hacer, que lo que nos dice qué hacer.
9. El objetivo de un buen diseño de software es minimizar los recursos humanos requeridos para construir y mantener el sistema requerido.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 4 | Frases de las páginas 191-262

1. Dijkstra una vez dijo: 'Las pruebas muestran la presencia, no la ausencia, de errores.'
2. El problema que Dijkstra reconoció, desde el principio, fue que programar es difícil, y que los programadores no lo hacen muy bien.
3. Eisenhower dijo: Tengo dos tipos de problemas, los urgentes y los importantes. Los urgentes no son importantes, y los importantes nunca son urgentes.
4. Si la arquitectura viene al final, el sistema se volverá cada vez más costoso de desarrollar, y eventualmente el cambio se volverá prácticamente imposible para parte o la totalidad del sistema.
5. La programación estructurada nos obliga a descomponer recursivamente un programa en un conjunto de pequeñas funciones demostrables.
6. La única forma de ir rápido es hacerlo bien.
7. Programar es difícil, y los programadores no lo hacen muy bien.

Más Libros Gratis
Escanea para descargar
Escúchalo

8. El desarrollo de software no es un esfuerzo matemático, aunque parece manipular construcciones matemáticas.

Capítulo 5 | Frases de las páginas 263-350

1. La base de una buena arquitectura es la comprensión y aplicación de los principios del diseño orientado a objetos (OO).
2. La encapsulación se cita como parte de la definición de OO, ya que los lenguajes OO proporcionan una encapsulación de datos y funciones fácil y efectiva.
3. Usar un lenguaje OO hace que el polimorfismo sea trivial.
4. El hecho de que los lenguajes OO proporcionen un polimorfismo seguro y conveniente significa que cualquier dependencia de código fuente, sin importar dónde esté, puede ser invertida.
5. La base de datos y la interfaz de usuario pueden ser complementos de las reglas de negocio.

Capítulo 6 | Frases de las páginas 351-443

1. Las variables en los lenguajes funcionales no varían.

Más Libros Gratis
Escanea para descargar
Escúchalo

2. No puedes tener una condición de carrera o un problema de actualización concurrente si ninguna variable es actualizada.
3. La respuesta a esa pregunta es afirmativa, si tienes almacenamiento infinito y velocidad de procesador infinita.
4. Los arquitectos harían bien en llevar a cabo tanto procesamiento como sea posible en los componentes inmutables, y en sacar tanto código como sea posible de aquellos componentes que deben permitir la mutación.
5. El 'event sourcing' es una estrategia en la que almacenamos las transacciones, pero no el estado.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 7 | Frases de las páginas 449-546

1. Un módulo debería tener una, y solo una, razón para cambiar.
2. Un módulo debería ser responsable ante uno, y solo uno, actor.
3. El SRP dice que separemos el código del que dependen diferentes actores.
4. Estos problemas ocurren porque colocamos código del que dependen diferentes actores en estrecha proximidad.
5. No es inconcebible que el CFO también pudiera verse afectado.

Capítulo 8 | Frases de las páginas 547-649

1. Un artefacto de software debería estar abierto a la extensión, pero cerrado a la modificación.
2. Claramente, si extensiones simples a los requisitos obligan a cambios masivos en el software, entonces los arquitectos de ese sistema de software han incurrido en un fracaso espectacular.
3. Al separar adecuadamente las cosas que cambian por

Más Libros Gratis
Escanea para descargar
Escúchalo

diferentes razones (el Principio de Responsabilidad Única), y luego organizar las dependencias entre esas cosas de manera adecuada (el Principio de Inversión de Dependencias).

4. Si el componente A debe estar protegido de cambios en el componente B, entonces el componente B debe depender del componente A.
5. El objetivo es hacer que el sistema sea fácil de extender sin incurrir en un alto impacto de cambio.

Capítulo 9 | Frases de las páginas 650-758

1. Si para cada objeto o1 del tipo S, existe un objeto o2 del tipo T tal que para todos los programas P definidos en términos de T, el comportamiento de P no cambia cuando o1 se sustituye por o2, entonces S es un subtipo de T.
2. Este diseño se ajusta al LSP porque el comportamiento de la aplicación de facturación no depende, de ninguna manera, de cuál de los dos subtipos utiliza.
3. El infame problema del cuadrado/rectángulo ilustra una

Más Libros Gratis
Escanea para descargar
Escúchalo

violación del LSP porque la altura y el ancho del Rectángulo son mutables independientemente; en cambio, la altura y el ancho del Cuadrado deben cambiar juntos.

4. El LSP ha evolucionado hacia un principio más amplio del diseño de software que se relaciona con las interfaces y las implementaciones.
5. La violación del LSP puede causar que la arquitectura de un sistema se contamine con una cantidad significativa de mecanismos adicionales.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 10 | Frases de las páginas 759-864

1. Es perjudicial depender de módulos que contengan más de lo que necesitas.
2. Los cambios en esas características dentro de D pueden obligar a redeplegar F y, por lo tanto, a redeplegar S.
3. Depender de algo que arrastra una carga que no necesitas puede causarte problemas que no esperabas.

Capítulo 11 | Frases de las páginas 865-981

1. El Principio de Inversión de Dependencias (DIP) nos dice que los sistemas más flexibles son aquellos en los que las dependencias del código fuente se refieren solo a abstracciones, no a concreciones.
2. Cada cambio en una interfaz abstracta corresponde a un cambio en sus implementaciones concretas.
3. Por lo tanto, las interfaces son menos volátiles que las implementaciones.
4. Los buenos diseñadores y arquitectos de software trabajan duro para reducir la volatilidad de las interfaces.
5. No te refieras a clases concretas volátiles. En su lugar,

Más Libros Gratis
Escanea para descargar
Escúchalo

refiérete a interfaces abstractas.

6. El flujo de control cruza la línea curva en la dirección opuesta a las dependencias del código fuente.
7. Las violaciones al DIP no pueden ser eliminadas por completo, pero pueden concentrarse en un pequeño número de componentes concretos y mantenerse separados del resto del sistema.
8. Esto significa que todas las relaciones de componentes son unidireccionales, como se muestra en el gráfico de componentes en la Figura 8.3.
9. El código que implementa políticas de alto nivel no debería depender del código que implementa detalles de bajo nivel.

Capítulo 12 | Frases de las páginas 984-1110

1. Independientemente de cómo se implementen al final, los componentes bien diseñados siempre conservan la capacidad de ser desplegados de manera independiente y, por lo tanto, desarrollados de manera independiente.
2. Compilar un programa grande puede tomar horas. Para

Más Libros Gratis
Escanea para descargar
Escúchalo

acortar los tiempos de compilación, los programadores separaron el código fuente de la biblioteca de funciones de las aplicaciones.

3. La solución fueron los binarios relocatables. La idea detrás de ellos era muy simple. El compilador se modificó para producir código binario que pudiera ser reubicado en memoria por un cargador inteligente.
4. Un cargador de vinculación permitió a los programadores dividir sus programas en segmentos que se podían compilar y cargar por separado.
5. A mediados de la década de 1990, el tiempo dedicado a la vinculación comenzó a disminuir más rápido de lo que nuestras ambiciones podían hacer crecer los programas.
6. Hoy en día, enviamos rutinariamente archivos .jar o DLLs o bibliotecas compartidas como complementos a aplicaciones existentes.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 13 | Frases de las páginas 1111-1250

1. El grano de reutilización es el grano de liberación.
2. Agrupa en componentes aquellas clases que cambian por las mismas razones y al mismo tiempo.
3. No fuerces a los usuarios de un componente a depender de cosas que no necesitan.
4. La tensión entre estos principios que los buenos arquitectos buscan resolver.
5. Agrupa aquellas cosas que cambian al mismo tiempo y por las mismas razones. Separa aquellas cosas que cambian en diferentes momentos o por diferentes razones.

Capítulo 14 | Frases de las páginas 1251-1433

1. No permitir ciclos en el gráfico de dependencias de componentes.
2. La solución a este problema es dividir el entorno de desarrollo en componentes liberables.
3. Si hay ciclos en la estructura de dependencias, entonces el 'síndrome de la resaca' no se puede evitar.
4. Los cambios realizados en un componente no necesitan

Más Libros Gratis
Escanea para descargar
Escúchalo

tener un efecto inmediato en otros equipos.

5. La estructura de componentes no se puede diseñar de arriba hacia abajo.
6. Los diseños no pueden ser completamente estáticos.
7. Al conformarnos con el Principio de Cierre Común (CCP), creamos componentes que son sensibles a ciertos tipos de cambios pero inmunes a otros.
8. Los ‘Jitters’ implican que la estructura de componentes es volátil en presencia de requisitos cambiantes.

Capítulo 15 | Frases de las páginas 1436-1631

1. La arquitectura de un sistema de software es la forma que le dan aquellos que lo construyen.
2. El propósito principal de la arquitectura es apoyar el ciclo de vida del sistema.
3. El objetivo final es minimizar el costo de vida del sistema y maximizar la productividad del programador.
4. Una buena arquitectura hace que el sistema sea fácil de entender, fácil de desarrollar, fácil de mantener y fácil de implementar.

Más Libros Gratis
Escanea para descargar
Escúchalo

5. La estrategia detrás de esa facilitación es dejar tantas opciones abiertas como sea posible, durante el mayor tiempo posible.
6. La arquitectura de un sistema debe elevar los casos de uso, las características y los comportamientos requeridos del sistema a entidades de primera clase que sean hitos visibles para los desarrolladores.
7. El costo principal del mantenimiento radica en la exploración y el riesgo.
8. El objetivo del arquitecto es crear una forma para el sistema que reconozca la política como el elemento más esencial del sistema mientras que los detalles resultan irrelevantes para esa política.
9. Un buen arquitecto maximiza el número de decisiones no tomadas.
10. Si puedes desarrollar la política de alto nivel sin comprometerte con los detalles que la rodean, puedes retrasar y aplazar decisiones sobre esos detalles durante mucho tiempo.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 16 | Frases de las páginas 1632-1848

1. La arquitectura debe apoyar los casos de uso.
2. Una buena arquitectura apoya el funcionamiento del sistema.
3. Una buena arquitectura deja opciones abiertas para el futuro.
4. El modo de desacoplamiento es una de esas opciones que un buen arquitecto deja abiertas.
5. La única manera de ir rápido es ir bien.
6. Una buena arquitectura hace que el sistema sea fácil de cambiar, en todas las formas que debe cambiar, dejando opciones abiertas.

Capítulo 17 | Frases de las páginas 1849-2075

1. La arquitectura de software es el arte de trazar líneas que yo llamo límites.
2. Una buena arquitectura de sistema es aquella en la que decisiones como estas se vuelven accesorias y aplazables.
3. La ironía es que la empresa P nunca vendió un sistema que requiriera una granja de servidores.

Más Libros Gratis
Escanea para descargar
Escúchalo

4. La tragedia es que los arquitectos, al tomar una decisión prematura, multiplicaron enormemente el esfuerzo de desarrollo.
5. Una buena arquitectura hace que el sistema sea fácil de entender, fácil de desarrollar, fácil de mantener y fácil de desplegar.
6. El objetivo final es minimizar el costo de vida del sistema y maximizar la productividad del programador.
7. Los buenos arquitectos separan cuidadosamente los detalles de la política, y luego desacoplan la política de los detalles de tal manera que la política no tenga conocimiento de los detalles y no dependa de ellos de ninguna forma.
8. Una buena arquitectura deja el modo de desacople abierto como una opción para que las grandes implementaciones puedan usar un modo, mientras que las pequeñas implementaciones pueden usar otro.
9. Una buena arquitectura hace que el sistema sea fácil de cambiar, de todas las maneras que debe cambiar, dejando las opciones abiertas.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 18 | Frases de las páginas 2076-2299

1. En tiempo de ejecución, cruzar un límite no es más que una función de un lado del límite llamando a una función del otro lado y pasando algunos datos.
2. Gestionar y construir cortafuegos contra este cambio es de lo que se trata los límites.
3. Tales arquitecturas dependen casi siempre de algún tipo de polimorfismo dinámico para gestionar sus dependencias internas.
4. Incluso cuando están vinculados estáticamente en un solo ejecutable, la capacidad de desarrollar y unir de manera independiente los diversos componentes para el ensamblaje final es inmensamente valiosa.
5. Los componentes de alto nivel permanecen independientes de los detalles de bajo nivel.
6. En consecuencia, las comunicaciones a través de límites desacoplados a nivel de fuente pueden ser bastante ruidosas.
7. Un límite arquitectónico físico mucho más fuerte es el

Más Libros Gratis
Escanea para descargar
Escúchalo

proceso local.

8. El límite más fuerte es un servicio.
9. La comunicación a través de límites de procesos locales implica llamadas al sistema operativo, agrupamiento y decodificación de datos, y cambios de contexto entre procesos, lo que es moderadamente costoso.
10. Una buena arquitectura deja opciones abiertas.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 19 | Frases de las páginas 2300-2519

1. Un programa de computadora es una descripción detallada de la política por la cual las entradas se transforman en salidas.
2. El arte de desarrollar una arquitectura de software consiste en separar cuidadosamente esas políticas entre sí y reagruparlas según la forma en que cambian.
3. Las políticas que cambian por las mismas razones y al mismo tiempo, están en el mismo nivel y pertenecen juntas en el mismo componente.
4. En una buena arquitectura, la dirección de esas dependencias se basa en el nivel de los componentes que conectan. En todos los casos, los componentes de bajo nivel están diseñados para depender de los componentes de alto nivel.
5. Las políticas de alto nivel—las que están más alejadas de las entradas y salidas—tienden a cambiar con menos frecuencia y por razones más importantes que las políticas de bajo nivel.

Más Libros Gratis
Escanea para descargar
Escúchalo

6. Mantener estas políticas separadas, con todas las dependencias del código fuente apuntando en la dirección de las políticas de alto nivel, reduce el impacto del cambio.

Capítulo 20 | Frases de las páginas 2520-2751

1. Las reglas de negocio críticas suelen requerir algunos datos para funcionar. Por ejemplo, nuestro préstamo requiere un saldo de préstamo, una tasa de interés y un calendario de pagos.
2. La Entidad es pura negocio y nada más.
3. Un caso de uso describe las reglas de negocio específicas de la aplicación a diferencia de las Reglas Críticas de Negocio dentro de las Entidades.
4. Los casos de uso no describen cómo se ve el sistema para el usuario. En cambio, describen las reglas específicas de la aplicación que rigen la interacción entre los usuarios y las Entidades.
5. El propósito de estos dos objetos es muy diferente. Con el tiempo, cambiarán por razones muy distintas...

Capítulo 21 | Frases de las páginas 2752-2988

Más Libros Gratis
Escanea para descargar
Escúchalo

1. La arquitectura gritaría: 'HOGAR.'
2. Las arquitecturas no se tratan (ni deben tratarse) de frameworks. Las arquitecturas no deberían ser proporcionadas por frameworks.
3. Las buenas arquitecturas se centran en los casos de uso para que los arquitectos puedan describir con seguridad las estructuras que soportan esos casos de uso sin comprometerse a frameworks, herramientas y entornos.
4. La web es un mecanismo de entrega—un dispositivo de E/S—y tu arquitectura de aplicación debería tratarla como tal.
5. Una buena arquitectura permite posponer y retrasar decisiones sobre frameworks, bases de datos, servidores web y otros problemas y herramientas ambientales.
6. Una buena arquitectura enfatiza los casos de uso y los desacopla de preocupaciones periféricas.
7. Tus objetos de entidad deberían ser objetos comunes y corrientes que no tengan dependencias en frameworks ni bases de datos ni otras complicaciones.

Más Libros Gratis
Escanea para descargar
Escúchalo

8. Tu arquitectura debería hablarles a los lectores sobre el sistema, no sobre los frameworks que usaste en tu sistema.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 22 | Frases de las páginas 2989-3234

1. La regla principal que hace que esta arquitectura funcione es la Regla de Dependencia: las dependencias del código fuente deben apuntar solo hacia adentro, hacia políticas de nivel superior.
2. Cada uno tiene al menos una capa para las reglas de negocio y otra capa para las interfaces de usuario y del sistema.
3. Mantenemos estas cosas en el exterior donde pueden hacer poco daño.
4. Las Entidades encapsulan las Reglas Críticas de Negocio a nivel empresarial.
5. La arquitectura no depende de la existencia de alguna biblioteca de software repleta de características. Esto te permite usar tales marcos como herramientas, en lugar de obligarte a encajar tu sistema en sus limitaciones.
6. Si los detalles de un caso de uso cambian, entonces cierto código en esta capa se verá afectado.
7. Queremos proteger al Controlador de los cambios en los

Más Libros Gratis
Escanea para descargar
Escúchalo

Presentadores. Queremos proteger a los Presentadores de los cambios en las Vistas.

8. Una buena arquitectura permite que las decisiones sobre marcos, bases de datos, servidores web y otros problemas y herramientas ambientales se difieran y retrasen.
9. No queremos que nuestros sistemas exhiban ese tipo de fragilidad.
10. Esto nos lleva a nuestra última métrica. Si es deseable que los componentes estén en, o cerca, de la Secuencia Principal, entonces podemos crear una métrica que mida qué tan lejos está un componente de este ideal.

Capítulo 23 | Frases de las páginas 3235-3488

1. La arquitectura de software es la forma que se le da a ese sistema por quienes lo construyen.
2. La arquitectura de un sistema debe soportar todos los casos de uso necesarios, pero no sabe cuáles son todos esos casos de uso.
3. Una buena arquitectura facilita que el sistema cambie, de todas las maneras en que debe cambiar, dejando opciones

Más Libros Gratis
Escanea para descargar
Escúchalo

abiertas.

4. El patrón de Humble Object es un patrón de diseño que se identificó originalmente como una forma de ayudar a los testers de unidades a separar comportamientos que son difíciles de probar de comportamientos que son fáciles de probar.
5. La capacidad de prueba es un atributo de buenas arquitecturas.
6. Queremos que ciertos módulos sean inmunes a otros.
7. Si puedes desarrollar la política de alto nivel sin comprometerte a los detalles que la rodean, puedes retrasar y posponer las decisiones sobre esos detalles durante mucho tiempo.
8. Una buena arquitectura hace que el sistema sea fácil de entender, fácil de desarrollar, fácil de mantener y fácil de implementar.
9. Las dependencias del código fuente deben apuntar solo hacia adentro, hacia las políticas de nivel superior.
10. Los buenos arquitectos separan cuidadosamente los

Más Libros Gratis
Escanea para descargar
Escúchalo

detalles de la política y luego desacoplan la política de los detalles de tal manera que la política no tiene conocimiento de los detalles y no depende de los detalles de ninguna manera.

Capítulo 24 | Frases de las páginas 3489-3740

1. Una buena arquitectura hace que el sistema sea fácil de entender, fácil de desarrollar, fácil de mantener y fácil de desplegar.
2. ¿El objetivo de un buen diseño de software? Ese objetivo no es otro que mi descripción utópica: El objetivo de la arquitectura de software es minimizar los recursos humanos necesarios para construir y mantener el sistema requerido.
3. Una buena arquitectura proviene de entenderla más como un viaje que como un destino, más como un proceso continuo de indagación que como un artefacto congelado.
4. Una forma simple de romper un ciclo de componentes y restablecer el grafo de dependencias como un DAG es aplicar el Principio de Inversión de Dependencias (DIP).

Más Libros Gratis
Escanea para descargar
Escúchalo

5. Las reglas de negocio deben permanecer prístinas, no contaminadas por preocupaciones más básicas como la interfaz de usuario o la base de datos utilizada.
6. Los buenos arquitectos se esfuerzan por posicionar la mayoría de sus componentes en esos extremos. Sin embargo, en mi experiencia, una pequeña fracción de los componentes en un gran sistema no es ni perfectamente abstracta ni perfectamente estable.
7. Una buena arquitectura deja opciones abiertas, cuántos más experimentos puedas realizar, más cosas podrás probar y más información tendrás cuando llegues al punto en el que esas decisiones ya no puedan ser postergadas.
8. Una buena arquitectura no depende de la existencia de alguna biblioteca de software repleta de características. Esto permite usar tales marcos como herramientas, en lugar de obligarte a encajar tu sistema en sus limitaciones restringidas.
9. Los arquitectos deben supervisar las conexiones a través de los límites organizacionales entre componentes para

Más Libros Gratis
Escanea para descargar
Escúchalo

facilitar una mejor colaboración y eficiencia.

10. La arquitectura de software es el arte de trazar líneas que yo llamo límites.

Más Libros Gratis

Escanea para descargar

Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 25 | Frases de las páginas 3741-4006

1. El propósito principal de la arquitectura es apoyar el ciclo de vida del sistema. Una buena arquitectura hace que el sistema sea fácil de entender, fácil de desarrollar, fácil de mantener y fácil de implementar.
2. Una buena arquitectura permite que las decisiones sobre marcos, bases de datos, servidores web y otros problemas ambientales y herramientas se pospongan y se retrasen.
3. La arquitectura de software es el arte de trazar límites que yo llamo fronteras. Esas fronteras separan los elementos de software entre sí y restringen a los de un lado a no conocer los del otro.
4. El objetivo de una arquitectura de software, entonces, debe ser crear un sistema que se pueda implementar fácilmente con una sola acción.
5. Las políticas que cambian por las mismas razones y en los mismos momentos están al mismo nivel y pertenecen juntas en el mismo componente. Las políticas que cambian

Más Libros Gratis
Escanea para descargar
Escúchalo

por diferentes razones o en diferentes momentos están a diferentes niveles y deben separarse en diferentes componentes.

6. Una buena arquitectura hace que el sistema sea fácil de cambiar, en todas las formas en que debe cambiar, dejando opciones abiertas.
7. Los buenos arquitectos se esfuerzan por posicionar la mayoría de sus componentes en esos puntos finales [de la Secuencia Principal].

Capítulo 26 | Frases de las páginas 4007-4271

1. Llamo a este componente Main. EL DETALLE DEFINITIVO
2. El componente Main es el detalle definitivo: la política de nivel más bajo.
3. Nada, aparte del sistema operativo, depende de él.
4. Su trabajo es crear todas las Fábricas, Estrategias y otras instalaciones globales, y luego ceder el control a las partes abstractas de alto nivel del sistema.
5. Imagina que estás mirando los planos de un edificio... ¿Qué

Más Libros Gratis
Escanea para descargar
Escúchalo

grita la arquitectura de tu aplicación?

6. La arquitectura de un sistema de software es la forma que le dan aquellos que lo construyen.
7. Una buena arquitectura hace que el sistema sea fácil de cambiar, de todas las formas en que debe cambiar, dejando opciones abiertas.
8. Los gerentes deben aprender a reconocer la importancia de separar el comportamiento del sistema de los detalles de implementación.
9. Un objeto de caso de uso bien formado no debería tener ninguna idea sobre la forma en que se comunica la información al usuario, o a cualquier otro componente.
10. El objetivo final es minimizar el costo de vida del sistema y maximizar la productividad del programador.

Capítulo 27 | Frases de las páginas 4272-4558

1. La arquitectura de un sistema se define por límites que separan las políticas de alto nivel de los detalles de bajo nivel y siguen la Regla de Dependencia.

Más Libros Gratis
Escanea para descargar
Escúchalo

2. Los servicios son, después de todo, solo llamadas a funciones a través de límites de procesos y/o plataformas.
3. Una buena arquitectura deja las opciones abiertas.
4. La arquitectura debe revelar la operación.
5. Una buena arquitectura es aquella en la que decisiones como estas se vuelven accesorias y diferibles.
6. No dependas de cosas que no necesitas.
7. La Regla de Dependencia: Las dependencias del código fuente deben apuntar solo hacia adentro, hacia políticas de alto nivel.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 28 | Frases de las páginas 4559-4841

1. Las pruebas son parte del sistema y participan en la arquitectura como lo hace cualquier otra parte del sistema.
2. Las pruebas, por su propia naturaleza, siguen la Regla de Dependencia; son muy detalladas y concretas; y siempre dependen hacia adentro del código que se está probando.
3. Las pruebas que no están bien integradas en el diseño del sistema tienden a ser frágiles, y hacen que el sistema sea rígido y difícil de cambiar.
4. La solución es diseñar para la capacidad de prueba. La primera regla del diseño de software—ya sea para la capacidad de prueba o por cualquier otra razón—siempre es la misma: No dependas de cosas volátiles.
5. El propósito de la API de pruebas es desacoplar las pruebas de la aplicación.
6. Un programa puede ser demostrado incorrecto por una prueba, pero no puede ser demostrado correcto.

Capítulo 29 | Frases de las páginas 4842-5160

Más Libros Gratis
Escanea para descargar
Escúchalo

1. Aunque el software no se desgasta, el firmware y el hardware se vuelven obsoletos, lo que requiere modificaciones en el software.
2. Aunque el software no se desgasta, puede ser destruido desde dentro por dependencias no gestionadas en firmware y hardware.
3. Deja de escribir tanto firmware y dale a tu código una oportunidad para una larga vida útil.
4. Lograr que una aplicación funcione es lo que llamo la prueba de aptitud de un programador.
5. El software de una arquitectura embebida limpia es testeable fuera del hardware de destino.
6. El firmware no significa que el código viva en ROM. No es firmware por dónde se almacena; más bien, es firmware por lo que depende y lo difícil que es cambiarlo a medida que el hardware evoluciona.
7. Una arquitectura embebida limpia es buena para la salud a largo plazo de tu producto.

Capítulo 30 | Frases de las páginas 5163-5484

Más Libros Gratis
Escanea para descargar
Escúchalo

1. Desde un punto de vista arquitectónico, la base de datos es una no-entidad: es un detalle que no alcanza el nivel de un elemento arquitectónico.
2. Un buen arquitecto no permite que los mecanismos de bajo nivel contaminen la arquitectura del sistema.
3. Es muy poco probable que dejes los datos en forma de archivos o tablas.
4. Pero no importa cuán brillante, útil y matemáticamente sólida sea una tecnología, sigue siendo solo una tecnología. Y eso significa que es un detalle.
5. Muchos marcos de acceso a datos permiten que las filas y tablas de la base de datos se pasen por el sistema como objetos. Permitir esto es un error arquitectónico.
6. Para mitigar la demora impuesta por los discos, necesitas índices, cachés y esquemas de consulta optimizados; y necesitas algún tipo de medio regular para representar los datos para que estos índices, cachés y esquemas de consulta sepan con qué están trabajando.
7. Eso necesita quedar claro: no estoy hablando del modelo de

Más Libros Gratis
Escanea para descargar
Escúchalo

datos. La estructura que das a los datos dentro de tu aplicación es muy significativa para la arquitectura de tu sistema.

8. Una buena arquitectura hace que el sistema sea fácil de cambiar, en todas las formas en que debe cambiar, dejando las opciones abiertas.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 31 | Frases de las páginas 5485-5802

1. De hecho, la web no cambió nada. O, al menos, no debería haberlo hecho.
2. La interfaz gráfica es un detalle. La web es una interfaz gráfica. Así que la web es un detalle.
3. Como arquitectos, sin embargo, debemos mirar a largo plazo. Esas oscilaciones son solo problemas a corto plazo que queremos alejar del núcleo central de nuestras reglas de negocio.
4. Siempre hay genios del marketing esperando la mínima oportunidad de acoplar que tú crees.
5. La web es un dispositivo de IO. En la década de 1960, aprendimos el valor de escribir aplicaciones que fueran independientes del dispositivo.

Capítulo 32 | Frases de las páginas 5803-6127

1. Los frameworks no son arquitecturas, aunque algunos intentan serlo.
2. La relación entre tú y el autor del framework es extraordinariamente asimétrica. Debes hacer un gran

Más Libros Gratis
Escanea para descargar
Escúchalo

compromiso con el framework, pero el autor del framework no te hace ningún compromiso a ti.

3. ¡No te cases con el framework! Oh, puedes usar el framework, solo no te acoples a él. Mantenlo a distancia.
4. Ahora los declaro… Entiendan que cuando se casan con un framework en su aplicación, estarán atrapados con ese framework durante todo el ciclo de vida de esa aplicación.
5. La arquitectura del framework a menudo no es muy limpia. Los frameworks tienden a violar la Regla de Dependencia.
6. Si el framework quiere que derives tus objetos de negocio de sus clases base, ¡di que no! En su lugar, deriva proxies y manten esos proxies en componentes que sean plugins para tus reglas de negocio.
7. El anillo de boda está en tu dedo; y se va a quedar ahí.
8. Tus problemas probablemente se superpondrán bastante con esos otros problemas. Si ese no fuera el caso, los frameworks no serían tan populares.
9. Los autores de frameworks conocen sus propios problemas y los problemas de sus compañeros y amigos. Y escriben

Más Libros Gratis
Escanea para descargar
Escúchalo

sus frameworks para resolver esos problemas, no los tuyos.

10. Los frameworks son detalles que pertenecen a uno de los círculos exteriores de la arquitectura. No dejes que entre en los círculos internos.

Capítulo 33 | Frases de las páginas 6128-6457

1. ¿El objetivo de un buen diseño de software? Ese objetivo no es menos que mi descripción utópica: El objetivo de la Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software es minimizar los recursos humanos necesarios para construir y mantener el sistema requerido.
2. No es estrictamente necesario que yo cree esa abstracción. Podría haber dejado el caso de uso abstracto fuera del diagrama sin comprometer ninguna de las características del producto en general.
3. Cada vez que se añade una nueva función, o se cambia alguna función existente, ese paso se tomará para servir a uno de estos actores.

Más Libros Gratis
Escanea para descargar
Escúchalo

4. Una buena arquitectura hace que el sistema sea fácil de entender, fácil de desarrollar, fácil de mantener y fácil de implementar.
5. No todas las reglas de negocio son tan puras como las Entidades. Algunas reglas de negocio generan o ahorran dinero para la empresa al definir y restringir la forma en que funciona un sistema automatizado.
6. La arquitectura también juega un papel crucial en determinar la facilidad con la que se implementa el sistema. El objetivo es 'implementación inmediata.'
7. Es la tensión entre estos principios que los buenos arquitectos buscan resolver.
8. Una buena arquitectura no depende de la existencia de alguna biblioteca de software repleta de funciones. Esto te permite usar tales marcos como herramientas, en lugar de forzarte a encajar tu sistema dentro de sus limitaciones restringidas.
9. La arquitectura de software es el arte de dibujar líneas que yo llamo límites. Esos límites separan los elementos de

Más Libros Gratis
Escanea para descargar
Escúchalo

software entre sí y restringen a aquellos de un lado a no saber sobre los del otro.

10. La web es un dispositivo de entrada/salida. En la década de 1960, aprendimos el valor de escribir aplicaciones que fueran independientes del dispositivo. La motivación por esa independencia no ha cambiado.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 34 | Frases de las páginas 6458-6826

1. El diablo está en los detalles de implementación, y es muy fácil caer en la última trampa si no se le presta atención también.
2. El propósito de una buena arquitectura es dejar tantas opciones abiertas como sea posible, durante el mayor tiempo posible.
3. Los sistemas de software son declaraciones de política. De hecho, en su esencia, eso es todo lo que un programa informático realmente es.
4. Una buena arquitectura hace que el sistema sea fácil de entender, fácil de desarrollar, fácil de mantener y fácil de desplegar.
5. La arquitectura debe separar el código del que dependen diferentes actores.
6. Una buena arquitectura no depende de la existencia de alguna biblioteca de software cargada de características.
7. No dependas de cosas que no necesitas.

Capítulo 35 | Frases de las páginas 6829-6894

Más Libros Gratis
Escanea para descargar
Escúchalo

1. La gran fama del Datanet 30 radicaba en su capacidad para manejar un gran número de terminales asíncronos a una velocidad relativamente alta.
2. Cuando nuestro procesador arrancaba, escaneaba cada chip y cargaba la tabla de vectores al inicio de cada chip en los vectores de RAM.
3. Cada vez que añadíamos o eliminábamos código de nuestro ejecutable de 30K, cambiaban las direcciones en las que se cargaba cada instrucción.
4. La arquitectura la hacía liberable, desarrollable y desplegable de manera independiente.
5. No se puede crear un marco reutilizable hasta que primero se haga un marco usable.
6. La idea era relativamente simple y profundamente idealista. Cada programador orientado a objetos quería tener una base de datos orientada a objetos en su sistema.
7. Habíamos hecho que los chips fueran desplegables de manera independiente. Habíamos inventado la

Más Libros Gratis
Escanea para descargar
Escúchalo

despachación polimórfica. Habíamos inventado objetos.

8. La arquitectura debe ser lo suficientemente flexible para adaptarse al tamaño del problema.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1 millón de citas Resúmenes de más de 1000 libros

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software. Preguntas

Ver en el sitio web de Bookey

Capítulo 1 | ¿Qué es el diseño y la arquitectura?| Preguntas y respuestas

1.Pregunta

¿Cuál es el objetivo principal del diseño y la arquitectura de software?

Respuesta:El objetivo principal del diseño y la arquitectura de software es minimizar los recursos humanos necesarios para construir y mantener el sistema, asegurando que el esfuerzo requerido para satisfacer las necesidades del cliente se mantenga bajo a lo largo de la vida del sistema.

2.Pregunta

¿Cómo define el autor la diferencia entre diseño y arquitectura?

Respuesta:El autor afirma que no hay diferencia entre diseño y arquitectura; ambos abarcan estructuras de alto nivel y

Más Libros Gratis
Escanea para descargar
Escúchalo

detalles de bajo nivel como parte de un todo unificado.

3.Pregunta

¿Cuál es la consecuencia de un mal diseño de software según el autor?

Respuesta:Un mal diseño de software lleva a una mayor complejidad, costos más altos y menor productividad, causando, en última instancia, frustración tanto para los desarrolladores como para los ejecutivos.

4.Pregunta

¿Qué sugiere el autor sobre la naturaleza del código desordenado?

Respuesta:El autor sugiere que el código desordenado ralentiza el desarrollo, contradiciendo la creencia común de que permite una entrega inicial más rápida. Un código más limpio conduce a una mayor productividad y a costos a largo plazo más bajos.

5.Pregunta

¿Qué lección extrae el autor de la historia de la Tortuga y la Liebre?

Respuesta:La historia resalta que la sobreconfianza, similar a

Más Libros Gratis
Escanea para descargar
Escúchalo

la de la liebre, puede llevar a los desarrolladores a descuidar la importancia del código limpio y disciplinado, creyendo que pueden solucionarlo más tarde.

6.Pregunta

¿Qué demuestra el experimento de Jason Gorman sobre el Desarrollo Guiado por Pruebas (TDD)?

Respuesta:El experimento de Gorman muestra que usar TDD resulta en tiempos de finalización más rápidos y una productividad mejorada en comparación con no usarlo, reforzando la noción de que hacer las cosas bien puede mejorar realmente la velocidad.

7.Pregunta

¿Cómo pueden los desarrolladores evitar caer en la trampa de la sobreconfianza en sus habilidades de codificación?

Respuesta:Los desarrolladores pueden evitar esta trampa al tomarse en serio la calidad del software, reconociendo las complejidades de su trabajo y comprometiéndose a mantener una Arquitectura Limpia y diseño desde el principio.

8.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Qué implica el autor que es necesario para una arquitectura de software efectiva?

Respuesta:Una arquitectura de software efectiva requiere un entendimiento de los principios que rigen la calidad del diseño, incluyendo consideraciones a largo plazo para el mantenimiento y la flexibilidad.

9.Pregunta

¿Qué mensaje general transmite el autor sobre los esfuerzos de desarrollo de software?

Respuesta:El autor transmite que, contrariamente a la creencia popular, la única manera de lograr velocidad y eficiencia en el desarrollo de software es priorizar un buen diseño y arquitectura desde el principio.

10.Pregunta

¿Qué se entiende por 'la firma de un desorden' en la producción de software?

Respuesta:La 'firma de un desorden' se refiere al declive observable en la productividad y al aumento de costos que ocurre cuando los esfuerzos de programación son

Más Libros Gratis
Escanea para descargar
Escúchalo

descoordinados, caóticos y carecen de la debida disciplina en el diseño.

Capítulo 2 | Una Historia de Dos Valores| Preguntas y respuestas

1.Pregunta

¿Cuáles son los dos valores fundamentales que cada sistema de software proporciona a los interesados?

Respuesta:Comportamiento y estructura.

2.Pregunta

¿Por qué muchos programadores creen erróneamente que su trabajo solo consiste en corregir errores e implementar requisitos?

Respuesta:A menudo pasan por alto la importancia de la arquitectura del software, pensando que su única responsabilidad es asegurarse de que el sistema se comporte correctamente de acuerdo con las especificaciones.

3.Pregunta

¿A qué se refiere el término 'arquitectura' en el contexto del software?

Respuesta:La arquitectura se refiere al diseño estructural del

Más Libros Gratis
Escanea para descargar
Escúchalo

software que permite que sea fácil de cambiar, preservando así su naturaleza 'blanda'.

4.Pregunta

¿Cuál es la diferencia clave entre 'alcance' y 'forma' en el desarrollo de software?

Respuesta:El alcance se refiere al tamaño o extensión de un cambio, mientras que la forma se refiere a cómo ese cambio se integra en la arquitectura existente. La dificultad para implementar cambios suele estar relacionada con desajustes entre las características solicitadas y la arquitectura del software.

5.Pregunta

¿Qué valor priorizan típicamente los gerentes de negocio: comportamiento o arquitectura? ¿Por qué es esto un error?

Respuesta:Suelen priorizar el comportamiento porque es urgente e importante para la funcionalidad inmediata. Sin embargo, esto es un error porque si se descuida la arquitectura, los cambios futuros se volverán cada vez más difíciles y costosos, afectando en última instancia la

Más Libros Gratis
Escanea para descargar
Escúchalo

viabilidad a largo plazo del sistema.

6.Pregunta

¿Cómo categoriza la Matriz de Eisenhower la importancia y urgencia en el desarrollo de software?

Respuesta:1. Urgente e Importante 2. No Urgente pero Importante 3. Urgente pero No Importante 4. No Urgente y No Importante. La arquitectura encaja dentro de las dos primeras categorías, lo que indica que es crítica para el éxito a largo plazo.

7.Pregunta

¿Cuál es el argumento central sobre la importancia de la arquitectura del software frente al comportamiento?

Respuesta:Una arquitectura bien diseñada (aunque pueda parecer menos urgente) es crucial para permitir que los sistemas se adapten a los requisitos cambiantes, mientras que un sistema que solo se centra en el comportamiento inmediato puede volverse demasiado rígido y eventualmente inútil.

8.Pregunta

¿Qué responsabilidad tienen los desarrolladores y

Más Libros Gratis
Escanea para descargar
Escúchalo

arquitectos de software con respecto a la arquitectura?
Respuesta:Son responsables de abogar por principios arquitectónicos sólidos, enfatizando la necesidad de mantener una estructura sólida incluso cuando se priorizan comportamientos inmediatos por parte de la gestión.

9.Pregunta

¿Qué implica la frase 'luchar por la arquitectura'?
Respuesta:Enfatiza la necesidad de que los desarrolladores de software aboguen por la importancia de una arquitectura consistente y flexible en medio de las presiones para entregar funcionalidad inmediata.

10.Pregunta

¿Por qué argumenta el autor que 'la única forma de ir rápido es ir bien'?
Respuesta:Esta afirmación ilustra que mantener un código limpio y bien estructurado permitirá un desarrollo más rápido y eficiente a largo plazo, contrarrestando la idea errónea común de que apresurar el código puede ahorrar tiempo.

Capítulo 3 | Visión General de los Paradigmas|

Más Libros Gratis
Escanea para descargar
Escúchalo

Preguntas y respuestas

1.Pregunta

¿Qué es la programación estructurada y por qué es significativa en el desarrollo de software?

Respuesta:La programación estructurada impone disciplina en la transferencia directa de control dentro de los programas. Introducida por Edsger Dijkstra, reemplazó los saltos sin restricciones (como las instrucciones goto) por construcciones estructuradas como if/then/else y bucles (do/while). Este paradigma es significativo ya que mejora la claridad y mantenibilidad del programa al imponer un enfoque disciplinado sobre el flujo de control.

2.Pregunta

¿Cómo interactúa la programación orientada a objetos con los principios de la programación estructurada?

Respuesta:La programación orientada a objetos, descubierta por Dahl y Nygaard, se basa en la programación estructurada al imponer disciplina sobre la transferencia indirecta de

Más Libros Gratis
Escanea para descargar
Escúchalo

control a través de construcciones como clases y métodos. Permite que las variables locales persistan después de la ejecución de funciones, lo que apoya el polimorfismo y la encapsulación, mejorando así el diseño del software al promover la reutilización y reducir la complejidad.

3.Pregunta

¿Qué define la programación funcional y qué disciplina única introduce?

Respuesta:La programación funcional, que surge del trabajo de Alonzo Church sobre cálculo lambda, enfatiza la inmutabilidad, lo que significa que los valores simbólicos no cambian, y por lo tanto, generalmente carece de declaraciones de asignación. Esto introduce una disciplina estricta sobre la asignación, llevando a un código más predecible y con menos efectos secundarios, permitiendo un razonamiento más fácil sobre los programas y promoviendo un enfoque funcional para la resolución de problemas.

4.Pregunta

¿Qué patrón general revelan los tres paradigmas de programación sobre capacidades y control en la

Más Libros Gratis
Escanea para descargar
Escúchalo

programación?

Respuesta:Los tres paradigmas—estructurada, orientada a objetos y funcional—consistentemente eliminan ciertas capacidades de los programadores mientras imponen nuevas prácticas disciplinarias. En lugar de añadir capacidades, cada uno impone restricciones (como eliminar instrucciones goto, forzar la encapsulación o limitar la asignación) para mejorar la claridad y mantenibilidad del código. Este patrón indica que un diseño de software efectivo a menudo resulta de imponer estructura y disciplina en lugar de libertad desrestrigida.

5.Pregunta

¿Por qué podrían ser los paradigmas introducidos en el texto los únicos significativos que veremos?

Respuesta:Los paradigmas introducidos—estructurada, orientada a objetos y funcional—emergieron en un corto período de tiempo entre finales de los años 50 hasta 1968, y no se han desarrollado nuevos paradigmas importantes desde entonces. Esto sugiere que la exploración de límites y

Más Libros Gratis
Escanea para descargar
Escúchalo

principios subyacentes en la programación ha alcanzado una saturación, lo que nos lleva a entender que los avances futuros probablemente se construirán sobre estos paradigmas establecidos en lugar de introducir enfoques fundamentalmente nuevos.

6.Pregunta

¿Qué implica el autor sobre la necesidad de disciplina en la arquitectura de software derivada de estos paradigmas?

Respuesta:El autor implica que la disciplina es esencial para mantener una Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software, que se alinea con los principios de los tres paradigmas de programación. La eliminación de ciertas libertades (como saltos no restringidos, punteros a funciones y asignaciones) sirve para guiar a los desarrolladores hacia mejores elecciones arquitectónicas, fomentando sistemas que son manejables, adaptables y, en última instancia, más fáciles de modificar a medida que evolucionan los requisitos.

Más Libros Gratis
Escanea para descargar
Escúchalo

7.Pregunta

¿Cómo influyen los principios de los paradigmas de programación en el desarrollo de la arquitectura de software?

Respuesta:Los principios derivados de los paradigmas de programación influyen en la arquitectura de software al dar forma a cómo se organizan las funciones, imponiendo la separación de componentes y gestionando los datos de manera más efectiva. Por ejemplo, el uso de polimorfismo es crucial para crear arquitecturas modulares que faciliten cambios, mientras que la programación funcional ayuda en el control de acceso a los datos, impulsando en conjunto el diseño hacia sistemas más robustos y mantenibles.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

Escanear para descargar

Leer, Compartir, Empoderar

Completa tu desafío de lectura, dona libros a los niños africanos.

El Concepto

BOOKS FOR AFRICA × ×

Esta actividad de donación de libros se está llevando a cabo junto con Books For Africa. Lanzamos este proyecto porque compartimos la misma creencia que BFA: Para muchos niños en África, el regalo de libros realmente es un regalo de esperanza.

La Regla

Gana 100 puntos → **Canjea un libro** → **Dona a África**

Tu aprendizaje no solo te brinda conocimiento sino que también te permite ganar puntos para causas benéficas. Por cada 100 puntos que ganes, se donará un libro a África.

Prueba gratuita con Bookey

Capítulo 4 | Programación Estructurada| Preguntas y respuestas

1.Pregunta

¿Qué identificó Edsger Dijkstra como un problema significativo con la programación?

Respuesta:Dijkstra reconoció que programar es difícil y que los programadores a menudo no lo hacen bien, ya que los programas complejos contienen demasiados detalles para que el cerebro humano los gestione sin ayuda.

2.Pregunta

¿Cómo propuso Dijkstra resolver los desafíos enfrentados en la programación?

Respuesta:Sugirió aplicar la disciplina matemática de la prueba a la programación, imaginando una jerarquía de postulados y teoremas similar a las matemáticas euclidianas, que los programadores podrían usar para probar que su código es correcto.

3.Pregunta

¿Qué estructuras de control consideró Dijkstra esenciales

Más Libros Gratis
Escanea para descargar
Escúchalo

para crear módulos probables?

Respuesta:Dijkstra identificó que solo un conjunto limitado de estructuras de control—secuencia, selección (if/then/else) e iteración (do/while)—son capaces de permitir que los módulos se descompongan en unidades más pequeñas y probables.

4.Pregunta

¿Cuál fue el impacto de la carta de Dijkstra titulada "Go To Statement Considered Harmful"?

Respuesta:La carta provocó un debate significativo en la comunidad de programación sobre el uso de declaraciones goto, lo que llevó a su eliminación gradual de los lenguajes de programación modernos y al auge de la programación estructurada como un paradigma dominante.

5.Pregunta

¿Qué se entiende por descomposición funcional en la programación estructurada?

Respuesta:La descomposición funcional en la programación estructurada permite descomponer problemas grandes en

Más Libros Gratis
Escanea para descargar
Escúchalo

funciones más pequeñas y manejables, que luego pueden ser descompuestas recursivamente en funciones aún más finas, fomentando una prueba y comprensión más fáciles.

6.Pregunta

¿Qué concluyó Dijkstra sobre las pruebas formales en programación?

Respuesta:La aspiración de Dijkstra por una jerarquía euclidiana completa de funciones probables no se cumplió; la mayoría de los programadores hoy en día no consideran las pruebas formales como prácticas o necesarias para producir software de alta calidad.

7.Pregunta

¿Cómo se relaciona el desarrollo de software con la ciencia, según Dijkstra?

Respuesta:El desarrollo de software, al igual que la ciencia, no puede proporcionar pruebas definitivas de corrección. En cambio, muestra la corrección al intentar demostrar algo incorrecto mientras abraza la idea de que muchas leyes científicas se consideran verdaderas hasta que se demuestre

Más Libros Gratis
Escanea para descargar
Escúchalo

lo contrario.

8.Pregunta

¿Cuál fue una lección crítica de la experiencia de Dijkstra sobre la prueba de software?

Respuesta:Dijkstra afirmó famosamente que las pruebas demuestran la presencia pero no la ausencia de errores; por lo tanto, aunque las pruebas pueden mostrar si un programa es incorrecto, no pueden probar que un programa es correcto.

9.Pregunta

¿Qué exige la programación estructurada de las prácticas de programación según los principios discutidos?

Respuesta:La programación estructurada requiere disciplina y restringe el uso de transferencias de control descontroladas, alentando a los programadores a escribir código más limpio y mantenible a través del uso de estructuras secuenciadas, seleccionadas e iteradas.

10.Pregunta

¿Por qué es valiosa la programación estructurada hoy en día?

Respuesta:La programación estructurada sigue siendo valiosa

Más Libros Gratis
Escanea para descargar
Escúchalo

porque ayuda a crear funciones pequeñas y probables que se pueden probar, asegurando que sean manejables y se puedan validar contra comportamientos esperados.

Capítulo 5 | Programación Orientada a Objetos| Preguntas y respuestas

1.Pregunta

¿Cuál es una definición satisfactoria de la Programación Orientada a Objetos (POO)?

Respuesta:La Programación Orientada a Objetos (POO) es, en esencia, la aplicación combinada de datos y funciones, junto con los principios de encapsulamiento, herencia y polimorfismo.

2.Pregunta

¿Por qué es importante el encapsulamiento en la Programación Orientada a Objetos?

Respuesta:El encapsulamiento es vital porque crea un límite que separa un conjunto cohesivo de datos y funciones, ocultando así el funcionamiento interno y exponiendo solo lo necesario para el mundo exterior.

3.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Cómo difiere el encapsulamiento en la programación orientada a objetos del de los lenguajes de programación procedimentales como C?

Respuesta:Si bien el encapsulamiento existe en lenguajes procedimentales como C a través del uso de archivos de encabezado y declaraciones de struct, los lenguajes POO como C++ rompen la perfecta encapsulación al requerir que las variables miembro se declaren en archivos de encabezado, lo que lleva a una reducción del encapsulamiento.

4.Pregunta

¿Qué proporciona la herencia en los lenguajes Orientados a Objetos?

Respuesta:La herencia facilita la redeclaración de funciones y variables, permitiendo la reutilización del código y la creación de jerarquías en las estructuras de datos. Sin embargo, presenta desafíos como la complicación de dependencias y el aumento de la carga de mantenimiento.

5.Pregunta

¿Cómo sirve el polimorfismo como una característica de

Más Libros Gratis
Escanea para descargar
Escúchalo

la Programación Orientada a Objetos?

Respuesta:El polimorfismo permite que los objetos sean tratados como instancias de su clase padre, permitiendo una llamada a métodos más dinámica donde la implementación específica que se ejecuta se determina en tiempo de ejecución.

6.Pregunta

¿Por qué se considera poderoso al polimorfismo en el diseño de software?

Respuesta:El polimorfismo permite que los sistemas sean más flexibles y mantenibles; facilita la integración de nuevos dispositivos de entrada/salida o características relativamente fácil sin requerir cambios extensos en el código existente.

7.Pregunta

¿Qué es la inversión de dependencias y por qué es significativa?

Respuesta:La inversión de dependencias es un principio que permite invertir la dirección de las dependencias del código fuente en la estructura de un programa, lo que permite una

Más Libros Gratis
Escanea para descargar
Escúchalo

mayor flexibilidad y un diseño modular, promoviendo la desarrollabilidad independiente de los componentes del sistema.

8.Pregunta

¿Se pueden ver los principios de la POO como restrictivos? Si es así, ¿cómo?

Respuesta:Sí, los principios de la POO pueden ser vistos como restrictivos porque imponen disciplina a los desarrolladores al exigir principios estructurales como el encapsulamiento y la herencia, impidiéndoles emplear métodos potencialmente menos organizados pero más simples.

9.Pregunta

¿Cómo apoya la arquitectura POO el despliegue y desarrollo independiente de los componentes de software?

Respuesta:Al establecer interfaces claras entre componentes, la arquitectura POO permite que cada parte se desarrolle, pruebe y despliegue de manera independiente, lo que mejora significativamente la mantenibilidad y escalabilidad de los

Más Libros Gratis
Escanea para descargar
Escúchalo

sistemas de software.

10.Pregunta

¿Cuál puede verse como la conclusión última sobre la esencia de la Programación Orientada a Objetos?

Respuesta:La esencia de la Programación Orientada a Objetos es la capacidad de gestionar la complejidad y las dependencias a través de abstracciones, lo que, en última instancia, fomenta sistemas de software más robustos, adaptables y mantenibles.

Capítulo 6 | Programación Funcional| Preguntas y respuestas

1.Pregunta

¿Cuál es el concepto detrás de la programación funcional y cómo se diferencia de otros paradigmas de programación?

Respuesta:La programación funcional se basa en el concepto de inmutabilidad, lo que significa que una vez que las variables están definidas, no pueden ser cambiadas. Esto difiere de la programación procedural u orientada a objetos, donde las

Más Libros Gratis
Escanea para descargar
Escúchalo

variables pueden ser mutables, creando posibles problemas como condiciones de carrera en entornos concurrentes. En la programación funcional, los cálculos se logran utilizando funciones, y los datos se transforman sin alterar el estado original, lo que lleva a un código más claro y menos propenso a errores.

2.Pregunta

¿Por qué se considera que la inmutabilidad es una consideración arquitectónica importante?

Respuesta:La inmutabilidad es crucial en la arquitectura porque previene condiciones de carrera, bloqueos y otros problemas de concurrencia. Si no hay variables mutables, se eliminan las complejidades asociadas con el estado compartido en aplicaciones multihilo, lo que resulta en sistemas más robustos y predecibles.

3.Pregunta

¿Cuáles son algunas estrategias para implementar la inmutabilidad en la arquitectura de software?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Un enfoque es separar los componentes en elementos inmutables y mutables, permitiendo que los componentes inmutables realicen tareas funcionalmente sin que el estado mutable los interrumpa. Otra estrategia es usar memoria transaccional para proteger las variables mutables de actualizaciones concurrentes de manera efectiva. Esto asegura que el estado mutable se maneje bajo condiciones estrictamente controladas.

4.Pregunta

¿Cómo se relaciona la fuente de eventos con la inmutabilidad?

Respuesta:La fuente de eventos es un método que mantiene el estado de una aplicación almacenando solo la secuencia de eventos que conducen a ese estado en lugar del estado en sí. Este enfoque ejemplifica aún más la inmutabilidad, ya que permite que todos los datos históricos se conserven, minimizando el estado mutable y evitando problemas como actualizaciones concurrentes, ya que los datos históricos no pueden cambiar.

Más Libros Gratis
Escanea para descargar
Escúchalo

5.Pregunta

¿Cuál es la importancia de la comparación entre Java y Clojure en el manejo de variables?

Respuesta:La comparación resalta la diferencia fundamental en el manejo de variables; Java emplea variables mutables que pueden llevar a problemas de estado mutable, mientras que Clojure ejemplifica los principios de la programación funcional al usar variables inmutables. Las funciones de Clojure operan sobre datos de entrada sin modificarlos, mostrando los beneficios de la programación funcional para evitar problemas comunes de concurrencia.

6.Pregunta

¿Cómo proporciona eficiencia el concepto de 'listas interminables' en la programación funcional?

Respuesta:En la programación funcional, construcciones como 'listas interminables' permiten la evaluación perezosa. Solo se computan y acceden a los elementos necesarios cuando se necesitan, lo que ahorra recursos y mejora la eficiencia. Esto significa que los cálculos se posponen hasta

Más Libros Gratis
Escanea para descargar
Escúchalo

que son necesarios, minimizando el procesamiento innecesario o el uso de memoria durante la ejecución.

7.Pregunta

¿Cuáles son las posibles limitaciones para lograr una inmutabilidad total en las aplicaciones de software?

Respuesta:Si bien la inmutabilidad puede mejorar significativamente la robustez de las aplicaciones, lograrla puede no ser práctico debido a limitaciones en recursos de almacenamiento y procesamiento. Los desarrolladores pueden tener que comprometerse utilizando estados mutables de manera controlada o aplicando patrones como la fuente de eventos para mantener el rendimiento sin requerir recursos infinitos.

8.Pregunta

¿Por qué los desarrolladores podrían subestimar la importancia de la arquitectura sobre la funcionalidad inmediata?

Respuesta:Los desarrolladores a menudo priorizan la urgencia en la entrega de características para satisfacer necesidades inmediatas del mercado, pasando por alto la

Más Libros Gratis
Escanea para descargar
Escúchalo

importancia a largo plazo de la arquitectura. Esto puede llevar a una deuda técnica, ya que la organización estructural del código sufre, haciendo que futuros cambios sean más complejos y costosos, lo que en última instancia reduce la productividad en general.

9.Pregunta

¿Cómo se relaciona la disciplina de la programación estructurada con la programación funcional?

Respuesta:La programación estructurada impone una disciplina sobre el flujo de control de los programas, requiriendo una estructura clara y lógica que ayuda a la corrección del programa. La programación funcional se basa en esto al imponer disciplina sobre las asignaciones de variables, reforzando aún más la claridad y predictibilidad en la ejecución del código, lo que lleva a un mantenimiento más simple y a la reducción de errores.

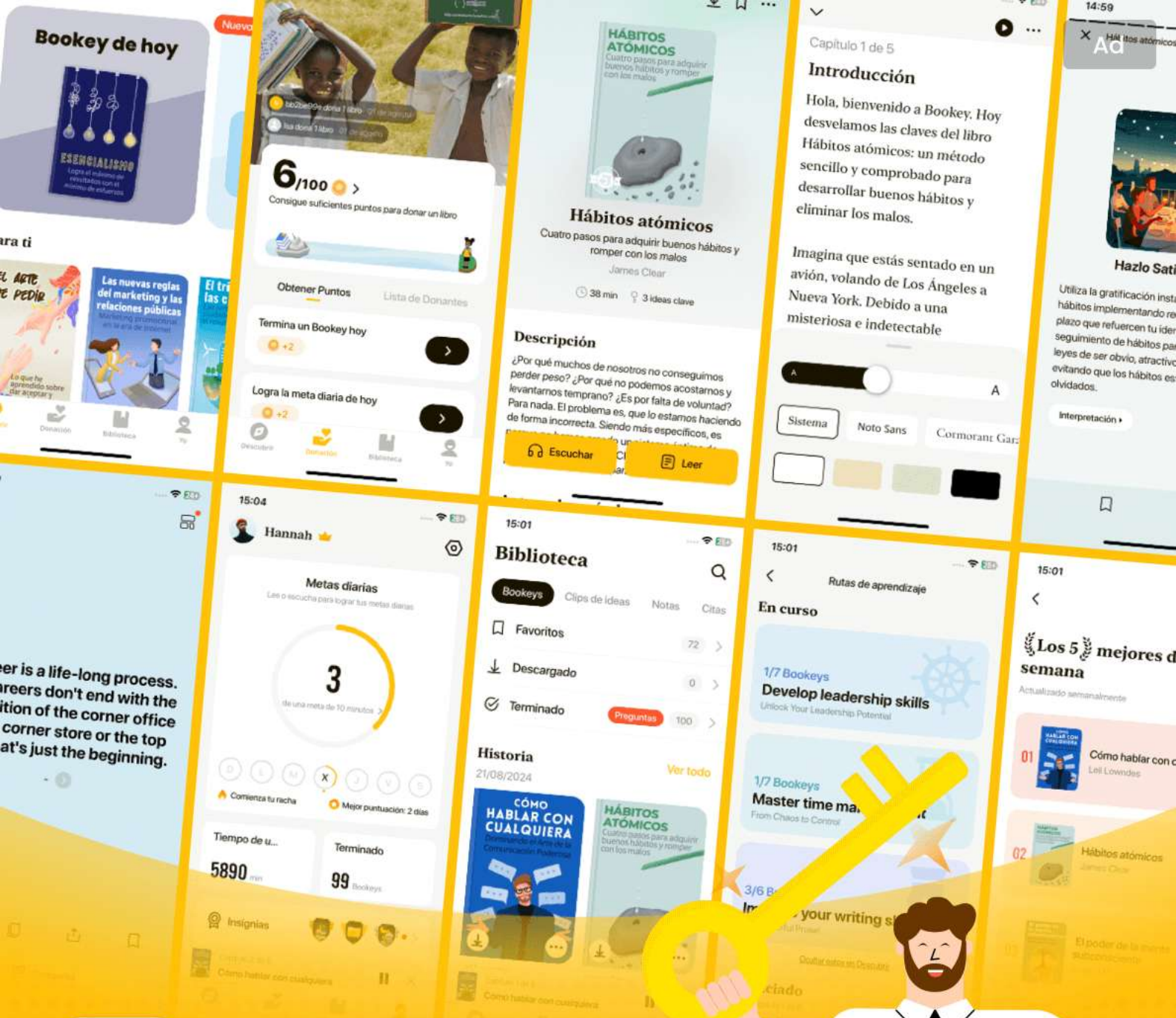
10.Pregunta

¿Qué se puede decir sobre la evolución y adaptabilidad de la arquitectura de software según el texto?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:La evolución de la arquitectura de software refleja una adaptación gradual hacia la imposición de más disciplina en las prácticas de codificación. A pesar de los avances tecnológicos, los principios fundamentales del software permanecen estáticos a lo largo de las décadas. La capacidad de gestionar la complejidad a través de estructuras como la inmutabilidad, la programación funcional y otros paradigmas mejora la simplificación del mantenimiento del código, sin embargo, las construcciones subyacentes de la programación no han cambiado drásticamente.

Más Libros Gratis
Escanea para descargar
Escúchalo



Las mejores ideas del mundo desbloquean tu potencial

Prueba gratuita con Bookey

Escanear para descargar

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 7 | SRP: El Principio de Responsabilidad Única| Preguntas y respuestas

1.Pregunta

¿Cuál es el Principio de Responsabilidad Única (SRP) según lo definido en el capítulo?

Respuesta:El Principio de Responsabilidad Única establece que un módulo debe tener una, y solo una, razón para cambiar. Esto significa que un módulo debe ser responsable ante un único actor.

2.Pregunta

¿Por qué se considera inapropiado el nombre 'Principio de Responsabilidad Única'?

Respuesta:El nombre puede llevar a los programadores a pensar que significa que cada módulo debe hacer solo una cosa, en lugar de entender que se refiere a que un módulo debe tener una razón para cambiar.

3.Pregunta

¿Cuál es un ejemplo de violación del SRP en un módulo de software?

Respuesta:Un ejemplo es una clase Empleado con métodos

Más Libros Gratis
Escanea para descargar
Escúchalo

para calcular el pago, reportar horas y guardar datos, que sirve a tres actores diferentes: el CFO, COO y CTO. Esto lleva a un acoplamiento no intencionado entre funciones responsables de diferentes actores.

4.Pregunta

¿Cuál es un síntoma común de violación del SRP como se discute en el capítulo?

Respuesta:Un síntoma común es la duplicación accidental, donde los cambios realizados para un actor afectan unintencionadamente a otro actor debido al código compartido en un solo módulo.

5.Pregunta

¿Cómo explica el capítulo los riesgos de fusionar archivos fuente que violan el SRP?

Respuesta:Las fusiones pueden ocurrir cuando diferentes desarrolladores de equipos responsables de diferentes actores cambian el mismo archivo fuente, lo que lleva a posibles conflictos, errores y un mayor riesgo para las partes interesadas.

Más Libros Gratis
Escanea para descargar
Escúchalo

6.Pregunta

¿Qué soluciones sugiere el capítulo para adherirse al SRP?

Respuesta:Las soluciones incluyen separar funcionalidades en diferentes clases, mantener métodos y estructuras de datos relacionados cohesivos, y utilizar patrones como Facade para gestionar la complejidad y las interacciones.

7.Pregunta

¿Cuál podría ser un inconveniente de separar funciones en diferentes clases para mantener el SRP?

Respuesta:El inconveniente es que los desarrolladores pueden necesitar rastrear e instanciar múltiples clases, aumentando la complejidad en términos de código repetitivo y gestión.

8.Pregunta

¿Cómo se relaciona el SRP con otros principios de diseño?

Respuesta:El SRP vuelve a aparecer en niveles más altos de la arquitectura del software, como el Principio de Cierre Común a nivel de componentes y afecta el Eje de Cambio a

Más Libros Gratis
Escanea para descargar
Escúchalo

nivel arquitectónico.

9.Pregunta

¿Qué implica el capítulo sobre la importancia del SRP en el diseño de software?

Respuesta:Adherirse al SRP es crucial ya que minimiza las interdependencias entre diferentes módulos, haciendo que el software sea más fácil de cambiar y gestionar para varias partes interesadas.

10.Pregunta

¿Puedes resumir la idea principal sobre el SRP del capítulo?

Respuesta:La idea principal es que comprender y aplicar el Principio de Responsabilidad Única ayuda a crear módulos de software que son cohesivos y débilmente acoplados, reduciendo la probabilidad de consecuencias no intencionadas al realizar cambios.

Capítulo 8 | OCP: El Principio Abierto-Cerrado| Preguntas y respuestas

1.Pregunta

¿Qué es el Principio Abierto-Cerrado (OCP)?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:El Principio Abierto-Cerrado establece que un artefacto de software debe estar abierto a extensiones pero cerrado a modificaciones. Esto significa que el comportamiento del software puede ser extendido sin cambiar el código existente.

2.Pregunta

¿Cómo se relaciona el OCP con la arquitectura de software?

Respuesta:El OCP enfatiza la importancia de diseñar sistemas de software de manera que se permita realizar extensiones fácilmente sin requerir modificaciones significativas, lo cual es crucial para mantener una arquitectura sostenible y adaptable.

3.Pregunta

¿Puedes proporcionar un experimento mental para ilustrar el OCP?

Respuesta:Imagina un sistema que muestra datos financieros. Si las partes interesadas solicitan un cambio en el formato de estos datos para un informe impreso, un sistema bien

Más Libros Gratis
Escanea para descargar
Escúchalo

arquitectado permitiría este cambio con una alteración mínima o nula del código existente, demostrando la eficacia del OCP.

4.Pregunta

¿Qué papel juega el Principio de Responsabilidad Única (SRP) en la consecución del OCP?

Respuesta:Al aplicar el SRP, podemos separar diferentes responsabilidades dentro de un sistema, lo que asegura que los cambios por una razón no afecten adversamente a otras, apoyando extensiones más fáciles según lo establece el OCP.

5.Pregunta

¿Cómo se relacionan las dependencias en una arquitectura de software con el OCP?

Respuesta:Las dependencias deben organizarse de tal manera que los componentes de alto nivel estén protegidos de los cambios en los componentes de bajo nivel. Así, modificar componentes de bajo nivel no requiere cambios en los de alto nivel.

6.Pregunta

¿Cuál es la importancia de las dependencias

Más Libros Gratis
Escanea para descargar
Escúchalo

unidireccionales en los componentes de software?
Respuesta:Las dependencias unidireccionales ayudan a mantener el control sobre la arquitectura de un sistema; si el componente A debe estar protegido de los cambios en el componente B, entonces B debe depender de A, y no al revés.

7.Pregunta

¿Por qué se considera que el Interactor es central en la arquitectura en el contexto del OCP?
Respuesta:El Interactor contiene las reglas de negocio y está posicionado para verse menos afectado por cambios en otros componentes, lo que lo convierte en un elemento crítico para preservar la integridad arquitectónica del sistema.

8.Pregunta

¿Qué implica el control direccional en el contexto del OCP y el diseño de software?
Respuesta:El control direccional implica estructurar correctamente las dependencias para que apunten hacia componentes de alto nivel. Esto previene impactos

Más Libros Gratis
Escanea para descargar
Escúchalo

innecesarios de cambios en componentes de bajo nivel.

9.Pregunta

¿Cómo contribuye el ocultamiento de información al cumplimiento del OCP?

Respuesta:El ocultamiento de información protege a los componentes de necesitar conocer el funcionamiento interno de otros componentes, reduciendo así las dependencias transitorias y permitiendo extensiones sin riesgo de romper la funcionalidad existente.

10.Pregunta

¿De qué manera busca el OCP minimizar el impacto del cambio en un sistema?

Respuesta:El OCP minimiza el impacto del cambio promoviendo un diseño que permite añadir nueva funcionalidad a través de nuevo código en lugar de alterar el código existente, reduciendo posibles errores e inestabilidad en el sistema.

Capítulo 9 | LSP: El Principio de Sustitución de Liskov| Preguntas y respuestas

1.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Qué es el Principio de Sustitución de Liskov (LSP)?

Respuesta:El Principio de Sustitución de Liskov establece que si para cada objeto o1 del tipo S, existe un objeto o2 del tipo T tal que para todos los programas P definidos en términos de T, el comportamiento de P no cambia cuando se sustituye o1 por o2, entonces S es un subtipo de T.

2.Pregunta

¿Puedes proporcionar un ejemplo de LSP en el diseño de software?

Respuesta:Un ejemplo es usar una clase llamada Licencia con subtipos LicenciaPersonal y LicenciaEmpresarial. La aplicación de facturación llama al método calcFee() en el tipo Licencia, y si funciona correctamente sin saber si se trata de una LicenciaPersonal o LicenciaEmpresarial, entonces los subtipos se ajustan al LSP.

3.Pregunta

¿Cuál es el problema del Cuadrado/Rectángulo y cómo viola el LSP?

Más Libros Gratis

Escanea para descargar

Escúchalo

Respuesta:El problema del Cuadrado/Rectángulo ocurre porque un Cuadrado, que requiere que tanto el ancho como la altura cambien juntos, se trata como un subtipo de Rectángulo, que permite cambios independientes en ancho y altura. Esto lleva a inconsistencias cuando se espera el comportamiento de un Rectángulo y se entrega un Cuadrado, violando la sustituibilidad.

4.Pregunta

¿Cómo ha evolucionado la comprensión del LSP de ser sobre la herencia a abarcar un diseño de software más amplio?

Respuesta:Inicialmente, el LSP se veía como un principio que guiaba la herencia, pero se ha ampliado para abarcar el diseño de interfaces e implementaciones, asegurando que los sistemas se basen en interfaces bien definidas y que las implementaciones sean sustituibles.

5.Pregunta

¿Qué puede suceder arquitectónicamente cuando se viola el LSP?

Respuesta:Las violaciones del LSP pueden llevar a que los

Más Libros Gratis
Escanea para descargar
Escúchalo

sistemas se vuelvan excesivamente complicados, requiriendo mecanismos adicionales (como condicionales para diferentes tipos) que degradan la arquitectura, aumentan los errores y complican el mantenimiento.

6.Pregunta

¿Puedes describir un ejemplo práctico de violación del LSP con un sistema de despacho de taxis?

Respuesta:En un servicio de despacho de taxis, si una API REST para el despacho no se ajusta a un formato consistente entre los servicios (por ejemplo, taxis Acme usando 'dest' en lugar de 'destination'), se necesita agregar código especial para manejar estas excepciones, complicando la arquitectura y llevando a posibles errores.

7.Pregunta

¿Cuál es la principal conclusión sobre el impacto de adherirse al LSP durante el desarrollo de software?

Respuesta:Adherirse al LSP conduce a un diseño de sistema más limpio y sencillo donde los componentes se pueden intercambiar fácilmente sin modificar el código existente,

Más Libros Gratis
Escanea para descargar
Escúchalo

mejorando el mantenimiento, reduciendo errores y asegurando una arquitectura más robusta.

8.Pregunta

¿Por qué es crucial entender el LSP para los arquitectos de software?

Respuesta:Entender el LSP ayuda a los arquitectos a diseñar sistemas que sean adaptables al cambio sin un redearrollo significativo. Asegura que los componentes sean intercambiables, lo que minimiza las intersecciones entre funcionalidades diferentes y, por tanto, mejora la integridad general del sistema.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

Escanear para descargar

Prueba la aplicación Bookey para leer más de 1000 resúmenes de los mejores libros del mundo

Desbloquea de 1000+ títulos, 80+ temas

Nuevos títulos añadidos cada semana

Brand | Liderazgo & Colaboración | Gestión del tiempo | Relaciones & Comunicación | Kno

ategia Empresarial | Creatividad | Memorias | Dinero e Inversiones | Conózcase a sí mismo

nprendimiento | Historia del mundo | Comunicación entre Padres e Hijos | Autocuidado | M

Perspectivas de los mejores libros del mundo

Prueba gratuita con Bookey

Capítulo 10 | ISP: El Principio de Segregación de Interfaces| Preguntas y respuestas

1.Pregunta

¿Cuál es la esencia del Principio de Segregación de Interface (ISP)?

Respuesta:La esencia del Principio de Segregación de Interface (ISP) es descomponer una interfaz más grande en interfaces más pequeñas y específicas que satisfacen las necesidades de diferentes clientes. Esta separación evita que los clientes se vean obligados a depender de métodos que no utilizan, reduciendo así la recompilación innecesaria y manteniendo una arquitectura limpia.

2.Pregunta

¿Cómo se relaciona el ISP con los lenguajes de programación estáticamente tipados como Java en comparación con los lenguajes de tipado dinámico como Python?

Respuesta:En lenguajes estáticamente tipados como Java, las dependencias son explícitas y requieren declaraciones, lo que

Más Libros Gratis
Escanea para descargar
Escúchalo

puede llevar a que los clientes recompilen cuando cambian métodos no relacionados. En lenguajes de tipado dinámico como Python, las dependencias se infieren en tiempo de ejecución, lo que permite una mayor flexibilidad y menos problemas de acoplamiento.

3.Pregunta

¿Por qué es dañino que un sistema dependa de módulos que contengan más de lo necesario?

Respuesta:Es dañino porque tales dependencias pueden llevar a complicaciones innecesarias, como la recompilación y redeploying forzados de componentes aparentemente no relacionados cuando se realizan cambios, lo que puede aumentar el riesgo de introducir errores y reducir la estabilidad general del sistema.

4.Pregunta

Ilustra cómo el ISP puede prevenir la recompilación innecesaria con un ejemplo.

Respuesta:Imagina una clase OPS con métodos op1, op2 y op3. Si el Usuario1 solo necesita op1, sin el ISP, cualquier

Más Libros Gratis
Escanea para descargar
Escúchalo

cambio en op2 o op3 obliga a Usuario1 a recompilar. Con el ISP, Usuario1 solo dependería de la interfaz U1Ops que expone op1, aislando a Usuario1 de los cambios en op2 y op3.

5.Pregunta

¿Qué preocupaciones arquitectónicas más amplias están ligadas al ISP?

Respuesta:El ISP se extiende más allá de las interfaces a las dependencias arquitectónicas. Un arquitecto debe asegurarse de que los componentes no dependan de marcos o módulos con características que no utilizan. Esto ayuda a prevenir problemas como redeployments innecesarios o fallos sistémicos debido a cambios en funciones no relacionadas.

6.Pregunta

¿Cuál es la lección aprendida al no adherirse al ISP en la arquitectura de sistemas?

Respuesta:La lección es que depender de módulos con funcionalidad no relacionada puede crear complicaciones y dependencias imprevistas, llevando a un aumento en los

Más Libros Gratis
Escanea para descargar
Escúchalo

costos de mantenimiento y limitando la flexibilidad en futuros cambios del sistema.

7.Pregunta

¿Cómo se alinea el ISP con las prácticas de desarrollo de software sostenible?

Respuesta:El ISP promueve una arquitectura limpia y mantenible al reducir dependencias innecesarias, permitiendo que los equipos adapten y extiendan el software de manera más eficiente a medida que evolucionan los requisitos, lo cual es esencial para el desarrollo de software sostenible.

8.Pregunta

¿Qué implicaciones tiene el ISP para la modularidad del software?

Respuesta:El ISP promueve una alta modularidad al fomentar la creación de interfaces enfocadas, lo que facilita que los sistemas sean más comprensibles, probables y mantenibles. Cada módulo se convierte en un componente bien definido que puede desarrollarse o modificarse de manera independiente.

Más Libros Gratis
Escanea para descargar
Escúchalo

9.Pregunta

¿Por qué deberían los arquitectos de software considerar el ISP en sus diseños?

Respuesta:Los arquitectos de software deberían considerar el ISP para crear arquitecturas que sean resilientes al cambio, minimizar el retrabajo y los defectos, y permitir una adaptación más fácil en respuesta a los requisitos comerciales en evolución.

Capítulo 11 | DIP: El Principio de Inversión de Dependencias| Preguntas y respuestas

1.Pregunta

¿Qué es el Principio de Inversión de Dependencias (DIP)?

Respuesta:El Principio de Inversión de Dependencias (DIP) sostiene que las dependencias del código fuente deben apuntar solo a abstracciones (como interfaces o clases abstractas) y no a implementaciones concretas. Este enfoque busca crear sistemas más flexibles al minimizar las dependencias en clases concretas.

2.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Por qué las dependencias del código fuente deben referirse a abstracciones?

Respuesta:Referenciar abstracciones permite que los sistemas se vean menos afectados por los cambios en las implementaciones concretas, lo que lleva a una mayor estabilidad y flexibilidad en el diseño del software.

3.Pregunta

¿Cómo ayuda el DIP a gestionar los cambios en el software?

Respuesta:El DIP facilita el manejo de cambios al definir límites claros entre las abstracciones de alto nivel y los detalles de bajo nivel, asegurando que los cambios en una parte del sistema no requieran cambios en toda la arquitectura.

4.Pregunta

¿Qué papel juegan las abstracciones estables en la arquitectura del software según el DIP?

Respuesta:Las abstracciones estables son menos volátiles que las implementaciones concretas; por lo tanto, deben ser

Más Libros Gratis
Escanea para descargar
Escúchalo

favorecidas. Los cambios en las interfaces abstractas no deberían requerir idealmente cambios en sus implementaciones concretas, reduciendo así el riesgo de fallos durante el desarrollo.

5.Pregunta

¿Qué es un ejemplo de una clase concreta que se puede usar sin violar el DIP?

Respuesta:La clase String en Java puede verse como una clase concreta que es estable y se utiliza sin violar el DIP, ya que sus cambios son raros y están controlados de manera estricta.

6.Pregunta

¿Qué es una Fábrica Abstracta en el contexto del DIP?

Respuesta:Una Fábrica Abstracta es un patrón de diseño que proporciona una interfaz para crear objetos, pero permite que las subclases alteren el tipo de objetos que se crearán. En el DIP, ayuda a gestionar las dependencias sin depender directamente de clases concretas específicas.

7.Pregunta

¿Cómo se relaciona el uso de una Fábrica Abstracta con

Más Libros Gratis
Escanea para descargar
Escúchalo

el flujo de control y las dependencias?

Respuesta:En una Fábrica Abstracta, el flujo de control podría utilizar una interfaz para instanciar implementaciones concretas, permitiendo que las dependencias fluyan en dirección de las abstracciones en lugar de las concreciones, manteniendo así los principios de la Inversión de Dependencias.

8.Pregunta

¿Cuál es la importancia de no referenciar clases concretas volátiles según el DIP?

Respuesta:No referenciar clases concretas volátiles ayuda a prevenir la introducción de dependencias que cambian con frecuencia, lo que a su vez mejora la mantenibilidad y estabilidad de la base de código.

9.Pregunta

¿Qué conclusión se puede extraer sobre la importancia de gestionar dependencias en la arquitectura del software?

Respuesta:Gestionar las dependencias de manera efectiva a través de principios como el DIP es crucial para crear

Más Libros Gratis
Escanea para descargar
Escúchalo

arquitecturas de software limpias, mantenibles y flexibles, asegurando que los cambios en los sistemas de bajo nivel no se propaguen a las reglas de negocio de alto nivel.

10.Pregunta

¿Qué implica el término 'componentes concretos volátiles' en el contexto del DIP?

Respuesta:Los componentes concretos volátiles son aquellas partes del software que se cambian o modifican con frecuencia; bajo el DIP, el objetivo es minimizar las dependencias en estos componentes para garantizar la estabilidad general del sistema.

Capítulo 12 | Componentes| Preguntas y respuestas

1.Pregunta

¿Qué son los componentes en la arquitectura de software?

Respuesta:Los componentes son las unidades fundamentales de despliegue en los sistemas de software. Pueden ser desplegados de forma independiente, lo que significa que pueden ser desarrollados y mantenidos de manera aislada. En

Más Libros Gratis
Escanea para descargar
Escúchalo

lenguajes como Java, los componentes suelen representarse como archivos jar; en .NET, como DLL; y en Ruby, como gems. También pueden empaquetarse en archivos como .war o desplegarse como plugins independientes.

2.Pregunta

¿Por qué es esencial que los componentes sean desplegables de forma independiente?

Respuesta:Los componentes desplegables de forma independiente permiten una mayor flexibilidad y agilidad en el desarrollo de software. Si un componente puede ser actualizado o reemplazado sin afectar a todo el sistema, se minimiza el riesgo de introducir errores en la aplicación y se facilita un mantenimiento más sencillo y despliegues de funcionalidades más rápidos.

3.Pregunta

¿Qué desafíos históricos enfrentaron los primeros programadores con respecto al uso de la memoria?

Respuesta:Los primeros programadores tenían que gestionar

Más Libros Gratis
Escanea para descargar
Escúchalo

la memoria manualmente, especificando direcciones de memoria y lidiando con programas no relocables. Compilar bibliotecas requería integrar directamente el código fuente, ya que la memoria era limitada, lo que llevaba a tiempos de compilación lentos. Enfrentaron desafíos cuando el tamaño de las aplicaciones crecía, necesitando una gestión efectiva de la memoria más allá de sus asignaciones iniciales.

4.Pregunta

¿Cómo mejoró la introducción de binarios relocables el proceso de desarrollo de software?

Respuesta:Los binarios relocables permitieron que el código se cargara en memoria en cualquier dirección sin necesidad de modificar el código en sí, acelerando significativamente el proceso de enlace. Esta innovación significó que los programadores podían cargar solo las funciones que necesitaban, reduciendo el tiempo de compilación total y permitiendo aplicaciones más complejas.

5.Pregunta

¿Qué impacto tuvo la ley de Moore en los tiempos de programación y enlace en los años 90?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:A medida que el hardware mejoraba bajo la ley de Moore, con discos más rápidos y memoria más barata, el tiempo requerido para el enlace comenzó a disminuir a meros segundos para trabajos más pequeños. Este avance permitió a los desarrolladores adoptar un enfoque de enlace más dinámico, allanando el camino para arquitecturas de plugins y ciclos de desarrollo rápidos.

6.Pregunta

¿Cuál es la importancia de los cargadores de enlace en el desarrollo histórico de software?

Respuesta:Los cargadores de enlace desempeñaron un papel crucial en la separación de los procesos de carga y enlace en las aplicaciones de software. Esto permitió una ejecución más rápida de los programas, ya que el enlace podía hacerse por adelantado, reduciendo significativamente los tiempos de carga al ejecutar aplicaciones grandes y permitiendo un uso eficiente de los recursos del sistema.

7.Pregunta

¿Qué problemas surgieron del uso a gran escala de los cargadores de enlace en los años 60 y 70?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:A medida que los sistemas de software crecían, los cargadores de enlace comenzaron a tener problemas con la velocidad, tardando excesivo tiempo en resolver muchas referencias externas de grandes bibliotecas almacenadas en dispositivos lentos como cintas magnéticas. Esto requería un diseño más sofisticado en los procesos de desarrollo de software para gestionar efectivamente el uso de recursos.

8.Pregunta

¿Qué destaca el capítulo como una práctica moderna en el diseño de componentes?

Respuesta:El capítulo enfatiza que las mejores prácticas actuales en el diseño de componentes se centran en crear componentes que no solo sean independientes, sino también fáciles de usar y combinar, permitiendo mayor flexibilidad y modularidad en las aplicaciones, como el uso de archivos .jar o DLL como plugins.

9.Pregunta

¿Qué significa 'la única forma de ir rápido es ir bien' en el contexto del desarrollo de software?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Esta frase encapsula la filosofía de que mantener un código limpio y bien estructurado es crítico para la eficiencia a largo plazo. Apresurarse a entregar funcionalidades sin una arquitectura adecuada lleva a un código desordenado, lo que en última instancia ralentiza el desarrollo y aumenta la dificultad al realizar cambios o agregar características.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

Escanear para descargar

Por qué Bookey es una aplicación imprescindible para los amantes de los libros

Contenido de 30min
Cuanto más profunda y clara sea la interpretación que proporcionamos, mejor comprensión tendrás de cada título.

Formato de texto y audio
Absorbe conocimiento incluso en tiempo fragmentado.

Preguntas
Comprueba si has dominado lo que acabas de aprender.

Y más
Múltiples voces y fuentes, Mapa mental, Citas, Clips de ideas...

Prueba gratuita con Bookey

Capítulo 13 | Cohesión de Componentes| Preguntas y respuestas

1.Pregunta

¿Cuáles son los tres principios de cohesión de componentes discutidos en el capítulo 13 de Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software?

Respuesta:Los tres principios de cohesión de componentes discutidos son: 1. El Principio de Equivalencia de Reutilización/Liberación (REP) 2. El Principio de Cierre Común (CCP) 3. El Principio de Reutilización Común (CRP).

2.Pregunta

¿Cómo guía el Principio de Equivalencia de Reutilización/Liberación (REP) el diseño de componentes?

Respuesta:El REP establece que la granularidad de la reutilización debe coincidir con la granularidad de la liberación, lo que significa que las clases agrupadas en componentes deben ser cohesivas y liberables juntas. Esto

Más Libros Gratis
Escanea para descargar
Escúchalo

maximiza la compatibilidad e informa a los desarrolladores sobre los cambios cuando se actualizan los componentes, facilitando así una mejor gestión de las versiones de los componentes.

3.Pregunta

¿Puedes explicar el Principio de Cierre Común (CCP)?

Respuesta:El CCP enfatiza reunir clases que cambian juntas por las mismas razones en el mismo componente, mientras que separa clases que cambian por diferentes razones en diferentes componentes. Esto minimiza la necesidad de redeplegar a través de múltiples componentes y mejora la mantenibilidad al consolidar los cambios.

4.Pregunta

¿Cuál es la importancia del Principio de Reutilización Común (CRP)?

Respuesta:El CRP se centra en garantizar que los componentes consistan en clases que se reutilizan juntas, evitando dependencias innecesarias entre clases que no están estrechamente relacionadas. Afirma que los usuarios de un

Más Libros Gratis
Escanea para descargar
Escúchalo

componente no deben verse obligados a depender de clases no utilizadas, optimizando así el proceso de reutilización.

5.Pregunta

¿Cómo crea la interacción del REP, CCP y CRP tensión en la arquitectura de componentes?

Respuesta:Estos principios a menudo se contradicen: el REP y el CCP tienden a crear componentes más grandes para una mejor reutilización y mantenimiento, mientras que el CRP aboga por componentes más pequeños y enfocados. Esta tensión requiere que los arquitectos encuentren un equilibrio basado en los objetivos actuales del equipo de desarrollo, reconociendo que este equilibrio puede cambiar con el tiempo.

6.Pregunta

¿Por qué es importante agrupar clases en componentes basándose en propósitos cohesivos?

Respuesta:Agrupar clases en componentes cohesivos asegura que las clases trabajen juntas de manera armoniosa bajo un tema común, lo que facilita una gestión más sencilla de los

Más Libros Gratis
Escanea para descargar
Escúchalo

cambios, reduce la complejidad durante las actualizaciones y mejora la mantenibilidad general del sistema.

7.Pregunta

¿De qué manera puede evolucionar el enfoque en la cohesión de componentes a medida que un proyecto madura?

Respuesta:A medida que un proyecto madura, el enfoque inicial podría priorizar la mantenibilidad (Principio de Cierre Común), pero a medida que la base de código se vuelve más estable, el énfasis puede cambiar hacia la reutilización (Principio de Equivalencia de Reutilización/Liberación), lo que lleva a ajustes en la estructura de los componentes a lo largo del tiempo.

Capítulo 14 | Acoplamiento de Componentes| Preguntas y respuestas

1.Pregunta

¿Qué es el Principio de Dependencias A cíclicas (ADP)?

Respuesta:El Principio de Dependencias A cíclicas establece que no debería haber ciclos en el gráfico de dependencias de los componentes. Esto significa que

Más Libros Gratis
Escanea para descargar
Escúchalo

en un sistema bien diseñado, deberías ser capaz de rastrear las dependencias de los componentes sin regresar jamás al componente inicial. Esto previene problemas como el 'síndrome de la resaca', donde los cambios realizados por un desarrollador interrumpen el trabajo de otro.

2.Pregunta

¿Cómo funciona el método de compilación semanal y cuáles son sus desventajas?

Respuesta:En el método de compilación semanal, los desarrolladores trabajan en aislamiento durante la mayor parte de la semana e integran sus cambios solo al final de la semana. Si bien esto permite un trabajo enfocado, genera una carga significativa de integración al final de la semana. A medida que aumenta el tamaño del proyecto, la integración puede llevar a retrasos prolongados y frustración, extendiéndose hasta los fines de semana y afectando la eficiencia general del equipo.

3.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Cómo puedes eliminar ciclos de dependencia en la arquitectura de componentes?

Respuesta:Puedes eliminar los ciclos de dependencia implementando el Principio de Inversión de Dependencias (DIP), que consiste en crear abstracciones o interfaces que desacoplen los componentes dependientes. Alternativamente, puedes crear nuevos componentes de los cuales dependen ambas dependencias en conflicto, rompiendo así el ciclo.

4.Pregunta

¿Qué es el Principio de Dependencias Estables (SDP)?

Respuesta:El Principio de Dependencias Estables afirma que las dependencias deben fluir hacia componentes estables. Un componente que es estable tiene menos dependencias volátiles y es poco probable que cambie con frecuencia, mientras que los componentes volátiles no deben ser dependidos por los estables para mantener la flexibilidad general y evitar interrumpir la funcionalidad estable.

5.Pregunta

¿Qué se entiende por estabilidad en la arquitectura de

Más Libros Gratis
Escanea para descargar
Escúchalo

componentes?

Respuesta:La estabilidad se refiere a qué tan probable es que un componente cambie. La estabilidad puede medirse por el número de componentes que dependen de él: un componente estable tiene muchas dependencias, lo que dificulta los cambios, mientras que un componente inestable tiene pocas o ninguna dependencia, permitiendo modificaciones fácilmente.

6.Pregunta

¿Cómo se correlacionan los conceptos de estabilidad y abstracto según el Principio de Abstracciones Estables?

Respuesta:El Principio de Abstracciones Estables establece que un componente estable debe ser abstracto y un componente inestable debe ser concreto. Esto asegura que los componentes estables se mantengan adaptables y flexibles al cambio sin estar atados por implementaciones innecesarias.

7.Pregunta

¿Por qué es importante permitir la desplegabilidad independiente en los componentes?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:La desplegabilidad independiente permite actualizar o modificar componentes sin necesidad de redeplegar todo el sistema. Esto reduce el tiempo de inactividad y el esfuerzo necesario para gestionar actualizaciones, permitiendo una mejor capacidad de respuesta a cambios y procesos de desarrollo más eficientes.

8.Pregunta

¿Cuál es la relación entre el Principio de Cierre Común (CCP) y el Principio de Responsabilidad Única (SRP)?

Respuesta:El Principio de Cierre Común es la versión de componentes del Principio de Responsabilidad Única. Mientras que el SRP busca asegurar que una clase tenga una única razón para cambiar, el CCP aconseja que los componentes solo deben contener clases que cambian por las mismas razones, ayudando a localizar cambios y minimizar el impacto de las modificaciones.

9.Pregunta

¿Qué afirma el Principio de Reutilización Común (CRP) sobre la organización de clases en los componentes?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:El Principio de Reutilización Común establece que las clases que se reutilizan juntas deben ubicarse en el mismo componente. Por el contrario, las clases que no es probable que se reutilicen juntas no deben compartir un componente; de lo contrario, pueden surgir dependencias innecesarias que complican el mantenimiento y las actualizaciones.

10.Pregunta

¿Cuáles son las implicaciones de violar el principio de dependencias a cíclicas?

Respuesta:Violar el principio de dependencias a cíclicas lleva a una mayor complejidad y riesgos en la arquitectura, como la incapacidad para aislar componentes para pruebas, dificultad para establecer un orden claro de compilaciones y una tendencia a crear un acoplamiento fuerte entre componentes, lo que a su vez invita a posibles problemas de integración.

Capítulo 15 | ¿Qué es la arquitectura?| Preguntas y respuestas

1.Pregunta

¿Cuál es el papel principal de un arquitecto de software?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Un arquitecto de software es, ante todo, un programador que continúa escribiendo código mientras guía a su equipo hacia un diseño que maximiza la productividad. No se alejan de la programación; sus experiencias con el código les ayudan a entender y abordar problemas dentro del equipo.

2.Pregunta

¿Cuál es el propósito principal de la arquitectura de software?

Respuesta:El propósito principal de la arquitectura de software es facilitar el desarrollo, despliegue, operación y mantenimiento de un sistema de software. Su objetivo es minimizar el costo de vida del sistema y maximizar la productividad del programador.

3.Pregunta

¿Cómo afecta la arquitectura a la longevidad y mantenibilidad de un sistema de software?

Respuesta:Una buena arquitectura hace que un sistema de

Más Libros Gratis
Escanea para descargar
Escúchalo

software sea fácil de entender, desarrollar, mantener y desplegar. Soporta cambios futuros, reduciendo la deuda técnica y las complejidades asociadas con la evolución del sistema.

4.Pregunta

¿Por qué es importante mantener las opciones abiertas en la arquitectura de software?

Respuesta:Mantener las opciones abiertas permite flexibilidad en la toma de decisiones, habilitando a los equipos para adaptarse a los cambios a medida que nueva información esté disponible. Previene compromisos prematuros con tecnologías o implementaciones específicas que podrían resultar inadecuadas más adelante.

5.Pregunta

¿Cuáles son los dos elementos principales en los que se pueden descomponer todos los sistemas de software?

Respuesta:Los sistemas de software se pueden descomponer en dos elementos principales: políticas, que consisten en reglas y procedimientos de negocio, y detalles, que incluyen

Más Libros Gratis
Escanea para descargar
Escúchalo

implementaciones específicas como dispositivos de entrada y salida, bases de datos y protocolos.

6.Pregunta

¿Cuál es el significado de la afirmación 'una buena arquitectura de software comunica las necesidades operativas del sistema'?

Respuesta:Esto significa que la arquitectura debe proporcionar una clara visibilidad sobre los casos de uso y características del sistema, facilitando a los desarrolladores entender los requisitos e implementar soluciones de manera efectiva.

7.Pregunta

¿Cuáles son los dos tipos de valor que tiene el software, según el texto?

Respuesta:Los dos tipos de valor que tiene el software son el valor de su comportamiento y el valor de su estructura.

8.Pregunta

¿Cómo mitiga una arquitectura que apoya un buen mantenimiento de software los costos?

Respuesta:Al separar el sistema en componentes bien

Más Libros Gratis
Escanea para descargar
Escúchalo

definidos y aislar esos componentes a través de interfaces estables, una buena arquitectura ilumina las vías futuras para la adición de características y reduce el riesgo de roturas involuntarias.

9.Pregunta

¿Cuál es un ejemplo de independencia de dispositivos del capítulo?

Respuesta:Un ejemplo de independencia de dispositivos es la transición de usar métodos específicos de hardware (como llamadas directas a una impresora) a usar abstracciones del sistema operativo que permiten a los programas interactuar con dispositivos de entrada y salida de manera genérica, independientemente del hardware subyacente.

10.Pregunta

¿Qué se entiende por el término 'espeleología' en el mantenimiento de software?

Respuesta:La espeleología se refiere al costo y esfuerzo de navegar a través del software existente para identificar el lugar y método adecuados para agregar nuevas características

Más Libros Gratis
Escanea para descargar
Escúchalo

o corregir defectos, lo que a menudo conlleva un mayor riesgo de errores no intencionados.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

App Store
Selección editorial

22k reseñas de 5 estrellas

Escanear para descargar

Retroalimentación Positiva

Alondra Navarrete

itas después de cada resumen
en a prueba mi comprensión,
cen que el proceso de
rtido y atractivo."

¡Fantástico! Beltrán Fuentes

Me sorprende la variedad de libros e idiomas que soporta Bookey. No es solo una aplicación, es una puerta de acceso al conocimiento global. Además, ganar puntos para la caridad es un gran plus!

Fi

Lo
re
co
pr

a Vásquez

nábito de
le y sus
o que el
todos.

¡Me encanta! Darian Rosales

Bookey me ofrece tiempo para repasar las partes importantes de un libro. También me da una idea suficiente de si debo o no comprar la versión completa del libro. ¡Es fácil de usar!

¡Ahorra tiempo!

Bookey es mi aplicación de
crecimiento intelectual. Lo
perspicaces y bellamente c
acceso a un mundo de con

icación increíble! Elvira Jiménez

ncantan los audiolibros pero no siempre tengo tiempo
escuchar el libro entero. ¡Bookey me permite obtener
esumen de los puntos destacados del libro que me
esa! ¡Qué gran concepto! ¡Muy recomendado!

Aplicación hermosa

Esta aplicación es un salvavidas para los a
los libros con agendas ocupadas. Los resu
precisos, y los mapas mentales ayudan a
que he aprendido. ¡Muy recomendable!

Prueba gratuita con Bookey

Capítulo 16 | Independencia| Preguntas y respuestas

1.Pregunta

¿Qué cualidades esenciales debe poseer una buena arquitectura de software?

Respuesta:Una buena arquitectura de software debe apoyar los casos de uso y el funcionamiento del sistema, facilitar el mantenimiento y desarrollo, permitir el despliegue y dejar abiertas opciones para futuros cambios.

2.Pregunta

¿Cómo influye la arquitectura en la clara visibilidad de los comportamientos del sistema?

Respuesta:Una buena arquitectura aclara y expone los comportamientos del sistema al estructurarlo de una manera que hace visible la intención y los casos de uso. Los desarrolladores pueden identificar los comportamientos sin buscar, ya que estos se convierten en elementos destacados dentro de la arquitectura, representados por clases o funciones con nombres claros.

Más Libros Gratis
Escanea para descargar
Escúchalo

3.Pregunta

¿Por qué es importante la arquitectura para la operación y el rendimiento del sistema?

Respuesta:La arquitectura determina si el sistema puede manejar las cargas requeridas, como gestionar 100,000 clientes por segundo o consultar grandes volúmenes de datos de manera eficiente. Debe estar estructurada para satisfacer las demandas de rendimiento, lo que se puede lograr a través de métodos como el paralelismo o arreglos de servicios.

4.Pregunta

¿Qué implica la Ley de Conway sobre la arquitectura y la organización del equipo?

Respuesta:La Ley de Conway sugiere que la arquitectura de un sistema imitará las estructuras de comunicación de la organización que lo diseña. Por lo tanto, una arquitectura debe facilitar acciones independientes por parte de los equipos de desarrollo, requiriendo una partición adecuada de los sistemas en componentes desacoplados.

5.Pregunta

¿Cómo debería ayudar la arquitectura en el despliegue

Más Libros Gratis
Escanea para descargar
Escúchalo

del sistema?

Respuesta:Una buena arquitectura debe simplificar el despliegue, buscando que sea inmediatamente desplegable sin una configuración compleja. Esto se logra a través de un aislamiento adecuado de los componentes, permitiendo un despliegue efectivo del sistema tras la construcción.

6.Pregunta

¿Qué principios ayudan a mantener una arquitectura flexible y adaptable?

Respuesta:Principios como el Principio de Responsabilidad Única y el Principio de Cierre Común fomentan la separación de componentes que cambian por diferentes razones y la agrupación de aquellos que cambian por las mismas razones, mejorando así la adaptabilidad.

7.Pregunta

¿Qué sucede si hay duplicación en el diseño de software y cómo deberían manejarlo los arquitectos?

Respuesta:La duplicación puede llevar a desafíos si dos fragmentos de código evolucionan de manera diferente con el

Más Libros Gratis
Escanea para descargar
Escúchalo

tiempo. Los arquitectos deben ser cautelosos al fusionar estructuras de código que parecen similares a primera vista, ya que pueden divergir. Es esencial evaluar si la duplicación es verdadera o accidental antes de eliminarla.

8.Pregunta

Describe los modos de desacoplamiento en el contexto de la arquitectura.

Respuesta:El desacoplamiento puede ocurrir a diferentes niveles: a nivel de código fuente (módulos que se ejecutan en el mismo espacio de direcciones), a nivel de despliegue (unidades desplegables de forma independiente) y a nivel de servicio (servicios independientes que se comunican a través de la red). La elección del modo de desacoplamiento puede cambiar a lo largo del ciclo de vida de un proyecto.

9.Pregunta

¿Cómo ayuda una buena arquitectura a gestionar cambios futuros en un sistema?

Respuesta:Al dejar opciones abiertas para modificaciones futuras y separar preocupaciones, una buena arquitectura

Más Libros Gratis
Escanea para descargar
Escúchalo

permite una adaptación más fácil a los cambios en los requisitos, la tecnología y la estructura del equipo, lo que minimiza los costos y maximiza la satisfacción del usuario.

10.Pregunta

¿Cuál es el papel de la abstracción en el mantenimiento de la estabilidad de los componentes en la arquitectura de software?

Respuesta:La abstracción permite que los componentes sean estables y, al mismo tiempo, flexibles. Al asegurar que los componentes estables representen políticas de alto nivel a través de clases abstractas o interfaces, los arquitectos pueden proteger estos componentes de cambios en detalles de menor nivel.

Capítulo 17 | Fronteras: Dibujando Líneas| Preguntas y respuestas

1.Pregunta

¿Cuál es el objetivo principal de la arquitectura de software según se describe en el texto?

Respuesta:El objetivo principal de la arquitectura de software es minimizar los recursos humanos

Más Libros Gratis
Escanea para descargar
Escúchalo

necesarios para construir y mantener un sistema, creando límites que permiten diferir decisiones y mantienen la lógica central del negocio libre de acoplamiento innecesario y complicaciones.

2.Pregunta

¿Qué significa para el autor 'decisiones prematuras' en la arquitectura de software?

Respuesta:Las decisiones prematuras son aquellas que se toman al inicio del proceso de desarrollo y que no están relacionadas con los requisitos del negocio, como las elecciones sobre marcos y tecnologías que pueden ser pospuestas hasta que sean realmente necesarias, evitando que contaminen la arquitectura.

3.Pregunta

¿Cómo puede ayudar el trazado de límites en la arquitectura a minimizar el esfuerzo de desarrollo?

Respuesta:Trazar límites permite la separación de preocupaciones, lo que asegura que los componentes puedan evolucionar de manera independiente, evitando dependencias

Más Libros Gratis
Escanea para descargar
Escúchalo

innecesarias que complican los cambios y pueden multiplicar los esfuerzos de desarrollo.

4.Pregunta

¿Puedes proporcionar un ejemplo del texto que destaque los problemas causados por tomar decisiones arquitectónicas prematuras?

Respuesta:La historia de la empresa P ilustra esto; construyeron una compleja arquitectura de tres capas anticipando una granja de servidores, pero todas las implementaciones terminaron en un solo servidor. Esto incrementó su carga de trabajo de desarrollo debido a una complejidad innecesaria que al final no era necesaria.

5.Pregunta

¿Qué lección transmite la historia del autor sobre FitNesse respecto a las decisiones arquitectónicas?

Respuesta:La historia de FitNesse muestra la ventaja de posponer decisiones. Al evitar elecciones tempranas sobre marcos web y bases de datos, el equipo se centró primero en las características centrales, lo que les permitió adaptarse más tarde sin una extensa reingeniería. Esta flexibilidad resultó en

Más Libros Gratis
Escanea para descargar
Escúchalo

un ahorro significativo de tiempo.

6.Pregunta

Según el texto, ¿cuál debería ser el enfoque al trazar límites entre componentes?

Respuesta:Los límites deben trazarse entre cosas que importan, como separar la interfaz gráfica de usuario de las reglas de negocio o las reglas de negocio de las bases de datos, para asegurar que los cambios en un área no afecten indebidamente a otra.

7.Pregunta

¿Cuál es la principal enseñanza respecto a la relación entre las reglas de negocio y las bases de datos?

Respuesta:La principal enseñanza es que las reglas de negocio no deben depender de las bases de datos, lo que significa que solo deben interactuar a través de una abstracción que permita cambiar la base de datos sin afectar la lógica del negocio.

8.Pregunta

¿Cómo ejemplificó el autor la implementación exitosa de límites en la arquitectura de software?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:El autor ejemplificó esto a través de políticas en el desarrollo de FitNesse, donde retrasaron con éxito las decisiones sobre la base de datos mediante la implementación de una interfaz que abstraía el acceso a los datos, permitiendo flexibilidad en la elección de la base de datos más tarde.

9.Pregunta

Discute la dualidad de las decisiones arquitectónicas, tal como se describe en el capítulo.

Respuesta:Las decisiones arquitectónicas deben equilibrar la minimización de la complejidad inmediata mientras permiten cambios futuros. Estas decisiones necesitan acomodar las necesidades cambiantes de los usuarios y los avances tecnológicos sin encasillar la arquitectura en marcos rígidos que podrían obstaculizar el desarrollo futuro.

10.Pregunta

¿Cómo define el texto una arquitectura exitosa en términos de soporte operativo?

Respuesta:Una arquitectura exitosa proporciona claridad y

Más Libros Gratis
Escanea para descargar
Escúchalo

visibilidad para casos de uso importantes, facilitando su comprensión y operación mientras acomoda la escalabilidad y los cambios necesarios para soportar esas necesidades operativas.

Capítulo 18 | Anatomía de Límites| Preguntas y respuestas

1.Pregunta

¿Qué es un cruce de frontera en la arquitectura de software?

Respuesta:Un cruce de frontera ocurre cuando una función de un lado de una frontera llama a una función del otro lado y pasa datos. Es esencial gestionar las dependencias del código fuente durante este proceso para prevenir problemas durante la compilación y el despliegue.

2.Pregunta

¿Cuál es la importancia del temido monolito en la anatomía de las fronteras?

Respuesta:El temido monolito carece de fronteras físicas y representa una segregación disciplinada de funciones y datos

Más Libros Gratis
Escanea para descargar
Escúchalo

dentro de un solo ejecutable. A pesar de no tener fronteras visibles en el despliegue, sigue teniendo un significado arquitectónico significativo al facilitar el desarrollo independiente y las pruebas de componentes.

3.Pregunta

¿Cómo ayuda el polimorfismo dinámico en la gestión de dependencias a través de las fronteras?

Respuesta:El polimorfismo dinámico permite que las dependencias en tiempo de ejecución se opongan a las dependencias en tiempo de compilación, lo que permite que los clientes de alto nivel llamen a servicios de bajo nivel sin acoplarse fuertemente. Esto ayuda a mantener una clara separación de preocupaciones.

4.Pregunta

¿Cuáles son los componentes de despliegue en el contexto de la anatomía de las fronteras?

Respuesta:Los componentes de despliegue son bibliotecas dinámicamente vinculadas o paquetes (como DLLs de .NET o archivos JAR de Java) que no requieren compilación al

Más Libros Gratis
Escanea para descargar
Escúchalo

momento del despliegue. Permiten un desacoplamiento a nivel de despliegue, donde los componentes pueden ser modificados sin recompilar todo el sistema.

5.Pregunta

¿Cómo se puede utilizar el threading dentro de componentes monolíticos o de despliegue?

Respuesta:Los hilos se pueden usar para organizar la ejecución dentro de componentes tanto monolíticos como de despliegue, ayudando a gestionar el orden y la programación sin crear fronteras arquitectónicas. Pueden mejorar el rendimiento a través de la concurrencia entre componentes.

6.Pregunta

¿Qué define una frontera de proceso local en la arquitectura de software?

Respuesta:Una frontera de proceso local es una frontera arquitectónica más fuerte donde los procesos se ejecutan en espacios de direcciones separados en la misma máquina física o multicore, comunicándose principalmente a través de métodos de comunicación entre procesos como sockets o

Más Libros Gratis
Escanea para descargar
Escúchalo

colas de mensajes.

7.Pregunta

¿Qué caracteriza una frontera de servicio y por qué es significativa?

Respuesta:Una frontera de servicio caracteriza un proceso de comunicación que puede operar de manera independiente, a menudo a través de una red, sin suposiciones sobre la proximidad física. Esta frontera es significativa debido a su potencial para escalabilidad y flexibilidad, dado que los servicios pueden ser desplegados y modificados de manera independiente.

8.Pregunta

¿Qué desafíos surgen de los cruces de frontera mal gestionados en los sistemas de software?

Respuesta:Los cruces de frontera mal gestionados pueden llevar a una estructura de dependencias enredada que complica el desarrollo, aumenta el riesgo de errores, eleva los costos de despliegue y crea dificultades durante las pruebas de integración.

Más Libros Gratis
Escanea para descargar
Escúchalo

9.Pregunta

¿Cómo puede la arquitectura encapsular políticas de alto nivel mientras pospone decisiones sobre detalles operativos?

Respuesta:Una buena arquitectura separa las políticas de alto nivel de los detalles operativos mediante la abstracción, permitiendo a los desarrolladores concentrarse en la lógica de negocio mientras posponen decisiones específicas de implementación hasta que sean necesarias, manteniendo así flexibilidad.

10.Pregunta

¿Qué papel debería jugar la arquitectura de software en la minimización de recursos humanos para el mantenimiento del sistema?

Respuesta:La arquitectura de software debería facilitar una estructura clara que minimice el esfuerzo necesario para el mantenimiento del sistema al asegurar que los componentes estén débilmente acoplados y puedan ser modificados de manera independiente, reduciendo así el riesgo y la complejidad asociados con los cambios.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

Escanear para descargar

Leer, Compartir, Empoderar

Completa tu desafío de lectura, dona libros a los niños africanos.

El Concepto

BOOKS FOR AFRICA × ×

Esta actividad de donación de libros se está llevando a cabo junto con Books For Africa. Lanzamos este proyecto porque compartimos la misma creencia que BFA: Para muchos niños en África, el regalo de libros realmente es un regalo de esperanza.

La Regla

Gana 100 puntos → **Canjea un libro** → **Dona a África**

Tu aprendizaje no solo te brinda conocimiento sino que también te permite ganar puntos para causas benéficas. Por cada 100 puntos que ganes, se donará un libro a África.

Prueba gratuita con Bookey

Capítulo 19 | Política y Nivel| Preguntas y respuestas

1.Pregunta

¿Cuál es el concepto principal de la arquitectura de software según se describe en este capítulo?

Respuesta:Los sistemas de software son, en última instancia, declaraciones de políticas, que detallan cómo se transforman las entradas en salidas. La arquitectura implica separar y organizar cuidadosamente estas políticas según cómo y cuándo cambian.

2.Pregunta

¿Cómo deben agruparse las políticas en la arquitectura de software?

Respuesta:Las políticas que cambian por las mismas razones y al mismo tiempo deben agruparse dentro del mismo componente, mientras que aquellas que cambian por diferentes razones o en momentos distintos deben separarse en componentes diferentes.

3.Pregunta

¿Qué significa cuando decimos que una buena

Más Libros Gratis
Escanea para descargar
Escúchalo

arquitectura tiene componentes de bajo nivel que dependen de componentes de alto nivel?

Respuesta:Significa que los detalles de menor nivel están abstraídos, asegurando que los cambios en los componentes de bajo nivel no afecten directamente a las políticas de alto nivel, reduciendo así el riesgo y la complejidad en la arquitectura.

4.Pregunta

¿Cómo se relaciona el nivel de una política con su distancia de las entradas y salidas?

Respuesta:El nivel se define como la distancia de las entradas y salidas; las políticas de alto nivel están más lejos de estos puntos finales y, por lo tanto, cambian con menos frecuencia ya que implican decisiones más significativas y estratégicas.

5.Pregunta

¿Por qué es importante mantener una clara separación entre las políticas de alto nivel y los detalles de bajo nivel en la arquitectura?

Respuesta:Esta separación minimiza el impacto de los cambios: los cambios triviales en los niveles inferiores no

Más Libros Gratis
Escanea para descargar
Escúchalo

afectarán a las políticas más importantes de alto nivel, asegurando mayor estabilidad y flexibilidad en el sistema.

6.Pregunta

¿A qué se refiere el término 'grafo acíclico dirigido' en el contexto de la arquitectura de software?

Respuesta:Se refiere a un grafo donde los nodos representan componentes que contienen políticas en el mismo nivel, y los bordes dirigidos indican dependencias entre estos componentes, asegurando que no haya ciclos.

7.Pregunta

¿Cuáles son los beneficios de tener componentes de bajo nivel como plugins para componentes de alto nivel?

Respuesta:Permite cambios independientes y la evolución de detalles de bajo nivel sin afectar las políticas de alto nivel, manteniendo así la integridad y funcionalidad general del sistema.

8.Pregunta

¿De qué manera pueden los cambios en las políticas de bajo nivel afectar a las políticas de alto nivel?

Respuesta:Los cambios en las políticas de bajo nivel

Más Libros Gratis
Escanea para descargar
Escúchalo

deberían ser principalmente independientes a menos que interactúen directamente con las políticas de alto nivel; idealmente, los cambios de bajo nivel deberían tener un impacto mínimo o nulo en la funcionalidad de alto nivel.

9.Pregunta

¿Por qué tienden las políticas a cambiar con menos frecuencia que los detalles en los sistemas de software?

Respuesta:Las políticas a menudo se basan en reglas comerciales generales o decisiones estratégicas, que tienden a ser más estables, mientras que detalles como los elementos de la interfaz de usuario o los formatos de datos cambian regularmente debido a las necesidades del usuario o los avances tecnológicos.

Capítulo 20 | Reglas de Negocio| Preguntas y respuestas

1.Pregunta

¿Qué califica como una Regla Crítica de Negocio?

Respuesta:Una Regla Crítica de Negocio es un procedimiento o directriz fundamental que contribuye directamente a generar o ahorrar dinero

Más Libros Gratis
Escanea para descargar
Escúchalo

para un negocio, independientemente de si se implementa a través de tecnologías automatizadas.
Por ejemplo, un banco que cobra intereses por un préstamo es una Regla Crítica de Negocio.

2.Pregunta

¿Qué es una Entidad en el contexto de un sistema de software?

Respuesta:Una Entidad es un objeto dentro de un sistema de software que encapsula un pequeño conjunto de Reglas Críticas de Negocio y opera sobre Datos Críticos de Negocio, uniendo efectivamente las reglas de negocio y los datos necesarios.

3.Pregunta

¿Cómo difieren los Casos de Uso de las Entidades?

Respuesta:Los Casos de Uso definen las maneras específicas en que se utiliza un sistema automatizado, incluyendo requisitos de entrada, expectativas de salida y pasos de procesamiento, mientras que las Entidades representan las reglas de negocio y datos más amplios que rigen las

Más Libros Gratis
Escanea para descargar
Escúchalo

operaciones del sistema.

4.Pregunta

¿Por qué es importante mantener los modelos de solicitud y respuesta independientes?

Respuesta:Mantener los modelos de solicitud y respuesta independientes evita dependencias innecesarias de sistemas o marcos externos que pueden cambiar con el tiempo, preservando así la integridad de los casos de uso y reduciendo la complejidad de mantenimiento.

5.Pregunta

¿Cómo se aplica el Principio de Inversión de Dependencias a la relación entre Entidades y Casos de Uso?

Respuesta:Según el Principio de Inversión de Dependencias, los conceptos de alto nivel como las Entidades no deberían depender de conceptos de bajo nivel como los Casos de Uso. En cambio, los Casos de Uso deben depender de las Entidades, asegurando que la lógica empresarial central permanezca estable mientras que las reglas específicas de la aplicación son flexibles.

Más Libros Gratis
Escanea para descargar
Escúchalo

6.Pregunta

¿Qué principios ayudan a minimizar los recursos humanos en la arquitectura de software?

Respuesta:Los principios incluyen el Principio de Responsabilidad Única (SRP), que aconseja mantener separadas las diferentes responsabilidades; el Principio Abierto-Cerrado (OCP), que fomenta diseños que se pueden extender sin modificar el código existente; y el Principio de Inversión de Dependencias (DIP), que promueve la referencia a abstracciones en lugar de implementaciones concretas.

7.Pregunta

¿Cuál es el papel de un arquitecto en el diseño de software según el texto?

Respuesta:El papel del arquitecto implica asegurar que la arquitectura del software minimice los costos de ciclo de vida del sistema al facilitar el desarrollo, la implementación y el mantenimiento, mientras se mantienen abiertas tantas opciones como sea posible para futuros cambios.

Más Libros Gratis
Escanea para descargar
Escúchalo

8.Pregunta

¿Por qué es problemático si los componentes de diferentes estabilidad (métricas I) dependen entre sí?

Respuesta:Si los componentes inestables de bajo nivel dependen de componentes estables de alto nivel, cualquier cambio en el componente inestable puede potencialmente interrumpir la funcionalidad del componente estable, introduciendo riesgos y complicaciones que podrían socavar la estabilidad del sistema.

9.Pregunta

¿Cuál es la importancia de separar políticas de detalles en la arquitectura de software?

Respuesta:Separar políticas de detalles permite arquitecturas flexibles que pueden adaptarse a requisitos cambiantes, ya que los cambios en las implementaciones de detalles no afectarán la lógica empresarial central, ayudando a mantener la integridad y longevidad del sistema.

10.Pregunta

¿Cómo puede el uso de plugins mejorar la arquitectura de software?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:El uso de plugins puede hacer que las reglas de negocio fundamentales de un sistema sean independientes de componentes o funcionalidades opcionales como GUI o bases de datos, permitiendo una fácil intercambiabilidad y reduciendo el impacto de cambios en componentes opcionales sobre la lógica empresarial central.

Capítulo 21 | Arquitectura Gritando| Preguntas y respuestas

1.Pregunta

¿Qué comunica la arquitectura de una aplicación de software?

Respuesta:Debería comunicar los casos de uso de la aplicación, indicando su propósito y funcionalidad principales, en lugar de los frameworks o tecnologías utilizados para construirla.

2.Pregunta

¿Por qué debería una buena arquitectura priorizar los casos de uso sobre los frameworks?

Respuesta:Una buena arquitectura permite flexibilidad y cambios más fáciles en el futuro al desacoplar la

Más Libros Gratis
Escanea para descargar
Escúchalo

funcionalidad principal de la aplicación (casos de uso) de los frameworks y entornos específicos que pueden cambiar con el tiempo.

3.Pregunta

¿Cómo deberían los arquitectos ver los frameworks?

Respuesta:Los frameworks deberían ser considerados como herramientas a utilizar, no como arquitecturas en sí mismas; es esencial asegurarse de que la arquitectura no se vincule a un framework específico.

4.Pregunta

¿Cómo puede ayudar una buena arquitectura a las pruebas unitarias?

Respuesta:Una buena arquitectura permite las pruebas unitarias al asegurar que las reglas de negocio (entidades) y los casos de uso estén desacoplados de los frameworks, permitiendo que las pruebas se realicen sin necesidad del servidor web o la base de datos.

5.Pregunta

¿Cuál es el propósito de separar las reglas de negocio y las bases de datos en la arquitectura?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:La separación permite diferir las decisiones sobre la base de datos y evita dependencias innecesarias, haciendo que los cambios en una no afecten a la otra.

6.Pregunta

¿Qué tipo de decisiones debería una arquitectura tratar de diferir?

Respuesta:La arquitectura debería tratar de diferir decisiones sobre frameworks, bases de datos y otros detalles técnicos que no afectan directamente la funcionalidad principal de la aplicación.

7.Pregunta

¿Qué se entiende por un sistema 'independiente del dispositivo'?

Respuesta:La independencia del dispositivo significa que la funcionalidad principal de la aplicación debería ser ajena a cómo se entrega (web, móvil, etc.), permitiendo flexibilidad en cómo se implementa o escala.

8.Pregunta

¿Qué sucede cuando la arquitectura de un sistema no apoya eficazmente sus casos de uso?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Puede aumentar los costos de mantenimiento, crear dificultades de despliegue y hacer que el desarrollo sea más complejo, llevando a un sistema que no es amigable para el usuario o adaptable a cambios futuros.

9.Pregunta

¿Qué debería revelar la estructura de una arquitectura de software?

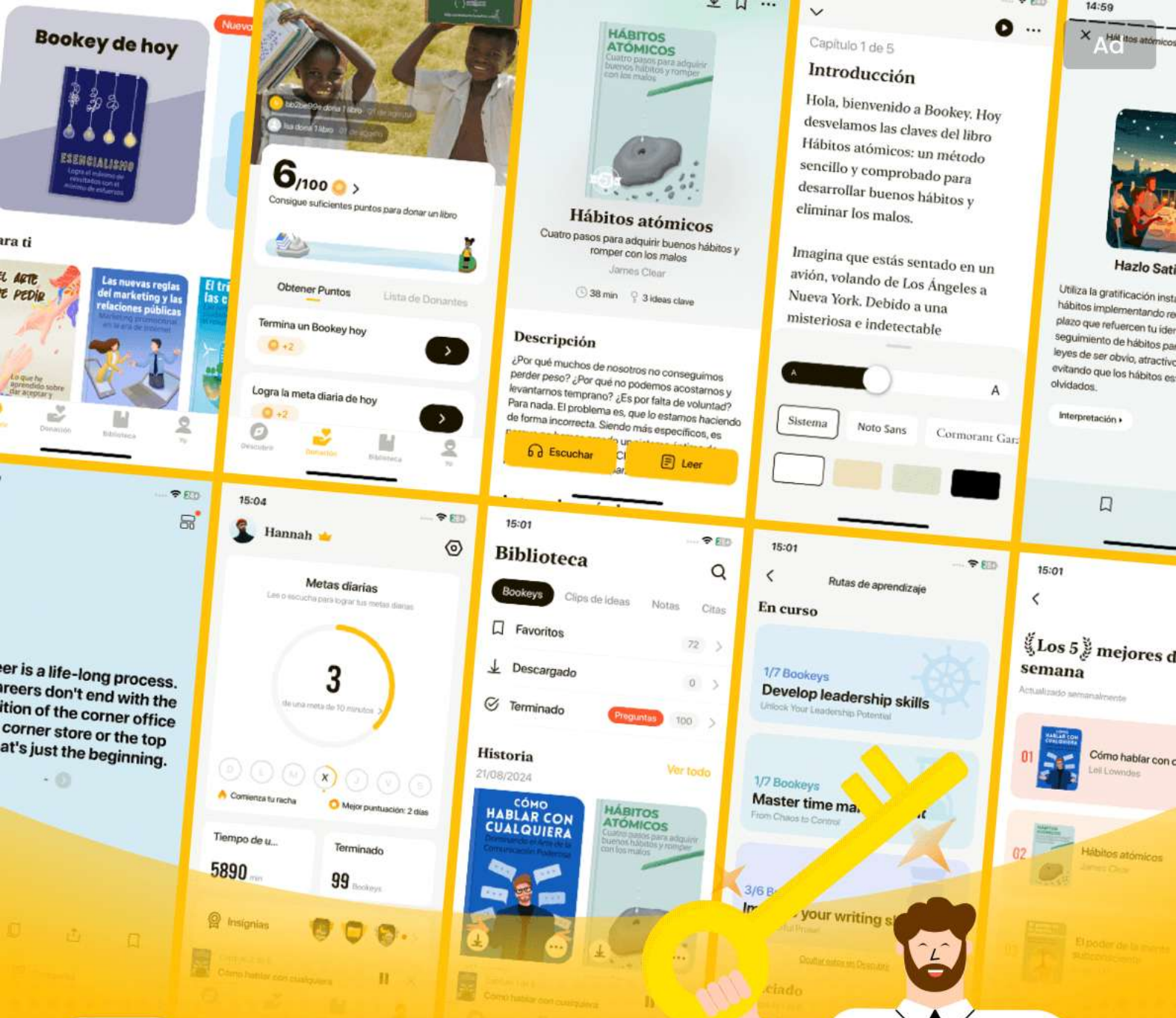
Respuesta:La estructura debería revelar las necesidades operativas y los casos de uso del sistema, haciéndolo claro y comprensible para los desarrolladores.

10.Pregunta

¿Cómo pueden las decisiones sobre casos de uso individuales influir en la arquitectura general?

Respuesta:Al reconocer qué casos de uso cambian por diferentes razones, los arquitectos pueden diseñar el sistema para mantener estos elementos separados y minimizar el impacto de los cambios.

Más Libros Gratis
Escanea para descargar
Escúchalo



Las mejores ideas del mundo desbloquean tu potencial

Prueba gratuita con Bookey

Escanear para descargar

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 22 | La Arquitectura Limpia| Preguntas y respuestas

1.Pregunta

¿Cuál es el objetivo principal de Arquitectura Limpia según Robert C. Martin?

Respuesta:El objetivo principal de Arquitectura Limpia es la separación de preocupaciones mediante la división del software en capas, lo que permite un mantenimiento más fácil, mejor capacidad de prueba y la independencia de varios componentes del sistema frente a influencias externas como frameworks y interfaces de usuario.

2.Pregunta

¿Cómo garantiza la Arquitectura Limpia que las reglas de negocio se puedan probar de forma independiente?

Respuesta:La Arquitectura Limpia permite que las reglas de negocio se prueben de forma independiente al asegurarse de que no dependen de elementos externos como la interfaz de usuario, bases de datos u otros frameworks, gracias a una estricta separación de capas.

Más Libros Gratis
Escanea para descargar
Escúchalo

3.Pregunta

¿Qué establece la Regla de Dependencia en Arquitectura Limpia?

Respuesta:La Regla de Dependencia establece que las dependencias del código fuente deben apuntar únicamente hacia adentro, hacia políticas de mayor nivel, lo que significa que las capas internas no deben ser conscientes de los detalles de las capas externas.

4.Pregunta

¿Puedes explicar el concepto de entidades en Arquitectura Limpia?

Respuesta:Las entidades representan reglas de negocio críticas a nivel empresarial y encapsulan las reglas más generales de la aplicación, haciéndolas reutilizables en diferentes aplicaciones sin verse afectadas por cambios externos como modificaciones de la interfaz de usuario o de la base de datos.

5.Pregunta

¿Qué papel juegan los casos de uso dentro de la Arquitectura Limpia?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Los casos de uso encapsulan e implementan reglas de negocio específicas de la aplicación, orquestando el flujo de datos hacia y desde las entidades y asegurando que los cambios en el comportamiento de la aplicación no afecten las reglas de negocio o entidades subyacentes.

6.Pregunta

¿Cuál es la importancia de los adaptadores de interfaz en Arquitectura Limpia?

Respuesta:Los adaptadores de interfaz adaptan los datos entre los casos de uso/entidades y las interfaces externas como bases de datos o servicios web, aislando las capas internas de las especificidades de las interacciones externas correspondientes, cumpliendo así con la Regla de Dependencia.

7.Pregunta

¿Cómo facilita la Arquitectura Limpia arquitecturas que sean probables?

Respuesta:Al separar las reglas de negocio y asegurar que sean independientes de frameworks, bases de datos y

Más Libros Gratis
Escanea para descargar
Escúchalo

elementos de UI, la Arquitectura Limpia garantiza que la lógica de negocio se pueda probar de forma aislada sin requerir la configuración del sistema completo.

8.Pregunta

¿Cuál es la importancia de mantener la independencia de los frameworks según Robert C. Martin?

Respuesta:Mantener la independencia de los frameworks permite a los desarrolladores evitar estar limitados por las complejidades o restricciones de frameworks específicos, lo que permite ainsi flexibilidad y adaptabilidad a medida que evolucionan los requisitos del proyecto.

9.Pregunta

¿Cómo se relaciona el concepto de 'mantener opciones abiertas' con la arquitectura de software?

Respuesta:El concepto de mantener opciones abiertas en la arquitectura de software implica diseñar sistemas de tal manera que las decisiones sobre herramientas, tecnologías e implementaciones específicas puedan diferirse y ajustarse sin comprometer la funcionalidad central.

Más Libros Gratis
Escanea para descargar
Escúchalo

10.Pregunta

¿Qué puede llevar al denominado 'síndrome de la mañana siguiente' en el desarrollo de software?

Respuesta:El síndrome de la mañana siguiente ocurre cuando múltiples desarrolladores trabajan en componentes interconectados sin una configuración clara de límites, lo que resulta en roturas inesperadas cuando los cambios de un desarrollador afectan elementos compartidos utilizados por otros.

Capítulo 23 | Presentadores y Objetos Humildes| Preguntas y respuestas

1.Pregunta

¿Qué es el patrón Humble Object y por qué es importante en Arquitectura Limpia?

Respuesta:El patrón Humble Object es un patrón de diseño que ayuda a separar las partes de un sistema que son difíciles de probar de aquellas que son fáciles de probar. Lo hace dividiendo las funcionalidades en dos clases: una clase humilde sin cambios que no contiene lógica compleja y es difícil

Más Libros Gratis
Escanea para descargar
Escúchalo

de probar, y otro presentador amigable para las pruebas que encapsula la lógica de negocio, haciendo que la arquitectura sea más comprobable y mantenible. Su importancia radica en definir claros límites arquitectónicos para mejorar la organización general del sistema y facilitar las pruebas.

2.Pregunta

¿Cómo trabajan juntos los Presentadores y las Vistas en Arquitectura Limpia?

Respuesta:En Arquitectura Limpia, las Vistas actúan como los objetos humildes responsables de mostrar información, mientras que los Presentadores manejan el procesamiento de datos. El Presentador formatea los datos recibidos de la aplicación en una estructura (el Modelo de Vista) que la Vista puede mostrar fácilmente. Al separar las responsabilidades, el Presentador puede ser probado de manera independiente, mientras que la Vista permanece simple y se enfoca únicamente en los aspectos de presentación de la interfaz de usuario.

Más Libros Gratis
Escanea para descargar
Escúchalo

3.Pregunta

¿Qué papel juegan las puertas de acceso a la base de datos en Arquitectura Limpia?

Respuesta:Las puertas de acceso a la base de datos son interfaces entre los casos de uso y la base de datos, definiendo métodos para todas las operaciones CRUD sin incrustar directamente SQL en la capa de casos de uso. Siguen el patrón Humble Object actuando como una capa humilde que simplemente ejecuta comandos de base de datos, mientras que los interactores (lógica de casos de uso) permanecen libres de detalles de la base de datos y están diseñados para ser testeables.

4.Pregunta

¿Cuál es la importancia de los modelos de datos como entidades y casos de uso en una aplicación empresarial?

Respuesta:Las entidades encapsulan reglas y datos críticos del negocio, representando conceptos centrales que el sistema gestiona y asegurando que permanezcan estables a pesar de los cambios en factores externos. Los casos de uso describen

Más Libros Gratis
Escanea para descargar
Escúchalo

comportamientos e interacciones específicas de la aplicación, orquestando cómo se utilizan las entidades sin exponer sus complejidades. Juntos, permiten un código limpio y mantenible que puede evolucionar con las necesidades del negocio.

5.Pregunta

¿Por qué se considera la capacidad de prueba un atributo de una buena arquitectura?

Respuesta:La capacidad de prueba indica cuán fácilmente se puede validar un sistema contra sus requisitos, lo cual es esencial para mantener la calidad en el desarrollo de software. Un sistema bien arquitectado, al emplear patrones como el patrón Humble Object, define interfaces claras y desacopla componentes, permitiendo que se realicen pruebas automatizadas sobre la lógica de negocio de alto nivel sin la necesidad de complejidades subyacentes, reduciendo así las tasas de defectos y facilitando ciclos de desarrollo más rápidos.

6.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Cómo se puede diseñar una arquitectura de software para mantener las opciones abiertas para cambios futuros?

Respuesta:Para mantener opciones abiertas, la arquitectura debe construirse en torno a abstracciones que permitan intercambiar componentes (como bases de datos, marcos de UI o protocolos) sin afectar la lógica de negocio central. Esto se puede lograr a través de diseños basados en interfaces, promoviendo el uso de inyección de dependencias y separando las reglas de negocio de detalles como interfaces de usuario o bases de datos, lo que permite una adaptación flexible a medida que evolucionan los requerimientos.

7.Pregunta

¿Qué errores suelen cometer las empresas al implementar arquitecturas de software?

Respuesta:Los errores comunes incluyen tomar decisiones prematuras sobre herramientas, marcos y tecnologías antes de comprender completamente los requisitos o casos de uso, lo que puede resultar en arquitecturas rígidas que son difíciles

Más Libros Gratis
Escanea para descargar
Escúchalo

de cambiar. También pueden fallar en desacoplar las reglas de negocio de las implementaciones técnicas, lo que lleva a sistemas que son excesivamente complejos y costosos de mantener.

8.Pregunta

¿De qué manera la Arquitectura Limpia enfatiza la importancia de separar preocupaciones?

Respuesta:La Arquitectura Limpia enfatiza la importancia de separar preocupaciones abogando por una estructura donde la lógica de negocio es distinta de la interfaz de usuario y las dependencias externas. Esta clara separación garantiza que los cambios en un aspecto del sistema no impacten adversamente a otros, promoviendo la mantenibilidad y permitiendo que cada parte evolucione de manera independiente.

9.Pregunta

¿Cuáles son algunos patrones arquitectónicos asociados con la Arquitectura Limpia?

Respuesta:Los patrones arquitectónicos asociados con la

Más Libros Gratis
Escanea para descargar
Escúchalo

Arquitectura Limpia incluyen Arquitectura Hexagonal (Puertos y Adaptadores), DCI (Datos, Contexto e Interacción) y BCE (Comportamiento, Contexto y Entidad). Cada uno de estos patrones tiene como objetivo lograr la misma meta: la separación de preocupaciones, lo que conduce a sistemas más mantenibles y flexibles.

10.Pregunta

¿Cómo se relaciona la Regla de Dependencia con la Arquitectura Limpia?

Respuesta:La Regla de Dependencia es un principio fundamental de la Arquitectura Limpia que establece que todas las dependencias del código fuente deben apuntar hacia adentro, hacia políticas de niveles superiores, en lugar de permitir que las capas internas dependan de las capas externas. De esta manera, la arquitectura minimiza el impacto de los cambios y facilita una mayor adaptabilidad, mantenibilidad y facilidad de prueba.

Capítulo 24 | Fronteras Parciales| Preguntas y respuestas

1.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Qué es un límite parcial en la arquitectura de software?
Respuesta:Un límite parcial permite a los arquitectos de software crear límites entre componentes sin implementar completamente todas las interfaces recíprocas necesarias y la gestión de dependencias requeridas para una completa separación. Este enfoque puede diferir el trabajo intensivo en recursos mientras se mantiene algún nivel de separación.

2.Pregunta

¿Por qué se considera que los límites arquitectónicos son costosos de mantener?
Respuesta:Los límites arquitectónicos requieren una gestión integral, incluyendo interfaces polimórficas recíprocas y un manejo cuidadoso de las estructuras de datos de entrada/salida, lo que hace que sean costosos en términos de configuración inicial y mantenimiento continuo.

3.Pregunta

¿Cómo entra en conflicto el diseño anticipatorio con la

Más Libros Gratis
Escanea para descargar
Escúchalo

filosofía Agile?

Respuesta:El diseño anticipatorio, que implica crear límites y estructuras para características que pueden no ser necesarias de inmediato, puede violar el principio YAGNI ('No lo vas a necesitar') que a menudo se enfatiza en el desarrollo ágil.

4.Pregunta

¿Cuál es el beneficio de omitir el último paso al implementar límites parciales?

Respuesta:Omitir el último paso permite crear componentes con todo el código necesario y la arquitectura de diseño sin la carga de gestionar múltiples componentes desplegables, reduciendo la complejidad en el seguimiento de versiones y la gestión de lanzamientos.

5.Pregunta

¿Qué riesgo está asociado con la estrategia de límites parciales como se demuestra en FitNesse?

Respuesta:El riesgo es que, a medida que las dependencias entre los componentes avanzan con el tiempo, la separación inicialmente prevista puede debilitarse, llevando a una

Más Libros Gratis
Escanea para descargar
Escúchalo

arquitectura más enredada que se vuelve difícil de deshacer.

6.Pregunta

¿Qué es un límite unidimensional y cómo se diferencia de un límite completo?

Respuesta:Un límite unidimensional utiliza interfaces recíprocas para mantener la separación en una estructura más simple, ejemplificada por el patrón Estrategia. Es menos costoso de mantener que un límite arquitectónico completo, que requiere una gestión más compleja de las dependencias.

7.Pregunta

¿Cómo puede un patrón Facade simplificar la gestión de límites?

Respuesta:El patrón Facade simplifica la gestión de límites al proporcionar una interfaz simplificada que permite a los clientes acceder a servicios sin exponer todas las complejidades y dependencias, aunque corre el riesgo de crear dependencias transitivas.

8.Pregunta

¿Cuál es la importancia de aislar a los clientes de las implementaciones de servicios?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Aislar a los clientes de las implementaciones de servicios mejora la mantenibilidad, ya que reduce la probabilidad de que los cambios en la implementación del servicio afecten a los clientes y permite un intercambio más fácil de implementaciones.

9.Pregunta

¿Cómo deberían los arquitectos decidir sobre la implementación de límites arquitectónicos?

Respuesta:Los arquitectos deberían evaluar dónde podría existir eventualmente un límite, basándose en su comprensión de las necesidades actuales y futuras del sistema, equilibrando los costos de límites parciales frente a límites completos en relación con los requisitos organizacionales y del proyecto.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

Escanear para descargar

Prueba la aplicación Bookey para leer más de 1000 resúmenes de los mejores libros del mundo

Desbloquea de 1000+ títulos, 80+ temas

Nuevos títulos añadidos cada semana

Brand | Liderazgo & Colaboración | Gestión del tiempo | Relaciones & Comunicación | Kno

ategia Empresarial | Creatividad | Memorias | Dinero e Inversiones | Conózcase a sí mismo

nprendimiento | Historia del mundo | Comunicación entre Padres e Hijos | Autocuidado | M

Perspectivas de los mejores libros del mundo

Prueba gratuita con Bookey

Capítulo 25 | Capas y Límites| Preguntas y respuestas

1.Pregunta

¿Qué principio arquitectónico clave permite la separación de la interfaz de usuario (UI) de la lógica de negocio, facilitando la adaptación a diferentes lenguajes o interfaces?

Respuesta:La Regla de Dependencia permite la separación entre la UI y la lógica de negocio, facilitando la adaptación y reutilización de las mismas reglas de negocio sin preocuparse por el lenguaje o formato de la UI.

2.Pregunta

En el contexto de un juego como 'Hunt the Wumpus', ¿cuáles son los posibles límites arquitectónicos a considerar para mejorar la flexibilidad?

Respuesta:Los límites arquitectónicos potenciales incluyen desacoplar las reglas del juego de la UI y establecer APIs independientes del lenguaje que permitan adaptar el juego para localización o diferentes mecanismos de comunicación.

Más Libros Gratis
Escanea para descargar
Escúchalo

3.Pregunta

¿Cómo impacta la estructura organizativa de los componentes en la capacidad de un sistema para adaptarse a cambios sin requerir un trabajo extensivo?

Respuesta:Al organizar los componentes por su funcionalidad e isolando las dependencias, los componentes de alto nivel pueden permanecer estables incluso cuando las implementaciones de bajo nivel cambian, reduciendo significativamente el esfuerzo requerido para adaptarse a los nuevos requisitos.

4.Pregunta

¿Por qué es importante mantener la independencia de las reglas de negocio con respecto a frameworks y bases de datos externas?

Respuesta:Mantener la independencia permite que las reglas de negocio permanezcan estables y no se vean afectadas por cambios en sistemas externos, facilitando así el mantenimiento, la implementación y la adaptación de la arquitectura del software.

5.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Qué analogía se utiliza para explicar por qué la arquitectura del software debe ser flexible y abierta a cambios?

Respuesta:Se utiliza la analogía de la arquitectura de un edificio: así como los planos arquitectónicos de una casa permiten diferentes elecciones de materiales, la arquitectura del software debe permitir flexibilidad en frameworks y tecnologías sin comprometerse a decisiones específicas demasiado pronto.

6.Pregunta

¿Qué papel juegan las interfaces para garantizar que las políticas de alto nivel no se vean afectadas por cambios en los componentes de bajo nivel?

Respuesta:Las interfaces actúan como una barrera, permitiendo que las políticas de alto nivel interactúen con componentes de bajo nivel sin conocer los detalles de su implementación, lo que facilita modificaciones y mejoras.

7.Pregunta

¿Cuál es la importancia de crear una clara distinción entre las reglas de negocio centrales y los casos de uso en

Más Libros Gratis
Escanea para descargar
Escúchalo

la arquitectura del software?

Respuesta:Las reglas de negocio centrales representan las políticas fundamentales que dictan las funciones del negocio, mientras que los casos de uso dictan cómo los usuarios interactúan con esas reglas; mantenerlas distintas permite una clara responsabilidad y adaptaciones más sencillas.

8.Pregunta

¿Cómo se extiende el principio de 'Responsabilidad Única' más allá de la estructura del código hacia el diseño arquitectónico?

Respuesta:En el diseño arquitectónico, el Principio de Responsabilidad Única fomenta que cada componente maneje un aspecto de la funcionalidad, asegurando que los cambios en una área no afecten inadvertidamente a otras, mejorando así la mantenibilidad.

9.Pregunta

¿Qué verdad fundamental sobre la naturaleza del software enfatiza Robert C. Martin respecto a la relación entre la simplicidad y la arquitectura?

Respuesta:La relación enfatiza que los sistemas complejos a

Más Libros Gratis
Escanea para descargar
Escúchalo

menudo surgen de intentos de gestionar la sobreingeniería, mientras que una arquitectura efectiva se centra en la simplicidad, manteniendo las opciones abiertas sin abrumar el software con complejidad innecesaria.

10.Pregunta

¿De qué manera contribuyen las interfaces polimórficas a la robustez de la arquitectura del software?

Respuesta:Las interfaces polimórficas crean abstracciones que permiten utilizar diferentes implementaciones de manera intercambiable, minimizando las dependencias y promoviendo la flexibilidad dentro de la arquitectura.

11.Pregunta

¿Por qué podría un arquitecto de software elegir implementar un límite parcial en lugar de un límite arquitectónico completo inicialmente?

Respuesta:Un límite parcial permite anticipar necesidades futuras sin la sobrecarga inicial de una implementación completa, ayudando a gestionar los costos de desarrollo mientras deja espacio para mejoras futuras.

12.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Cuál es la relación entre las métricas de gestión de dependencias y la calidad de la arquitectura del software?
Respuesta:Las métricas de gestión de dependencias ayudan a evaluar la estabilidad, cohesión y niveles de abstracción de los componentes, guiando a los arquitectos en el mantenimiento de una arquitectura estructurada y equilibrada que minimiza la complejidad y mejora la flexibilidad.

13.Pregunta

¿Cómo se manifiesta la tensión entre diferentes principios arquitectónicos (como REP, CCP y CRP) durante el diseño y la implementación?
Respuesta:Esta tensión requiere que los arquitectos equilibren el deseo de reutilización con la necesidad de complejidad manejable y los riesgos de los cambios, asegurando que las decisiones arquitectónicas apoyen el mantenimiento y la adaptabilidad a largo plazo del sistema.

14.Pregunta

¿Qué efecto tiene seguir el Principio de Inversión de Dependencias sobre la organización del código y los componentes?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Seguir el Principio de Inversión de Dependencias ayuda a garantizar que los módulos de alto nivel permanezcan independientes de los cambios en los módulos de bajo nivel, promoviendo una arquitectura más estable y adaptable.

Capítulo 26 | El Componente Principal| Preguntas y respuestas

1.Pregunta

¿Cuál es el papel del componente Main en un sistema de software?

Respuesta:El componente Main actúa como el detalle final y la política de más bajo nivel del sistema. Sirve como el punto de entrada, creando y coordinando otros componentes, manejando inicializaciones y distribuyendo dependencias sin depender de un marco de Inyección de Dependencias.

2.Pregunta

¿Por qué se considera que el componente Main es sucio o el más sucio de todos los componentes?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Se considera que el componente Main es sucio porque trata directamente con detalles de bajo nivel, como la carga de recursos y la inicialización de configuraciones, que normalmente se evitan en arquitecturas limpias para mantener los componentes de alto nivel desacoplados de los detalles de implementación.

3.Pregunta

¿Cómo interactúa el componente Main con la arquitectura general?

Respuesta:El componente Main entrega el control a políticas de alto nivel y abstrae implementaciones específicas, asegurando que los cambios en los componentes de alto nivel no necesiten la recompilación de Main. Facilita una separación clara entre la inicialización del sistema y su lógica de negocio.

4.Pregunta

¿Qué principio se destaca al discutir cómo el componente Main maneja las dependencias?

Respuesta:Se destaca el Principio de Inversión de

Más Libros Gratis
Escanea para descargar
Escúchalo

Dependencias, enfatizando que las dependencias deben ser gestionadas de tal manera que promuevan una alta abstracción de nivel superior mientras mantienen los detalles de bajo nivel aislados de la lógica de negocio central.

5.Pregunta

¿De qué maneras pueden múltiples componentes Main mejorar un sistema de software?

Respuesta:Tener múltiples componentes Main permite diferentes configuraciones adaptadas a varios entornos, como Desarrollo, Pruebas y Producción. Esta flexibilidad de configuración mejora la mantenibilidad y adaptabilidad del sistema.

6.Pregunta

¿Qué podemos deducir sobre la importancia de separar preocupaciones en el componente Main?

Respuesta:Separar preocupaciones dentro del componente Main asegura que las políticas de alto nivel puedan evolucionar de manera independiente de los detalles de bajo nivel, lo que lleva a una arquitectura más robusta y

Más Libros Gratis
Escanea para descargar
Escúchalo

mantenible que minimiza el costo del cambio.

7.Pregunta

¿Por qué es importante que la arquitectura permita cambios en los requisitos sin un trabajo extensivo de reestructuración?

Respuesta:Permitir cambios sin un trabajo extensivo de reestructuración es crucial porque reduce el costo de modificación y mejora la productividad del desarrollador, facilitando la adaptación del sistema a nuevas condiciones y requisitos a lo largo del tiempo.

8.Pregunta

¿Cómo ejemplifica el componente Main el concepto de plugins en la arquitectura de software?

Respuesta:El componente Main ejemplifica los plugins al ser un punto de entrada personalizable que puede ser reemplazado o extendido para varias configuraciones, permitiendo diferentes comportamientos en tiempo de ejecución sin alterar la arquitectura subyacente del sistema.

Capítulo 27 | Servicios: Grandes y Pequeños| Preguntas y respuestas

Más Libros Gratis
Escanea para descargar
Escúchalo

1.Pregunta

¿Cuál es el argumento principal que presenta Robert C. Martin respecto a las arquitecturas orientadas a servicios?

Respuesta:Martin argumenta que solo porque un sistema esté estructurado alrededor de servicios, no significa automáticamente que constituya un marco arquitectónico adecuado. Sostiene que la verdadera arquitectura se define por límites que separan las políticas de alto nivel de los detalles de bajo nivel, por lo que los servicios que solo separan comportamientos de aplicación pueden no tener un peso arquitectónico significativo.

2.Pregunta

¿Cómo desafía el autor la suposición de que los servicios están fuertemente desacoplados?

Respuesta:Martin afirma que, aunque los servicios pueden parecer desacoplados ya que operan en procesos diferentes, pueden estar aun así fuertemente acoplados a través de recursos o datos compartidos. Los cambios en estructuras de

Más Libros Gratis
Escanea para descargar
Escúchalo

datos compartidos pueden requerir actualizaciones en múltiples servicios, revelando que la percepción de desacoplamiento a menudo es ilusoria.

3.Pregunta

¿Puedes dar un ejemplo de las complicaciones que pueden surgir de una arquitectura orientada a servicios, como se discute en el capítulo?

Respuesta:Un ejemplo ilustrativo es el sistema de agregador de taxis donde agregar una función de entrega de gatitos requiere cambios en todos los servicios existentes. Esto destacó que, a pesar de haber sido diseñados como servicios independientes, estaban inherentemente acoplados debido a su dependencia de datos y funcionalidades compartidos.

4.Pregunta

¿Qué papel juegan los principios de diseño orientado a objetos en la gestión de la arquitectura de servicios?

Respuesta:Los principios de diseño orientado a objetos, particularmente los principios SOLID, fomentan la creación de componentes flexibles y desacoplados. Esto permite agregar nuevas características o servicios sin alterar el código

Más Libros Gratis
Escanea para descargar
Escúchalo

existente, mejorando así la integridad arquitectónica en medio de una complejidad creciente.

5.Pregunta

¿Qué implica el autor sobre la responsabilidad de los arquitectos de software?

Respuesta:Los arquitectos de software deben reconocer que la arquitectura requiere una cuidadosa planificación y previsión. Necesitan diseñar sistemas que permitan cambios y mejoras futuras sin entrelazar nuevas características con las existentes, asegurando un equilibrio entre flexibilidad y mantenibilidad.

6.Pregunta

¿De qué manera se comparan los servicios con llamadas a funciones costosas?

Respuesta:Martin explica que si los servicios simplemente existen para delinear comportamientos sin imponer límites arquitectónicos sustanciales o separación de preocupaciones, pueden compararse a llamadas a funciones costosas, ya que no contribuyen de manera significativa a la arquitectura del

Más Libros Gratis
Escanea para descargar
Escúchalo

sistema.

7.Pregunta

¿Qué se entiende por 'preocupaciones transversales' en el contexto de la arquitectura de servicios?

Respuesta:Las preocupaciones transversales se refieren a características o funcionalidades que afectan a múltiples partes de un sistema de manera enredada, en lugar de estar localizadas. Esto puede llevar a dependencias donde los cambios en un servicio requieren ajustes en otros, complicando el mantenimiento y la escalabilidad.

8.Pregunta

¿Cómo sugiere el capítulo que se debería diseñar la estructura de los servicios?

Respuesta:El capítulo enfatiza que la arquitectura de los servicios debería estar orientada hacia adentro, diseñada de acuerdo con la Regla de Dependencia. Los servicios deben tener sus estructuras internas definidas por componentes que se adhieran a esta regla, aislando sus complejas interdependencias.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

Escanear para descargar

Por qué Bookey es una aplicación imprescindible para los amantes de los libros

Contenido de 30min
Cuanto más profunda y clara sea la interpretación que proporcionamos, mejor comprensión tendrás de cada título.

Formato de texto y audio
Absorbe conocimiento incluso en tiempo fragmentado.

Preguntas
Comprueba si has dominado lo que acabas de aprender.

Y más
Múltiples voces y fuentes, Mapa mental, Citas, Clips de ideas...

Prueba gratuita con Bookey

Capítulo 28 | El Límite de las Pruebas| Preguntas y respuestas

1.Pregunta

¿Cómo participan las pruebas en la arquitectura de software?

Respuesta:Las pruebas se consideran componentes integrales de la arquitectura de software, participando de la misma manera que otras partes del sistema. Siguen la Regla de Dependencia, dependiendo hacia el código de la aplicación en lugar de ser dependientes de este. Esta posición arquitectónica permite que las pruebas apoyen el desarrollo mientras son desplegables de manera independiente.

2.Pregunta

¿Cuál es el impacto de las pruebas frágiles en los sistemas de software?

Respuesta:Las pruebas frágiles pueden hacer que un sistema sea rígido y difícil de cambiar porque están fuertemente acopladas a la implementación. Cuando ocurren cambios en

Más Libros Gratis
Escanea para descargar
Escúchalo

el código de producción, estas pruebas a menudo fallan, desalentando a los desarrolladores de hacer los cambios necesarios en el sistema.

3.Pregunta

¿Cuál es la solución para mitigar el problema de las pruebas frágiles?

Respuesta:Diseñar para la testabilidad creando una API de pruebas específica que permita a las pruebas verificar las reglas de negocio sin depender de elementos volátiles como las interfaces gráficas de usuario. Esto permite que las pruebas operen a un nivel más alto de abstracción, haciéndolas más estables y menos dependientes de la estructura de la aplicación.

4.Pregunta

¿Por qué es importante desacoplar la estructura de las pruebas de la aplicación?

Respuesta:El desacoplamiento permite que tanto las pruebas como el código de producción evolucionen de manera independiente. Si las pruebas están estrechamente ligadas a

Más Libros Gratis
Escanea para descargar
Escúchalo

las especificaciones de la estructura de la aplicación, cualquier cambio en el código de producción puede llevar a muchas pruebas rotas, complicando el mantenimiento y desalentando cambios en el código.

5.Pregunta

¿Cuál es el propósito de una API de pruebas?

Respuesta:Una API de pruebas sirve para desacoplar las pruebas de la aplicación, creando un intermediario que permite a las pruebas interactuar con la aplicación sin requerir conocimiento de su estructura. Esto facilita modificaciones más fáciles tanto en las pruebas como en el código de producción.

6.Pregunta

¿Qué es el 'acoplamiento estructural' en el contexto de las pruebas?

Respuesta:El acoplamiento estructural se refiere a una relación estrecha donde las pruebas están directamente influenciadas por la estructura del código que validan. Si un conjunto de pruebas refleja la estructura de la aplicación, los

Más Libros Gratis
Escanea para descargar
Escúchalo

cambios en esa estructura pueden llevar a fallos generalizados en las pruebas.

7.Pregunta

¿Cómo deberían verse las pruebas en términos de importancia para el sistema?

Respuesta:Las pruebas deben ser consideradas como componentes vitales del sistema que requieren un diseño adecuado para asegurar que sigan siendo efectivas, mantenibles y de apoyo al desarrollo continuo sin causar sobrecarga innecesaria o rigidez.

8.Pregunta

¿Qué implica 'diseñar para la testabilidad'?

Respuesta:Diseñar para la testabilidad implica crear una arquitectura de sistema que facilite las pruebas al permitir que estas validen la funcionalidad sin estar estrechamente integradas con otros componentes de la aplicación, especialmente características volátiles como la interfaz de usuario.

9.Pregunta

¿Qué características deberían incorporar las pruebas

Más Libros Gratis
Escanea para descargar
Escúchalo

para ser efectivas?

Respuesta:Las pruebas efectivas deberían ser independientes, desacopladas de las especificaciones de la estructura de la aplicación, fácilmente mantenibles y capaces de funcionar sin la interfaz de usuario del sistema u otras dependencias externas.

10.Pregunta

¿Por qué es crucial evitar dependencias volátiles en el diseño de pruebas?

Respuesta:Evitar dependencias volátiles es crucial porque las pruebas que dependen de partes inestables de la aplicación son más propensas a romperse con cambios menores en el código, lo que puede llevar a una reducción de la confianza en el conjunto de pruebas y un aumento en la sobrecarga de mantenimiento.

Capítulo 29 | Arquitectura Embebida Limpia| Preguntas y respuestas

1.Pregunta

¿Cuál es la importancia de separar el software del firmware según James Grenning?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Grenning enfatiza que, aunque el software puede tener una larga vida útil, el firmware puede volverse obsoleto y crear dependencias no gestionadas que pueden 'infectar' el software, restringiendo su longevidad. Al separar el software del firmware, podemos mantener una Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software que permite actualizaciones y modificaciones más fáciles sin las complicaciones que imponen las dependencias de hardware.

2.Pregunta

¿Qué es la prueba de App-titude y por qué es importante para los desarrolladores de sistemas embebidos?

Respuesta:La prueba de App-titude consiste en tres etapas: primero hacerlo funcionar, luego hacerlo bien, y finalmente hacerlo rápido. Es importante que los desarrolladores de sistemas embebidos entiendan que centrarse únicamente en hacer que la aplicación funcione puede conducir a una mala

Más Libros Gratis
Escanea para descargar
Escúchalo

arquitectura. Los desarrolladores deben ser conscientes de estructurar su código para la longevidad y el mantenimiento.

3.Pregunta

¿Cómo afecta el cuello de botella del hardware objetivo al desarrollo de software embebido?

Respuesta:El cuello de botella del hardware objetivo se refiere a la situación en la que los desarrolladores solo pueden probar su código en el hardware real, lo que puede ralentizar significativamente el proceso de desarrollo. Al estructurar el software embebido con los principios de Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software, los desarrolladores pueden mitigar este cuello de botella y facilitar pruebas fuera del objetivo.

4.Pregunta

¿Cuáles son algunas de las estrategias mencionadas por Grenning para mantener limpia la arquitectura del software embebido?

Respuesta:Las estrategias incluyen usar una Capa de Abstracción de Hardware (HAL) para aislar los aspectos específicos del hardware del software, definir interfaces

Más Libros Gratis
Escanea para descargar
Escúchalo

claras para firmware y software, asegurarse de que la estructura del software no revele detalles de hardware, y aplicar principios como el Principio de Inversión de Dependencias para mantener la separación y minimizar el acoplamiento.

5.Pregunta

¿Cuál es la importancia de la arquitectura en capas en los sistemas embebidos según Grenning?

Respuesta:La arquitectura en capas permite a los desarrolladores separar preocupaciones dentro del sistema, asegurando que las dependencias de hardware, firmware y software se gestionen adecuadamente. Esta organización apoya mejores pruebas, mantenimiento y la capacidad de adaptarse a cambios sin reescrituras extensas, contribuyendo así a la longevidad del software.

6.Pregunta

¿Qué sugiere Grenning que conduce a la destrucción de la funcionalidad potencial del software a largo plazo?

Respuesta:Grenning sugiere que las dependencias no

Más Libros Gratis
Escanea para descargar
Escúchalo

gestionadas en firmware y hardware pueden infectar el software embebido, llevando a su obsolescencia y haciéndolo incapaz de adaptarse a nuevos requisitos, lo que destruye efectivamente su potencial para una funcionalidad a largo plazo.

7.Pregunta

¿Cuáles son las tres actividades descritas por Kent Beck en la construcción de software y cómo deberían ser priorizadas?

Respuesta:Las tres actividades son: 1) Hacer que funcione - asegurando funcionalidad; 2) Hacerlo bien - refactorizando para mejorar la claridad y la adaptabilidad; y 3) Hacerlo rápido - optimizando el rendimiento. La prioridad debe ser primero asegurarse de la corrección antes de refactorizar y optimizar, ya que esto sienta las bases para un desarrollo de software sostenible.

8.Pregunta

¿Por qué los desarrolladores embebidos deberían evitar escribir demasiado firmware según Grenning?

Respuesta:Incorporar demasiado firmware puede atar el

Más Libros Gratis
Escanea para descargar
Escúchalo

software a hardware específico, haciéndolo frágil y difícil de modificar cuando ocurren cambios en el hardware. Esto puede anular la potencial longevidad del software al crear dependencias que son difíciles de gestionar.

9.Pregunta

¿Cómo relaciona Grenning los conceptos de firmware y software con la naturaleza en evolución del hardware?

Respuesta:Explica que, aunque el software puede envejecer bien, puede verse negativamente afectado por la rápida evolución del firmware y hardware a menos que se gestione con cuidado. Las arquitecturas bien estructuradas y limpias pueden ayudar a mitigar estos impactos al asegurar separaciones más claras de las capas en relación con el hardware.

10.Pregunta

¿Qué es la capa de abstracción de hardware (HAL) y por qué es crucial en los sistemas embebidos?

Respuesta:La HAL es una interfaz que abstrae los detalles del hardware del software, permitiendo que este último

Más Libros Gratis
Escanea para descargar
Escúchalo

permanezca independiente de implementaciones de hardware específicas. Esto es crucial para mantener la usabilidad y adaptabilidad del software a largo plazo a medida que evoluciona el hardware.

11.Pregunta

¿De qué manera sugiere Grenning que el desarrollo de sistemas embebidos difiere del desarrollo no embebido?

Respuesta:Grenning señala que los desarrolladores de sistemas embebidos enfrentan desafíos únicos, como restricciones de tiempo real, recursos limitados y dependencias de hardware que exigen un mayor nivel de consideración arquitectónica para asegurar la flexibilidad y la longevidad del software.

Capítulo 30 | La Base de Datos Es un Detalle| Preguntas y respuestas

1.Pregunta

¿Por qué se considera que la base de datos es un detalle en la arquitectura de software?

Respuesta:La base de datos se considera un detalle porque es meramente una herramienta que

Más Libros Gratis
Escanea para descargar
Escúchalo

proporciona acceso a los datos, sin influir significativamente en la arquitectura general del sistema de software. Así como una manija es un detalle en la arquitectura de un hogar, la mecánica de la base de datos no debería eclipsar las políticas y estructuras primarias que definen el sistema.

2.Pregunta

¿Cuál es la clave diferencia entre una base de datos y un modelo de datos desde una perspectiva arquitectónica?

Respuesta:El modelo de datos da forma a cómo se estructuran y utilizan los datos dentro de la aplicación, lo cual es arquitectónicamente significativo. En contraste, la base de datos en sí es una tecnología implementada que sirve para el almacenamiento y recuperación de datos, y se considera un detalle de bajo nivel.

3.Pregunta

¿Qué ha contribuido a la prevalencia de los sistemas de bases de datos en la arquitectura de software?

Respuesta:Los sistemas de bases de datos se han vuelto

Más Libros Gratis
Escanea para descargar
Escúchalo

prevalentes en gran medida debido a la dependencia histórica de la tecnología de disco para el almacenamiento de datos, que ha sido la base durante décadas, impulsando la adopción de modelos relacionales por su elegancia y eficiencia.

4.Pregunta

¿Cómo deberían manejar las arquitecturas el acceso a la base de datos para evitar acoplar las reglas de negocio a las estructuras de bases de datos relacionales?

Respuesta:Las arquitecturas deberían encapsular el acceso a datos en funciones utilitarias o puertas de enlace que interactúen con la base de datos, restringiendo el conocimiento de la estructura relacional de la base de datos a las capas externas de la arquitectura. Este enfoque evita el acoplamiento directo de las reglas de negocio al diseño de la base de datos.

5.Pregunta

¿Qué sugiere el autor sobre las preocupaciones de rendimiento relacionadas con las bases de datos en la arquitectura?

Respuesta:Las preocupaciones de rendimiento deben ser

Más Libros Gratis
Escanea para descargar
Escúchalo

encapsuladas como detalles de implementación de bajo nivel, manejadas por mecanismos de acceso a datos eficientes, separados de las reglas de negocio de alto nivel y de la arquitectura general del sistema.

6.Pregunta

¿Por qué es ventajoso desarrollar software sin una base de datos durante las fases iniciales de un proyecto?

Respuesta:Evitar una base de datos en las fases tempranas previene que el equipo se vea abrumado con complejidades no deseadas relacionadas con el esquema de la base de datos, las consultas y las conexiones, permitiendo un desarrollo rápido y decisiones arquitectónicas flexibles.

7.Pregunta

¿Qué lección transmite la anécdota sobre la reticencia de la startup a usar un RDBMS?

Respuesta:La anécdota ilustra que a veces las decisiones arquitectónicas son impulsadas por las expectativas del cliente en lugar de las necesidades técnicas. Como resultado, los arquitectos deben tener cuidado al integrar tecnologías de

Más Libros Gratis
Escanea para descargar
Escúchalo

bajo nivel basadas únicamente en presiones del mercado.

8.Pregunta

¿Cómo pueden los arquitectos asegurar que sus sistemas se mantengan flexibles a medida que evolucionan los requisitos?

Respuesta:Al mantener límites arquitectónicos claros y enfatizar políticas de alto nivel mientras posponen decisiones sobre detalles de bajo nivel como bases de datos o frameworks, los arquitectos pueden acomodar mejor futuros cambios y requisitos.

9.Pregunta

¿Cuál es el significado de la Regla de Dependencia en Arquitectura Limpia?

Respuesta:La Regla de Dependencia exige que las dependencias del código fuente solo apunten hacia adentro, hacia políticas de nivel superior, asegurando que los componentes internos permanezcan inalterados por cambios en los componentes externos, promoviendo así la mantenibilidad y flexibilidad.

10.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

En el contexto de Arquitectura Limpia, ¿por qué se consideran significativas las entidades?

Respuesta:Las entidades encapsulan reglas de negocio críticas y representan la lógica central de la aplicación, haciéndolas centrales para la arquitectura. Su independencia de los detalles de implementación de bajo nivel les permite ser reutilizadas en diferentes aplicaciones sin verse afectadas por cambios en la tecnología.

Más Libros Gratis

Escanea para descargar

Escúchalo

Ad

App Store
Selección editorial

22k reseñas de 5 estrellas

Escanear para descargar

Retroalimentación Positiva

Alondra Navarrete

itas después de cada resumen
en a prueba mi comprensión,
cen que el proceso de
rtido y atractivo."

¡Fantástico! Beltrán Fuentes

Me sorprende la variedad de libros e idiomas que soporta Bookey. No es solo una aplicación, es una puerta de acceso al conocimiento global. Además, ganar puntos para la caridad es un gran plus!

Fi

Lo
re
co
pr

a Vásquez

nábito de
le y sus
o que el
todos.

¡Me encanta! Darian Rosales

Bookey me ofrece tiempo para repasar las partes importantes de un libro. También me da una idea suficiente de si debo o no comprar la versión completa del libro. ¡Es fácil de usar!

¡Ahorra tiempo!

Bookey es mi aplicación de
crecimiento intelectual. Lo
perspicaces y bellamente c
acceso a un mundo de con

icación increíble! Elvira Jiménez

ncantan los audiolibros pero no siempre tengo tiempo
escuchar el libro entero. ¡Bookey me permite obtener
esumen de los puntos destacados del libro que me
esa! ¡Qué gran concepto! ¡Muy recomendado!

Aplicación hermosa

Esta aplicación es un salvavidas para los a
los libros con agendas ocupadas. Los resu
precisos, y los mapas mentales ayudan a
que he aprendido. ¡Muy recomendable!

Prueba gratuita con Bookey

Capítulo 31 | La Web Es un Detalle| Preguntas y respuestas

1.Pregunta

¿Qué quiere decir Robert C. Martin al afirmar que 'la web es un detalle'?

Respuesta:Robert C. Martin argumenta que la web no cambia fundamentalmente la arquitectura de los sistemas de software; es simplemente un nuevo mecanismo para presentar aplicaciones. Por lo tanto, no debería dictar la lógica de negocio central y la estructura de las aplicaciones. Él enfatiza la importancia de separar la interfaz de usuario de las reglas de negocio para mantener la flexibilidad frente a las presiones de marketing y los cambios tecnológicos.

2.Pregunta

¿Cómo pueden los arquitectos de software proteger las aplicaciones de cambios innecesarios impulsados por demandas de marketing?

Respuesta:Los arquitectos pueden proteger las aplicaciones

Más Libros Gratis
Escanea para descargar
Escúchalo

desacoplando las reglas de negocio de la interfaz de usuario (UI). Esto significa diseñar la aplicación de tal manera que la lógica de negocio central sea independiente de cómo se presenta a los usuarios. Al centrarse en crear interfaces que permanezcan constantes incluso cuando cambia la UI, los arquitectos pueden prevenir alteraciones forzadas debido a tendencias o direcciones de marketing.

3.Pregunta

¿Cuáles son las implicaciones de las oscilaciones interminables entre la computación centralizada y descentralizada, según lo discutido en el texto?

Respuesta:Estas oscilaciones destacan la lucha continua de la industria por determinar la distribución óptima del poder computacional. Este vaivén indica que las arquitecturas de software deben ser resilientes y adaptables a los cambios en la tecnología sin afectar las funcionalidades centrales del negocio.

4.Pregunta

¿Por qué es importante que las reglas de negocio estén aisladas de detalles de menor nivel, como tecnologías de

Más Libros Gratis
Escanea para descargar
Escúchalo

UI y bases de datos?

Respuesta:Es crucial que las reglas de negocio permanezcan aisladas porque los cambios en la UI o en las tecnologías de bases de datos podrían llevar a esfuerzos de rediseño significativos si están directamente entrelazadas con la lógica de negocio. Tal aislamiento asegura que las reglas de negocio puedan evolucionar con las necesidades de la organización mientras se mantiene la estabilidad en la experiencia del usuario y la gestión de datos.

5.Pregunta

¿Qué ejemplo proporciona Martin sobre una empresa que se adaptó mal a las tecnologías web y qué podrían haber hecho de manera diferente?

Respuesta:Martin comparte la historia de la Empresa Q, que modificó su interfaz de usuario para asemejarse a un navegador web debido a las tendencias del mercado, lo que resultó en insatisfacción entre los usuarios. Si la arquitectura hubiera desacoplado las reglas de negocio centrales de los componentes de UI, podrían haber adaptado la interfaz sin

Más Libros Gratis
Escanea para descargar
Escúchalo

afectar la lógica de negocio, permitiendo una transición más suave y protegiendo la experiencia del usuario.

6.Pregunta

¿Cuál es la 'Regla de Dependencia' mencionada en la discusión de Martin sobre Arquitectura Limpia?

Respuesta:La Regla de Dependencia establece que las dependencias del código fuente deben apuntar hacia adentro, hacia políticas de mayor nivel y lógica de negocio, en lugar de hacia afuera, hacia detalles de menor nivel como marcos, componentes de UI y bases de datos. Esto asegura que las reglas de negocio de alto nivel estén aisladas de cambios en implementaciones de menor nivel.

7.Pregunta

En el contexto de Arquitectura Limpia, ¿qué papel juegan los casos de uso?

Respuesta:Los casos de uso representan reglas de negocio específicas de la aplicación, dirigiendo cómo las interacciones de los usuarios corresponden a las funciones del sistema. Sirven para orquestar la interacción entre la

Más Libros Gratis
Escanea para descargar
Escúchalo

interfaz de usuario y la lógica de negocio central, asegurando que la funcionalidad se adhiera a los requisitos establecidos para la aplicación.

8.Pregunta

¿Cómo diferencia Martin entre 'reglas de negocio' y 'casos de uso'?

Respuesta:Las reglas de negocio son principios generales que rigen cómo opera una organización y cuáles son sus decisiones, mientras que los casos de uso son implementaciones específicas de esas reglas dentro del contexto de una aplicación. Los casos de uso proporcionan un enfoque estructurado sobre cómo los usuarios interactúan con la lógica de negocio en el software.

9.Pregunta

¿Qué lecciones se pueden derivar sobre la gestión de componentes de software y sus dependencias?

Respuesta:La gestión eficaz de los componentes de software y sus dependencias se basa en mantener límites claros, adherirse a principios de desacoplamiento y entender que los

Más Libros Gratis
Escanea para descargar
Escúchalo

componentes de alto nivel no deben depender de componentes de menor nivel y volátiles. Este enfoque permite flexibilidad, un mantenimiento más fácil y reduce el riesgo de errores durante las modificaciones.

Capítulo 32 | Los Frameworks Son Detalles| Preguntas y respuestas

1.Pregunta

¿Cuál es el mensaje principal sobre los frameworks en la arquitectura de software?

Respuesta:Los frameworks no son arquitecturas; son detalles. Ayudan a desarrollar software, pero también pueden imponer un acoplamiento significativo, dificultando su cambio o sustitución posteriormente. Debes mantener a los frameworks a distancia y no permitir que dicten la estructura de tu aplicación central.

2.Pregunta

¿Por qué es arriesgado depender en gran medida de un framework para un proyecto de software?

Respuesta:Porque los frameworks pueden evolucionar,

Más Libros Gratis
Escanea para descargar
Escúchalo

imponer estructuras rígidas y pueden volverse obsoletos. Una vez que te acoplas estrechamente a un framework, cambiar a una solución mejor puede volverse extremadamente difícil.

3.Pregunta

¿Cómo deberían integrarse los frameworks en una arquitectura?

Respuesta:Los frameworks deben ser tratados como detalles que pertenecen a los círculos exteriores de la arquitectura. Utilízalos sin acoplar estrechamente la lógica de negocio central a la implementación del framework.

4.Pregunta

¿Qué consejo da Robert C. Martin sobre la relación entre los desarrolladores y los autores de frameworks?

Respuesta:Los autores de frameworks generalmente no tienen en mente los mejores intereses de los desarrolladores porque no comprenden los problemas específicos de los desarrolladores. Los desarrolladores deben ser cautelosos y críticos con respecto a los frameworks.

5.Pregunta

¿Cuál es la metáfora utilizada para describir la relación

Más Libros Gratis
Escanea para descargar
Escúchalo

con los frameworks?

Respuesta:La metáfora es similar al matrimonio. Utilizar un framework requiere un compromiso, y una vez que ese compromiso se establece, puede ser difícil separarse.

6.Pregunta

¿Qué deben hacer los desarrolladores si deben utilizar un framework específico?

Respuesta:Deben mantener la flexibilidad de su arquitectura mediante la creación de proxies o envolviendo el framework, en lugar de acoplarse directamente a él. Esto permite realizar cambios sin reescrituras significativas o compromisos.

7.Pregunta

¿Por qué es importante mantener los frameworks fuera del código de negocio central?

Respuesta:Preserva la independencia de las reglas de negocio de las dependencias externas, lo que permite un mantenimiento, pruebas y posible sustitución de diferentes tecnologías más fácil en el futuro.

8.Pregunta

¿De qué manera pueden los principios de Arquitectura

Más Libros Gratis
Escanea para descargar
Escúchalo

Limpia ayudar en el mantenimiento a largo plazo del software?

Respuesta:Al aislar y encapsular las reglas de negocio centrales, permitiendo una testabilidad independiente de los frameworks y posponiendo decisiones sobre tecnologías externas, la Arquitectura Limpia mejora la adaptabilidad y reduce la deuda técnica.

9.Pregunta

¿Cómo pueden los desarrolladores evitar acoplar demasiado sus aplicaciones a un framework que están utilizando?

Respuesta:Los desarrolladores pueden crear interfaces abstractas y utilizar la inyección de dependencias para mantener una capa de separación entre la lógica de su aplicación y las operaciones del framework.

10.Pregunta

Resume el riesgo de construir software que esté demasiado integrado con un framework específico.

Respuesta:El riesgo es que cualquier cambio en el framework o sus API podría requerir cambios significativos en el código

Más Libros Gratis
Escanea para descargar
Escúchalo

de la aplicación, lo que llevaría a costos de mantenimiento incrementados y a una reducción de la flexibilidad en el futuro.

Capítulo 33 | Estudio de Caso: Ventas de Videos| Preguntas y respuestas

1.Pregunta

¿Cuáles son las consideraciones clave que deben tenerse en cuenta al diseñar la arquitectura de software para un sistema de venta de videos?

Respuesta:Las consideraciones clave incluyen identificar a los principales actores (como usuarios individuales, clientes comerciales, autores de videos y administradores), organizar los casos de uso en torno a estos actores para permitir cambios independientes, y emplear principios como el principio de responsabilidad única para garantizar que cada componente cumpla un propósito distinto sin enlaces innecesarios.

2.Pregunta

¿Cómo influye la Regla de Dependencia en la

Más Libros Gratis
Escanea para descargar
Escúchalo

arquitectura del sistema?

Respuesta:La Regla de Dependencia establece que las dependencias del código fuente deben apuntar hacia adentro, hacia políticas de más alto nivel, lo que ayuda a asegurar que los cambios en componentes de bajo nivel no afecten negativamente las reglas o políticas de negocio de más alto nivel.

3.Pregunta

¿Cuál es la importancia del análisis de casos de uso en la arquitectura de software?

Respuesta:El análisis de casos de uso ayuda a identificar las interacciones entre los usuarios y el sistema, asegurando que la arquitectura esté diseñada para soportar las funcionalidades específicas requeridas por diferentes actores, manteniendo así claridad y enfoque en las necesidades del usuario.

4.Pregunta

¿Por qué es importante mantener la base de datos como un detalle en la arquitectura?

Más Libros Gratis
Escanea para descargar
Escúchalo

Respuesta:Mantener la base de datos como un detalle evita que afecte la lógica de negocio central de la aplicación. Esto permite mayor flexibilidad al cambiar soluciones de gestión de datos sin impactar la arquitectura general del sistema.

5.Pregunta

¿Cuáles son algunas maneras de asegurar que la arquitectura soporte el desarrollo y despliegue independientes?

Respuesta:Utilizar arquitecturas en capas para separar preocupaciones, definir interfaces claras para minimizar dependencias, e implementar componentes que puedan ser desplegados o probados de forma aislada para garantizar que los equipos puedan trabajar en partes del sistema sin interferir entre sí.

6.Pregunta

¿Cómo se aplica el concepto de 'mantener las opciones abiertas' a las decisiones arquitectónicas?

Respuesta:Diseñar la arquitectura de manera que se pospongan las decisiones relacionadas con frameworks, bases de datos e interfaces de usuario hasta que sean

Más Libros Gratis
Escanea para descargar
Escúchalo

absolutamente necesarias permite mayor agilidad y capacidad de respuesta ante cambios, minimizando costos y riesgos a largo plazo.

7.Pregunta

¿Cuál es el papel de los presentadores en la arquitectura de software según se discute en el capítulo?

Respuesta:Los presentadores sirven como un puente entre la interfaz de usuario (que puede ser un 'objeto humilde') y la lógica de negocio, formateando los datos para su presentación sin contener ninguna regla de negocio, lo que permite una prueba más fácil y mantiene una separación clara entre la interfaz de usuario y la lógica central de la aplicación.

8.Pregunta

¿Qué lecciones se pueden aprender del estudio de caso de ventas de videos en relación con la arquitectura de software?

Respuesta:El estudio de caso resalta la importancia de identificar actores, estructurar los casos de uso en función de sus necesidades, aplicar principios arquitectónicos para

Más Libros Gratis
Escanea para descargar
Escúchalo

mantener la separación de preocupaciones, y asegurar que la arquitectura sea lo suficientemente flexible para adaptarse a cambios mientras minimiza la interrupción en la lógica de negocio central.

9.Pregunta

¿Por qué se recomienda tratar los frameworks como detalles en una arquitectura?

Respuesta:Los frameworks pueden imponer un acoplamiento y restricciones innecesarias en la arquitectura. Tratarlos como detalles permite que el sistema se mantenga flexible y desacoplado de implementaciones específicas, lo que favorece mejores pruebas, mantenimiento y posibles cambios en el futuro.

10.Pregunta

¿Cuál es la importancia de las pruebas en el contexto del diseño arquitectónico según se discute?

Respuesta:Las pruebas deben integrarse en el diseño arquitectónico, asegurando que las reglas de negocio se puedan validar sin depender de interfaces externas o de la

Más Libros Gratis
Escanea para descargar
Escúchalo

dependencia del hardware, promoviendo así un código más limpio y mayor fiabilidad a lo largo del ciclo de vida del sistema.

Más Libros Gratis
Escanea para descargar
Escúchalo

Ad

Escanear para descargar

Leer, Compartir, Empoderar

Completa tu desafío de lectura, dona libros a los niños africanos.

El Concepto

BOOKS FOR AFRICA × ×

Esta actividad de donación de libros se está llevando a cabo junto con Books For Africa. Lanzamos este proyecto porque compartimos la misma creencia que BFA: Para muchos niños en África, el regalo de libros realmente es un regalo de esperanza.

La Regla

Gana 100 puntos → **Canjea un libro** → **Dona a África**

Tu aprendizaje no solo te brinda conocimiento sino que también te permite ganar puntos para causas benéficas. Por cada 100 puntos que ganes, se donará un libro a África.

Prueba gratuita con Bookey

Capítulo 34 | El Capítulo Faltante| Preguntas y respuestas

1.Pregunta

¿Por qué es importante separar el código en componentes basados en su funcionalidad?

Respuesta:Separar el código en componentes permite una mejor organización, límites más claros y dependencias controladas. Ayuda a gestionar la complejidad y mejora el mantenimiento, facilitando la adaptación a cambios en los requisitos o en la tecnología.

2.Pregunta

¿Cuáles son los aspectos negativos de una arquitectura en capas tradicional?

Respuesta:La arquitectura en capas tradicional puede volverse complicada a medida que el sistema crece, lo que lleva a capas interconectadas grandes que son difíciles de gestionar. A menudo carece de claridad específica del dominio, lo que dificulta entender la lógica de negocio.

3.Pregunta

Más Libros Gratis
Escanea para descargar
Escúchalo

¿Qué significa 'paquete por característica' y por qué es beneficioso?

Respuesta:El 'paquete por característica' organiza el código en torno a características o dominios de negocio distintos en lugar de por capas técnicas. Este enfoque facilita la localización del código relacionado, promueve una mejor cohesión y mejora la legibilidad del código.

4.Pregunta

¿Cómo ayuda la arquitectura de puertos y adaptadores en la desacoplamiento?

Respuesta:Los puertos y adaptadores crean una separación entre la lógica del dominio y los detalles de implementación técnica, permitiendo que la lógica de negocio central permanezca ajena a factores externos como marcos o bases de datos.

5.Pregunta

¿Cuál es el papel de la 'Regla de Dependencia' en la arquitectura?

Respuesta:La Regla de Dependencia establece que las

Más Libros Gratis
Escanea para descargar
Escúchalo

dependencias del código fuente deben apuntar hacia adentro, hacia políticas de nivel superior. Esto significa que las capas internas no deben conocer nada de las capas externas, promoviendo la separación de preocupaciones y mejorando el mantenimiento.

6.Pregunta

¿Por qué el uso de modificadores de acceso públicos en Java puede ser potencialmente dañino?

Respuesta:El uso excesivo de modificadores de acceso públicos puede exponer detalles de implementación y crear un acoplamiento innecesario entre componentes, lo que lleva a dificultades en el mantenimiento o cambio del código sin afectar a otras partes del sistema.

7.Pregunta

¿Cuáles son algunas opciones arquitectónicas disponibles para organizar las dependencias del código fuente?

Respuesta:Las opciones incluyen empaquetar por capas, empaquetar por características, usar marcos modulares como OSGi, o separar el código en diferentes estructuras para la

Más Libros Gratis
Escanea para descargar
Escúchalo

lógica del dominio y la infraestructura.

8.Pregunta

¿Cómo puede aplicar la encapsulación a través de modificadores de acceso mejorar la arquitectura del software?

Respuesta:Al usar modificadores de acceso más restrictivos, los desarrolladores pueden controlar mejor qué partes de la base de código pueden acceder a ciertos componentes. Esto reduce el potencial de dependencias no intencionadas y mejora la integridad de la arquitectura.

9.Pregunta

¿Cuál es la importancia de desacoplar capas en una arquitectura de software?

Respuesta:Desacoplar capas permite que partes individuales del software evolucionen de forma independiente. Esto significa que los cambios en una capa no necesariamente requerirán cambios en otra, lo que hace que el sistema sea más adaptable y resistente al cambio.

10.Pregunta

¿Por qué deberían mantenerse separadas las reglas de

Más Libros Gratis
Escanea para descargar
Escúchalo

negocio de las preocupaciones de la interfaz de usuario?
Respuesta:Mantener las reglas de negocio separadas de la interfaz de usuario asegura que los cambios en la UI no impacten la funcionalidad central de la lógica de negocio y viceversa. Esto permite escalabilidad y flexibilidad para adaptarse a nuevas tecnologías o diseños.

11.Pregunta

¿Cuál es el principio principal detrás de la Arquitectura Limpia según Robert C. Martin?
Respuesta:El principio principal de la Arquitectura Limpia es estructurar el software de tal manera que la lógica de negocio central sea independiente de preocupaciones externas, como marcos y UI, lo que facilita el cambio o reemplazo de esos detalles sin afectar la funcionalidad central.

12.Pregunta

¿Cómo puede una API de pruebas mejorar la capacidad de prueba de un sistema?
Respuesta:Una API de pruebas puede proporcionar acceso directo a la lógica de negocio del sistema mientras elude la

Más Libros Gratis
Escanea para descargar
Escúchalo

UI y otras complejidades. Esto permite pruebas más enfocadas, reduciendo las dependencias en la infraestructura del sistema y mejorando su fiabilidad.

13.Pregunta

¿Cuáles son los beneficios de definir límites claros en la arquitectura de software?

Respuesta:Definir límites claros ayuda a aislar componentes, reduciendo las dependencias que pueden crear problemas de mantenimiento. También mejora la claridad en cómo interactúan los componentes, facilita cambios más sencillos y apoya pruebas efectivas.

14.Pregunta

¿Qué se entiende por 'programar hacia interfaces'?

Respuesta:Programar hacia interfaces se refiere a la práctica de diseñar software de manera que las clases y componentes interactúen a través de interfaces definidas en lugar de implementaciones concretas. Esto promueve la flexibilidad y permite una integración más fácil de nuevos componentes.

15.Pregunta

¿Por qué es crucial seguir los principios SOLID en el

Más Libros Gratis
Escanea para descargar
Escúchalo

diseño de software?

Respuesta:Seguir los principios SOLID ayuda a crear sistemas que son robustos, mantenibles y fácilmente cambiables, llevando a una mayor calidad del software que puede adaptarse mejor a los requisitos comerciales en evolución.

16.Pregunta

¿Cómo presentan patrones arquitectónicos como microservicios tanto oportunidades como desafíos?

Respuesta:Los microservicios permiten el desarrollo y la escalabilidad independientes de los servicios, fomentando flexibilidad y resiliencia; sin embargo, también introducen complejidad en términos de comunicación, gestión de datos y cohesión del sistema en general.

17.Pregunta

¿Qué ciclo de problemas de desarrollo puede surgir de un sistema con acoplamiento estrecho?

Respuesta:El acoplamiento estrecho puede resultar en problemas de integración frecuentes, donde un cambio

Más Libros Gratis
Escanea para descargar
Escúchalo

requiere cambios en muchos módulos, lo que dificulta el mantenimiento, la extensión y provoca un aumento de errores y tiempo de desarrollo.

18.Pregunta

¿Cómo se aplica el concepto de 'El Diablo está en los Detalles' a la arquitectura de software?

Respuesta:Enfatiza que incluso una arquitectura bien planificada puede fallar si los detalles de implementación se manejan incorrectamente. Pequeños errores en la implementación pueden llevar a problemas significativos que comprometen la estructura y funcionalidad general.

19.Pregunta

¿Cuál es un anti-patrón común relacionado con el diseño de límites arquitectónicos?

Respuesta:Un anti-patrón común es permitir dependencias entre capas, donde detalles de bajo nivel impactan a componentes de alto nivel, llevando a un sistema acoplado que es difícil de probar, mantener y evolucionar.

20.Pregunta

¿Qué efecto tiene el uso indebido de la compilación

Más Libros Gratis
Escanea para descargar
Escúchalo

condicional en la mantenibilidad del código?
Respuesta:El uso indebido de la compilación condicional puede llevar a un código que es difícil de entender, desordenado con múltiples variaciones innecesarias, y aumenta la complejidad de mantener el código a través de diferentes plataformas.

21.Pregunta

¿Por qué es importante mantener una separación de preocupaciones en la arquitectura de software?
Respuesta:La separación de preocupaciones permite que diferentes aspectos del software evolucionen de manera independiente, simplificando la gestión del código, mejorando la claridad y aumentando la capacidad del sistema para adaptarse a cambios.

22.Pregunta

¿Qué enfoque debe seguirse al diseñar para la testabilidad en una arquitectura limpia?
Respuesta:Diseñar para la testabilidad implica asegurar que la lógica de negocio central esté desacoplada de marcos

Más Libros Gratis
Escanea para descargar
Escúchalo

externos, permitiendo que las pruebas unitarias se ejecuten en aislamiento, y confiar en interfaces y stubs para las pruebas.

Capítulo 35 | Apéndice A Arqueología de la Arquitectura| Preguntas y respuestas

1.Pregunta

¿Qué lección valiosa podemos aprender sobre la arquitectura de software a partir de la transición del Datanet 30 al Varian 620/f?

Respuesta:La transición del Datanet 30 al Varian 620/f nos enseña la importancia de adaptar la arquitectura para aprovechar nuevas tecnologías mientras se gestiona la complejidad. Al pasar al Varian, el equipo implementó un sistema de sobrecarga, lo que mejoró la eficiencia en el uso de la memoria y permitió que las aplicaciones intercambiaran datos dinámicamente. Esta decisión arquitectónica resalta la necesidad de flexibilidad y escalabilidad en el diseño de software.

2.Pregunta

¿Cómo evoluciona el concepto de límites de software en

Más Libros Gratis
Escanea para descargar
Escúchalo

los diferentes sistemas mencionados a lo largo del capítulo?

Respuesta:La evolución de los límites del software en el capítulo refleja una comprensión progresiva de la abstracción y el desacoplamiento. Inicialmente, los sistemas tenían límites difusos con componentes fuertemente acoplados (como se ve en el SAC), lo que conducía a pesadillas de mantenimiento. Con el tiempo, como se observa en el rediseño de DLU/DRU y 4-TEL, se formaron límites más claros, lo que facilitó el mantenimiento y la mejora de características, mostrando la importancia de definir interfaces y dependencias.

3.Pregunta

¿Qué revela la experiencia de desarrollar el proyecto de vectorización sobre los desafíos de las actualizaciones de firmware y software?

Respuesta:El proyecto de vectorización ilustra las complejidades involucradas en la gestión de actualizaciones de firmware y la importancia de crear componentes

Más Libros Gratis
Escanea para descargar
Escúchalo

modulares e independientes. La necesidad de recompilar los 30 chips para actualizaciones menores ejemplifica cómo el acoplamiento fuerte puede obstaculizar la agilidad. Al establecer un método para actualizar firmware de forma independiente a través de vectores, el equipo aprendió a mejorar la mantenibilidad y adaptabilidad de los sistemas embebidos.

4.Pregunta

¿Por qué es crítico escribir código limpio y aislado según la experiencia del SAC?

Respuesta:La experiencia del SAC enfatiza que el código limpio y aislado es vital para la mantenibilidad y escalabilidad. Un código mal organizado y fuertemente acoplado llevó a dificultades en el manejo de cambios de hardware y en la depuración. La realización de que la lógica de control de los módems estaba dispersa por toda la aplicación reforzó la necesidad de encapsular funcionalidades y promover la separación de preocupaciones, lo que en última instancia mejora la calidad del código y reduce la

Más Libros Gratis
Escanea para descargar
Escúchalo

deuda técnica.

5.Pregunta

¿Cómo impactó el salto a C y la programación estructurada en el desarrollo de software en Clear Communications?

Respuesta:La transición a C y la programación estructurada en Clear Communications marcó un cambio pivotal en las prácticas de desarrollo, permitiendo una organización más clara del código y una mejor gestión de la complejidad. Allana el camino para la adopción de principios orientados a objetos, lo que condujo a una arquitectura mejorada y bases de código más mantenibles a medida que los desarrolladores adoptaron metodologías estructuradas, influyendo en proyectos futuros.

6.Pregunta

¿Qué revela la historia del Recepcionista Electrónico sobre la comprensión del mercado en el desarrollo de productos?

Respuesta:El resultado del Recepcionista Electrónico destaca la importancia crítica de alinear el desarrollo de productos

Más Libros Gratis
Escanea para descargar
Escúchalo

con las necesidades del mercado. A pesar de contar con tecnología innovadora, el equipo no logró penetrar en el mercado de equipos de oficina debido a una falta de comprensión de los requisitos y estrategias de ventas de los clientes. Esto subraya que el dominio tecnológico por sí solo no es suficiente sin una estrategia sólida para la comercialización.

7.Pregunta

¿Qué lecciones podemos extraer del desarrollo y reescritura del marco ETS en cuanto a la importancia de la usabilidad antes de la reutilización?

Respuesta:La experiencia con el marco ETS subraya que la usabilidad debe preceder a la reutilización. Desarrollar un marco usable que acomode las necesidades inmediatas del proyecto garantiza que pueda ser reutilizado de manera efectiva más adelante. La falla inicial en crear un marco usable llevó a frustraciones y retrasos, enseñándonos que una comprensión profunda de los requisitos del usuario final es esencial para el diseño exitoso de marcos.

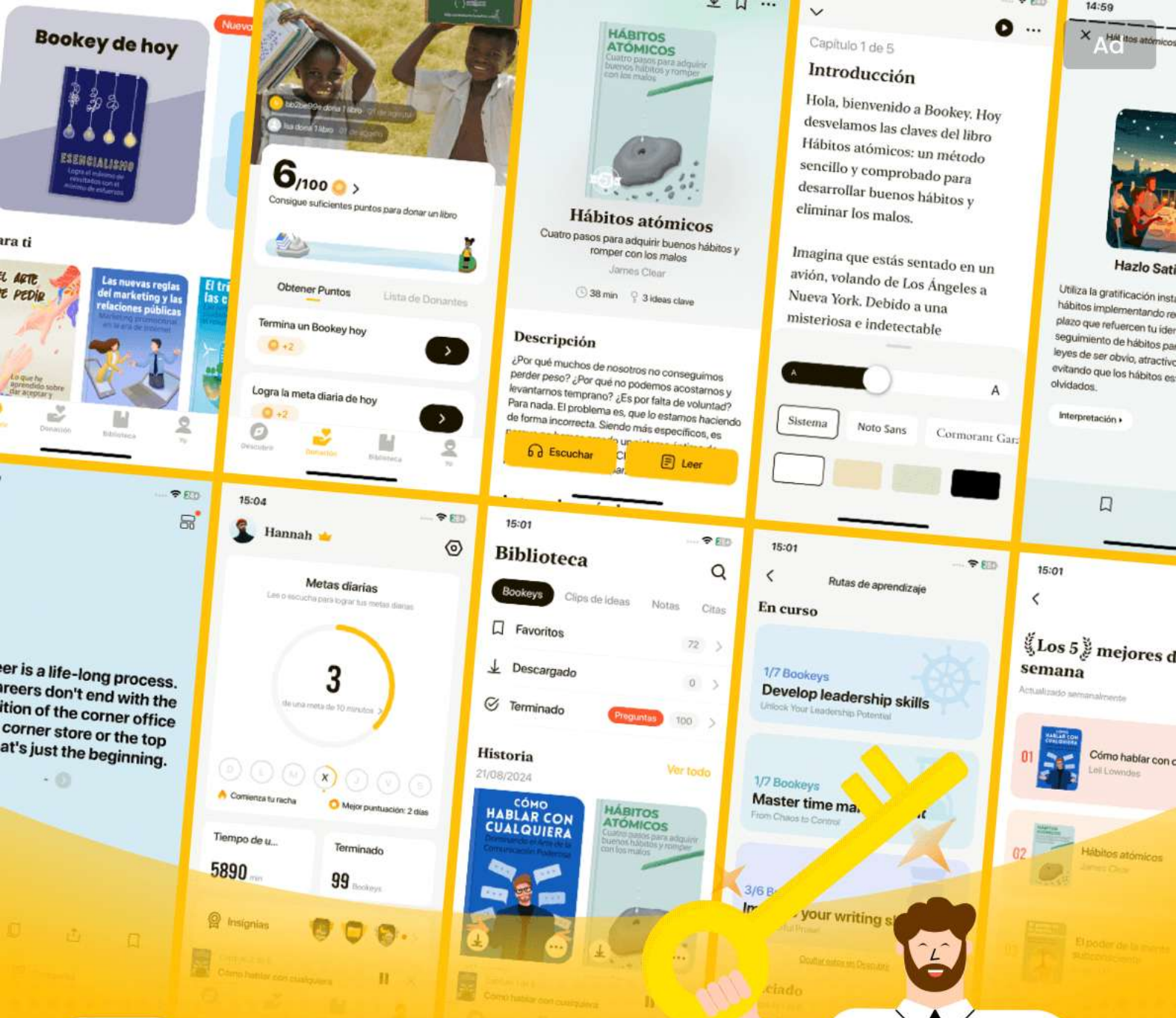
Más Libros Gratis
Escanea para descargar
Escúchalo

8.Pregunta

¿De qué manera las experiencias documentadas en los proyectos contribuyeron a la filosofía de Robert C. Martin sobre la Arquitectura Limpia?

Respuesta:Las experiencias citadas en los proyectos moldearon profundamente la filosofía de Robert C. Martin sobre la Arquitectura Limpia al enfatizar la importancia de límites claros, mantenibilidad, aislamiento de la lógica de negocio de las preocupaciones de infraestructura, y la priorización de las necesidades del usuario durante el diseño. Estas lecciones fomentaron la comprensión de que un software bien estructurado es integral para lograr resultados exitosos a largo plazo en producción.

Más Libros Gratis
Escanea para descargar
Escúchalo



Las mejores ideas del mundo desbloquean tu potencial

Prueba gratuita con Bookey

Escanear para descargar

Consíguelo en el App Store

DESCARGAR EN Google Play

Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software. Cuestionario y prueba

Ver la respuesta correcta en el sitio web de Bookey

Capítulo 1 | ¿Qué es el diseño y la arquitectura?| Cuestionario y prueba

1. El diseño y la arquitectura en el desarrollo de software son esencialmente lo mismo y trabajan juntos como un todo.
2. Un buen diseño de software debería aumentar los recursos humanos necesarios para construir y mantener el sistema.
3. Codificar rápidamente sin considerar la calidad conduce al éxito a largo plazo en los proyectos de software.

Capítulo 2 | Una Historia de Dos Valores| Cuestionario y prueba

1. El valor principal del software es su comportamiento, que incluye cumplir con los requisitos de los interesados y solucionar errores.
2. La integridad arquitectónica a menudo se prioriza sobre características urgentes según la matriz de Eisenhower.

Más Libros Gratis
Escanea para descargar
Escúchalo

3.Los desarrolladores deben abogar por una arquitectura sólida para evitar complicaciones y el aumento de costos en el futuro.

Capítulo 3 | Visión General de los Paradigmas| Cuestionario y prueba

1.La programación estructurada fue popularizada por Edsger W. Dijkstra en 1968, enfatizando el uso de las instrucciones goto.

2.La programación orientada a objetos fue desarrollada por Ole Johan Dahl y Kristen Nygaard en 1966, permitiendo que las variables de función locales persistan.

3.La programación funcional tiene sus raíces en la invención del »-cálculo por Alonzo Church, que se 1940.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 4 | Programación Estructurada| Cuestionario y prueba

1. Edsger Dijkstra fue el primer programador en los Países Bajos.
2. Dijkstra creía que el uso de sentencias 'goto' era beneficioso para las tareas de programación.
3. La programación estructurada enfatiza la producción de funciones complejas y no testables.

Capítulo 5 | Programación Orientada a Objetos| Cuestionario y prueba

1. La programación orientada a objetos (POO) integra datos y funciones.
2. La encapsulación en los lenguajes de POO oculta los datos completamente del acceso externo y no permite que ninguna función externa acceda a ellos.
3. El polimorfismo permite que las funciones se comporten de manera diferente según el tipo de entrada y puede promover una arquitectura de plugins.

Capítulo 6 | Programación Funcional| Cuestionario y prueba

Más Libros Gratis
Escanea para descargar
Escúchalo

1. Los conceptos de programación funcional son anteriores a la propia programación, ya que están arraigados en el cálculo » inventado por Church en la década de 1930.
2. Los lenguajes de programación primitiva utilizan universalmente variables mutables, lo que las hace cruciales para el desarrollo de software.
3. El Event sourcing permite almacenar el estado actual de una aplicación, minimizando problemas de concurrencia.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 7 | SRP: El Principio de Responsabilidad Única| Cuestionario y prueba

1. El Principio de Responsabilidad Única (SRP) establece que un módulo debe tener solo una razón para cambiar.
2. Según el SRP, cada módulo en una arquitectura de software debería realizar únicamente una función sin relación con ninguna parte interesada.
3. Violar el SRP puede conducir a complicaciones durante la integración del código, especialmente cuando varios desarrolladores modifican una clase que sirve a diferentes actores.

Capítulo 8 | OCP: El Principio Abierto-Cerrado| Cuestionario y prueba

1. El Principio de Abierto-Cerrado (OCP) establece que un artefacto de software debería ser cerrado para extensión pero abierto a modificación.
2. Una buena arquitectura de software permite que el código existente se cambie mínimamente cuando se añaden nuevas características.

Más Libros Gratis
Escanea para descargar
Escúchalo

3. El funcionamiento interno de los componentes debe ser expuesto para mantener las dependencias necesarias entre ellos.

Capítulo 9 | LSP: El Principio de Sustitución de Liskov| Cuestionario y prueba

1. El principio de sustitución de Liskov (LSP) fue definido por Barbara Liskov en 1988.
2. El problema cuadrado/rectángulo no viola el principio de sustitución de Liskov porque Cuadrado puede ser tratado como un Rectángulo.
3. El LSP solo es relevante para la herencia y no se aplica al diseño de software o a las interfaces.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 10 | ISP: El Principio de Segregación de Interfaces| Cuestionario y prueba

1. El Principio de Segregación de Interfaces (ISP) establece que un cliente debe tener acceso a todos los métodos de una interfaz, incluso si no los utiliza.
2. Al seguir el ISP, los cambios en una parte de un sistema pueden afectar a otras partes, lo que puede provocar recompilaciones y dificultades.
3. En lenguajes de programación con tipado estático, como Java, seguir el ISP es crucial para evitar que dependencias no deseadas afecten a los clientes.

Capítulo 11 | DIP: El Principio de Inversión de Dependencias| Cuestionario y prueba

1. El principio de inversión de dependencias (DIP) fomenta que los sistemas dependan de implementaciones concretas en lugar de abstracciones.
2. Las abstracciones estables son cruciales para construir arquitecturas estables y los cambios en las clases concretas

Más Libros Gratis
Escanea para descargar
Escúchalo

típicamente no requieren modificaciones en las interfaces.

3. Utilizar un patrón de fábrica para instanciar objetos concretos ayuda a prevenir dependencias directas de las clases concretas subyacentes, lo que está alineado con el DIP.

Capítulo 12 | Componentes| Cuestionario y prueba

1. Los componentes son las unidades desplegables más pequeñas en un sistema y pueden variar según el lenguaje de programación.
2. La invención de los binarios relocatables eliminó la necesidad de referencias externas en los programas compilados.
3. La arquitectura de complementos de componentes desarrollada en los años 80 y 90 permite el enlace dinámico de componentes en tiempo de ejecución.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 13 | Cohesión de Componentes| Cuestionario y prueba

1. El principio de equivalencia de reutilización/liberación establece que el grano de reutilización es el grano de liberación.
2. El principio de reutilización común aconseja colocar todas las clases dentro de un único componente, sin importar la dependencia.
3. El principio de cierre común sugiere que las clases que cambian juntas por las mismas razones deben reunirse en el mismo componente.

Capítulo 14 | Acoplamiento de Componentes| Cuestionario y prueba

1. El Principio de Dependencias Acíclicas (ADP) establece que no debe haber ciclos en el gráfico de dependencias de los componentes.
2. Los componentes deben diseñarse para depender en la dirección de la volatilidad en lugar de la estabilidad.
3. Un componente debe ser tan abstracto como estable, según el Principio de Abstracciones Estables.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 15 | ¿Qué es la arquitectura?| Cuestionario y prueba

1. Un arquitecto de software no es más que un gerente de proyecto que no participa en tareas de programación.
2. Una buena arquitectura de software incluye la definición clara de componentes y su comunicación efectiva.
3. Se considera que el mantenimiento de un sistema de software es el aspecto menos costoso de su ciclo de vida.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 16 | Independencia| Cuestionario y prueba

1. Una buena arquitectura debe soportar los casos de uso y el soporte operativo, entre otros objetivos.
2. La Ley de Conway establece que el diseño de un sistema no refleja la estructura de comunicación de un equipo.
3. Desacoplar componentes ayuda a reducir la interferencia entre equipos y permite un despliegue independiente.

Capítulo 17 | Fronteras: Dibujando Líneas| Cuestionario y prueba

1. La arquitectura de software debe evitar establecer límites entre los componentes para permitir una interacción libre entre ellos.
2. Una arquitectura de plugins permite reemplazar componentes opcionales sin afectar las funciones principales de un sistema.
3. Las decisiones arquitectónicas prematuras pueden simplificar el proceso de desarrollo y llevar a un mejor rendimiento del sistema.

Capítulo 18 | Anatomía de Límites| Cuestionario y

Más Libros Gratis
Escanea para descargar
Escúchalo

prueba

1. La arquitectura de un sistema consiste únicamente en componentes de software y no incluye ningún límite que los separe.
2. Las cruces de límites en la arquitectura del sistema son significativas porque implican la transferencia de datos a través de diferentes módulos, lo que puede llevar a cambios o recompilaciones en esos módulos.
3. Los hilos sirven como límites arquitectónicos por sí mismos, permitiendo la ejecución independiente dentro de un sistema.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 19 | Política y Nivel| Cuestionario y prueba

1. Los sistemas de software son, en esencia, expresiones de políticas que describen cómo se transforman las entradas en salidas.
2. Las políticas de mayor nivel están más cerca de las entradas y salidas y cambian con más frecuencia que las políticas de menor nivel.
3. Separar las políticas de mayor nivel de los detalles de menor nivel aumenta el impacto de los cambios en las políticas de entrada/salida.

Capítulo 20 | Reglas de Negocio| Cuestionario y prueba

1. Las reglas de negocio solo pueden ser aplicables y significativas cuando están automatizadas.
2. Las entidades en Arquitectura Limpia, Guía para especialistas en la estructura y el diseño de software encapsulan reglas de negocio y datos críticos, y permanecen ajenas a preocupaciones externas.
3. Los casos de uso necesitan depender en gran medida de los

Más Libros Gratis
Escanea para descargar
Escúchalo

detalles de la interfaz de usuario para ser efectivos en el orden de ejecución y la accesibilidad.

Capítulo 21 | Arquitectura Gritando| Cuestionario y prueba

1. La arquitectura cumple la misma función que los planos de un edificio, indicando fácilmente el uso previsto de una aplicación de software.
2. Una buena arquitectura se trata únicamente de elegir los marcos adecuados para la implementación.
3. Los marcos deben dictar las decisiones de arquitectura, ya que son esenciales para construir la aplicación.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 22 | La Arquitectura Limpia| Cuestionario y prueba

1. Las capas en Arquitectura Limpia están diseñadas para ser independientes de los frameworks, permitiendo que sean tratadas como herramientas.
2. En Arquitectura Limpia, la capa de entidades cambia con frecuencia según el contexto operacional.
3. La Regla de Dependencia establece que las dependencias deben apuntar hacia afuera, hacia los detalles de nivel inferior.

Capítulo 23 | Presentadores y Objetos Humildes| Cuestionario y prueba

1. El patrón de Objeto Humilde divide los comportamientos en clases testeables y no testeables.
2. El Presenter procesa los datos mientras que la Vista es responsable de formatearlos.
3. La testabilidad no es esencial para una buena arquitectura según el patrón de Objeto Humilde.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 24 | Fronteras Parciales| Cuestionario y prueba

1. Los límites arquitectónicos bien definidos son siempre necesarios para un diseño y mantenimiento efectivos.
2. Usar el patrón Facade sacrifica algo de inversión de dependencias pero ayuda a definir límites arquitectónicos.
3. Crear un límite parcial permite que los componentes se implementen como una sola unidad para ahorrar en costos administrativos.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 25 | Capas y Límites| Cuestionario y prueba

1. Los sistemas generalmente se pueden simplificar en tres componentes: Interfaz de Usuario (UI), Reglas de Negocio y Base de Datos.
2. El ejemplo de Cazar al Wumpus demuestra que el componente de Reglas del Juego debe estar estrechamente acoplado con la UI para una funcionalidad efectiva.
3. Una arquitectura efectiva requiere identificar los límites y asegurar que los componentes de nivel superior dependan de los de nivel inferior.

Capítulo 26 | El Componente Principal| Cuestionario y prueba

1. El componente Principal se considera la parte 'más sucia' de un sistema de software porque gestiona dependencias y orquesta el sistema sin exponer funcionalidades de alto nivel.
2. El componente Principal debería manejar funcionalidades de alto nivel directamente para asegurar una arquitectura limpia.

Más Libros Gratis
Escanea para descargar
Escúchalo

3. En un sistema de software, el componente Principal no juega un papel significativo en el mantenimiento de la limpieza y la organización.

Capítulo 27 | Servicios: Grandes y Pequeños| Cuestionario y prueba

1. Las arquitecturas orientadas a servicios y microservicios definen inherentemente una arquitectura por sí solas.
2. Los servicios pueden estar fuertemente acoplados a través de datos e interfaces compartidos a pesar de ejecutarse en diferentes procesos.
3. Una gestión adecuada de los componentes permite una mejor separación de preocupaciones en la arquitectura de servicios.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 28 | El Límite de las Pruebas| Cuestionario y prueba

1. Las pruebas se consideran componentes integrales de la arquitectura de un sistema, al igual que cualquier otra parte.
2. Diseñar pruebas de manera aislada del diseño del sistema es beneficioso, ya que conduce a una mayor flexibilidad.
3. Una API de pruebas especializada puede mejorar la independencia de las pruebas al permitir que éstas sorteen ciertas restricciones de la aplicación.

Capítulo 29 | Arquitectura Embebida Limpia| Cuestionario y prueba

1. El firmware se define como software que puede volverse obsoleto debido a las dependencias de hardware.
2. Los principios de Arquitectura Limpia sugieren que el software embebido debe estar estrechamente acoplado con el sistema operativo para mejorar su funcionalidad.
3. La Capa de Abstracción de Hardware (HAL) se introduce para aumentar las dependencias que pueden complicar el

Más Libros Gratis
Escanea para descargar
Escúchalo

software embebido.

Capítulo 30 | La Base de Datos Es un Detalle| Cuestionario y prueba

1. La base de datos debe considerarse una parte fundamental de la arquitectura del software.
2. Las bases de datos relacionales impactan significativamente en las decisiones arquitectónicas de las aplicaciones de software.
3. Las consideraciones de rendimiento deben influir en la arquitectura general del software.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 31 | La Web Es un Detalle| Cuestionario y prueba

1. La web representa un cambio fundamental en la tecnología más que un simple detalle en la arquitectura del sistema.
2. Desacoplar la lógica de negocio de las interfaces gráficas es esencial para proteger la arquitectura de influencias de marketing.
3. La independencia de dispositivos no debería ser una preocupación para los arquitectos al diseñar sistemas que utilizan interfaces web.

Capítulo 32 | Los Frameworks Son Detalles| Cuestionario y prueba

1. Los frameworks son equivalentes a la arquitectura de software.
2. El compromiso con un framework suele ser unilateral, con los usuarios invirtiendo mucho y sin recibir garantías.
3. Se recomienda acoplar estrechamente tu aplicación a un framework para lograr una mejor interacción.

Más Libros Gratis
Escanea para descargar
Escúchalo

Capítulo 33 | Estudio de Caso: Ventas de Videos| Cuestionario y prueba

1. La arquitectura del estudio de caso de ventas de video permite que los cambios en los componentes de bajo nivel impacten en las políticas de alto nivel.
2. Identificar actores y sus casos de uso es esencial para asegurar que los cambios de un actor no afecten a otros en la plataforma de ventas de video.
3. La arquitectura de componentes en el estudio de caso sugiere que cada componente corresponde a múltiples actores para maximizar la eficiencia.

Más Libros Gratis
Escanea para descargar
Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play

Capítulo 34 | El Capítulo Faltante| Cuestionario y prueba

1. El capítulo argumenta que el enfoque tradicional de 'Paquete por Capa' es siempre la mejor opción para el diseño de software.
2. 'Paquete por Funcionalidad' permite una mejor visibilidad del contexto empresarial en comparación con 'Paquete por Capa'.
3. Prácticas de implementación adecuadas, como controlar estrictamente los modificadores de acceso, pueden ayudar a mantener la integridad arquitectónica.

Capítulo 35 | Apéndice A Arqueología de la Arquitectura| Cuestionario y prueba

1. El capítulo discute el desarrollo de un sistema de monitoreo en tiempo real para fundición a presión de aluminio utilizando el System/7 de IBM, sin bibliotecas ni sistemas operativos.
2. La arquitectura desacoplada mejora los tiempos de respuesta y la eficiencia de la memoria, como se ilustra con la transición del sistema 4-TEL.

Más Libros Gratis
Escanea para descargar
Escúchalo

3.El capítulo sugiere que la arquitectura del software no necesita adaptarse a requisitos y entornos específicos.

Más Libros Gratis

Escanea para descargar

Escúchalo

Escanear para descargar

Descarga la app Bookey para disfrutar

Más de 1000 resúmenes de libros con cuestionarios

¡Prueba gratuita disponible!

Consíguelo en el App Store

DESCARGAR EN Google Play